PROJECT PLAN · DRAFT FOR YOUR REVIEW · NOT YET APPROVED

ebook-ab-platform

Plan v4.0: A/B testing for a fictional Latin American ebook marketplace, on Databricks Free Edition and AWS's always-free services

Antes de decidir · Before You Decide: free tools where writers ask real readers before they decide. A Duel between two covers or titles, an Ad Doctor for ad reports, and Real or luck? for sales. Behind them, a $0 data platform on two clouds: real votes in DynamoDB and a simulated store on Kafka → files in Amazon S3 → a Lakeflow pipeline on Databricks Free Edition builds a Delta Lake lakehouse (Bronze / Silver / 20 Gold views) governed by Unity Catalog → honest verdicts and a public site on CloudFront.

Date	**2026-10-19**
Prepared for	Pipe199x (github.com/Pipe199x)
Prepared by	Claude Code (Claude Opus 5.5), acting as technical lead
Status	Draft v4.0 for review: the only current version. It replaces v3.0. Nothing gets built until you approve. Then Phase 0 starts only when you say so.
What changed	Your decisions of 2026-10-19: AWS; serverless only; $0; Delta tables; keeping the simulated store with Kafka; and keeping Databricks (the hybrid, Q23). Databricks Free Edition runs the lakehouse; AWS runs the Duel service and the site, using only services with a monthly free allowance; the one exception, S3 requests, costs cents and is paid from the sign-up credits in year 1. Plus a new §3.0: how to answer "how did you use X, and why was it necessary?" in an interview.
Evidence	Every AWS and Databricks limit and allowance cites an official page, checked on 2026-10-19.
Demo	`docs/plan/DEMO.html` : the product screens are unchanged. Its For nerds tab still shows v3.0 and is redrawn after approval (Section 0, step 3).

How to read this plan

- **Part 1 · The story in pictures** (sections A–G, about 10 minutes): what we build, what you'll have at the end, where the numbers come from, the journey of one purchase and one vote, why small sellers struggle, who does what, and what it costs.
- **Part 2 · Technical plan** (sections 1–9): (1) summary and scope · (2) decisions log · (3) architecture, starting with **§3.0, your interview answers for every technology** · (4) bill of materials with every free allowance · (5) phases · (6) risk register · (7) the 20 Gold tables · (8) pre-registration drafts · (9) assumptions and open questions.
- **Context** and **Section 0** explain what is new in v4.0 and what approving it authorizes.
- In a hurry? Read **Context**, **G. What it costs**, **§2.10 Round 7** and **§3.0**.

Contents

Context . . . 5
0. Approving v4.0: what it authorizes, then STOP . . . 7
A. What we're building . . . 9
B. What you'll have at the end . . . 11
C. Where the numbers come from . . . 14
D. The journey of one purchase, and of one vote . . . 15
E. Why small sellers struggle with A/B tests . . . 16
F. Who does what, and when . . . 17
G. What it costs: $0 by design, with one honest footnote . . . 18
1. Executive summary and scope . . . 21
1.1 Summary . . . 21
1.2 In scope . . . 22
1.3 Out of scope . . . 22
1.4 Additions and deviations from the brief . . . 23
2. Decisions log . . . 25
2.1 Round 1 (plan v1 interview, 2026-10-15) . . . 25
2.2 Your 8 points on v1, and what changed . . . 25
2.3 Round 2 (2026-10-15) . . . 25
2.4 Verified instead of asked . . . 26
2.5 Lead decisions D1–D29 (my calls; override them at review) . . . 26
2.6 Round 3 (plan v2.1, 2026-10-15) . . . 27
2.7 Round 4 (plan v2.2, 2026-10-15) . . . 28
2.8 Round 5 (plan v2.3, 2026-10-15) . . . 28
2.9 Round 6 (plan v3.0, 2026-10-17) . . . 28
2.10 Round 7 (plan v4.0, 2026-10-19) . . . 29
2.11 Lead decisions added in v4.0 (my calls; override them at review) . . . 30
3. Architecture . . . 33
3.0 How you used each technology, and why it was necessary (interview answers) . . . 33
3.1 Ingestion: Kafka → S3 → volume → Auto Loader . . . 41
3.2 Storage: S3 for the raw history, Unity Catalog for the tables . . . 42
3.3 The bridge's commit protocol: why at-least-once plus deterministic names gives zero Bronze duplicates . . . 43
3.4 Phase 0 decision rules (feasibility at $0, on both platforms) . . . 43
3.5 Topics, keys, contract . . . 44
3.6 Datasets by layer . . . 45
3.7 Offsets and consumer groups . . . 45
3.8 Ingestion options (justified) . . . 46
3.9 Silver design . . . 46
3.10 Gold factory . . . 47
3.11 Analysis method (notebooks `10_ab_analysis`, `11_validation`; library `ebook_ab.stats`) . . . 47
3.12 Real-data track (Q9) . . . 48
3.13 Website, the writer tools and the Duel service . . . 49
3.14 Orchestration: the daily run . . . 52
3.15 Security and access . . . 53
3.16 Observability, performance, Delta . . . 53
3.17 CI/CD . . . 54
3.18 Compliance and ethics by design . . . 54
3.19 Sizing and throughput . . . 55
4.1 Rejected options . . . 58
5. Phases . . . 60
Phase 0: Feasibility spike (12–16 h) . . . 60
Phase 1: Lifesaver launch (40–56 h) . . . 61
Phase 2: Simulator + contracts + real catalog (13–17 h) . . . 62
Phase 3: Consumer groups + S3 landing + sync + Bronze (10–14 h) . . . 62
Phase 4: Silver, with Duel votes (14–19 h) . . . 63
Phase 5: Gold factory, 20 materialized views (9–12 h) . . . 63

Phase 6: Analysis, validation, real-data lessons, the library (18–25 h) 63
Phase 7: More tools and pages (50–70 h) 64
Phase 8: The daily run, maintenance and alarms (16–22 h) 64
Phase 9: Lab on credits: Glue Spark UI lab, Delta demos (4–6 h) 64
Phase 10: Docs, report and interview prep (6–11 h) 65
7. Gold catalog: draft of the 20 views 69
8. Pre-registration drafts 71
9. Assumptions and open questions 73
Appendix A. Repo tree (target) 75
Appendix B. Deliverables → phase 76
Appendix C. Changelog 77
Verification (end-to-end) 78

This is an A/B testing platform with free tools that save writers money and worry. Writers use **Antes de decidir · Before You Decide**: a free Duel that shows two covers (or two titles, blurbs or book ideas) to real readers, an Ad Doctor that says what to stop paying for in their ads, and Real or luck?, which says whether a change in sales is real. With the writer's OK, each Duel result joins a public library of what worked for writers like them. Behind the screen, a simulated marketplace and the real Duel votes land in Amazon S3 and flow into a Delta Lake lakehouse on Databricks Free Edition, built every morning by a Lakeflow pipeline and governed by Unity Catalog, where statistics separate what is real from what is luck. The product runs on AWS, inside its monthly free allowances.

Context

What's new in v4.0. This plan replaces v3.0 and every earlier version. The product does not change: the same Duel, game, Ad Doctor, Real or luck?, library and pages. Underneath, it becomes a hybrid, shaped by your decisions of 2026-10-19 (§2.10):

- **Databricks stays for the data** (Q23): Databricks Free Edition, serverless and $0, runs the lakehouse as in v3.0: one Lakeflow pipeline with Auto Loader, expectations, AUTO CDC and 20 materialized views, Unity Catalog for grants, masks, tags and lineage, and a daily Job with the notebooks.
- **AWS for the product** (decision 1) and **$0** (your follow-up): the Duel service and the site move to AWS, using only services with a monthly free allowance: Lambda function URLs behind CloudFront, DynamoDB on its free provisioned capacity, covers in S3, and a Step Functions state machine that runs each Duel's life (it closes the Duel on day 14 and deletes its covers 30 days later). Amazon S3 also becomes the durable landing zone and replay log.
- **The seam between them is files** (D62): Free Edition only reaches trusted domains, so the daily GitHub run lands files in S3 first and then pushes only the new ones into a Unity Catalog volume, where Auto Loader reads them.
- **Serverless only** (decision 2): no cluster anywhere. The local Docker Spark cluster, the job-cluster target and the plain Structured Streaming track are removed. Free Edition has no Spark UI, so a small Lab on AWS Glue, paid by the sign-up credits, covers it.
- **Delta tables** (your decision 4): every Bronze, Silver and Gold table is Delta Lake, as Unity Catalog managed tables.
- **Kafka stays with the simulated store** (Q22), and **where it is free** (Q18): the store is an invisible simulator, and Kafka's only real job. It runs in GitHub Actions and on your laptop; MSK and Kinesis aren't on the AWS Free plan, and Aiven is no longer needed. Duel votes stay daily files.
- **The one cost AWS won't waive:** S3 requests have no free allowance. We expect a few cents a month, and the sign-up credits pay for them in year 1 (section G). Free Edition has no payment method, so it can't bill.
- **One rule from AWS:** the Free plan closes the account after 6 months or when the credits run out, and CloudFront's $0 plan isn't offered to Free plan accounts. So you upgrade to the paid plan right before the public launch. The upgrade itself costs nothing (D54).
- **§3.0 is new:** for each technology, the question an interviewer asks, a 30-second answer built on a real example from this project, the follow-up, and where the proof sits in the repo.
- **Effort:** about 230 h (192–268), 12–14 weeks at 20 h/week (§5).

How we got here (details in Appendix C):

Version	Date	What changed
v1	2026-10-15	A fintech A/B platform: Kafka → Databricks, three experiments
v2.0	2026-10-15	Your 8 points: ebook marketplace, real data, React site, local job cluster, a daily run without the laptop
v2.1	2026-10-15	A writer-first ad-test tool with plain verdicts
v2.2	2026-10-15	Before You Decide: a community library and opt-in anonymous sharing
v2.3	2026-10-15	Your picks C1–C12: source badges, a designed launch day, plainer pages, 7-day ad tests
v3.0	2026-10-17	The lifesaver approach (L1–L9, L11), For nerds, the real use of each piece, one plan
v4.0	2026-10-19	Hybrid at $0: the Delta lakehouse on Databricks Free Edition; the Duel service and site on AWS's free allowances (Lambda, DynamoDB, Step Functions, S3, CloudFront); S3 as the replay log; serverless only; interview answers in §3.0

Starting point (from v3.0, verified 2026-10-17, read-only):

- Windows 10 + WSL2 Ubuntu 26.04, 7.7 GiB RAM, 8 vCPU, OpenJDK 25, gh logged in as Pipe199x.
- The PDF toolchain lives in WSL user space: pandoc, Typst, Node and mermaid-cli with Chrome headless shell.
- The public repo `Pipe199x/ebook-ab-platform` and its GitHub Pages landing page already exist (`pipe199x.github.io/ebook-ab-platform`, checked 2026-10-19). The landing page's "Ask" section still shows the placeholders "[amount]" and "[use of funds]": fill them in or remove the section.
- You already have a Databricks Free Edition workspace (Q1). There is no AWS account yet: you will sign up for a new account on the Free plan (Q20).

Intended outcome: this PDF. You review it, then ask for changes or say “start Phase 0”.

0. Approving v4.0: what it authorizes, then STOP

Approving v4.0 authorizes only steps 1–5 below. It does not start Phase 0. Until you say “start Phase 0”: no changes to the repo or the landing page, no AWS sign-up, and no AWS or Databricks actions.

1. **Write the plan.**
   - The source lives in the repo: `docs/plan/render/plan-source.md`.
   - `tools/make_plan_md.py` turns it into `docs/plan/PLAN.md`; the Mermaid blocks and the screen grid become images.
2. **Tidy `docs/plan/`.** v3.0 moves to `docs/plan/old/` with plans v1–v2.3. `docs/plan/README.md` lists what to read, what to try and what is old.
3. **Bring the demo in line with v4.0.**
   - Its For nerds tab shows the hybrid map of section B (Figure 4): Kafka in GitHub Actions; S3, Lambda, DynamoDB, Step Functions and CloudFront on AWS; the volume, Auto Loader, the Lakeflow pipeline, Delta and Unity Catalog on Databricks.
   - The stack table uses the verdicts of §3.0, and the interview answers become the cards’ “Why” lines.
   - `tools/build_companion.py` builds it, and `tools/check_demo.cjs` clicks through every screen.
4. **Render and publish the PDF** with the same pandoc → Typst pipeline (`tools/render_docs.sh`): ten Mermaid diagrams, one chart, six phone screens of the demo.
5. **STOP.**

Part 1 · The story in pictures

Part 1 is the plain-language version: what we build, why, what you get, who does what, and what it costs. Part 2 holds the technical detail behind every claim.

A. What we're building

> **In one sentence:** free tools that answer a writer's real questions in minutes (which cover, what to stop paying for, real or luck), with A/B testing as the engine underneath, and a $0 data platform on Databricks Free Edition and AWS's free allowances that proves the math every day.

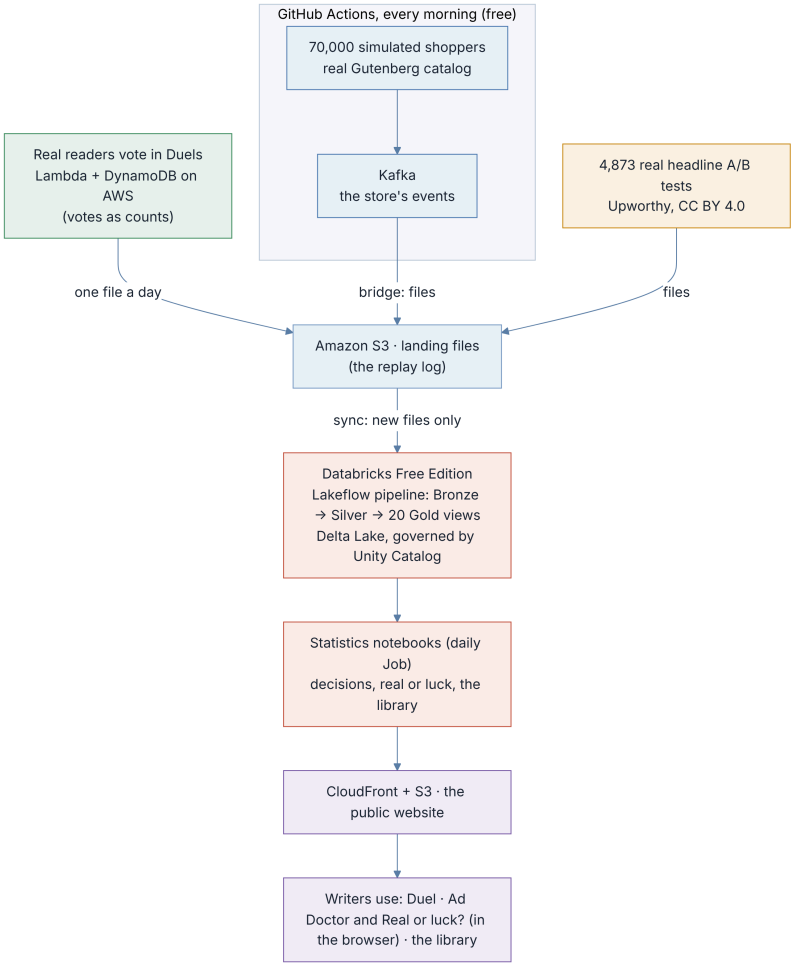
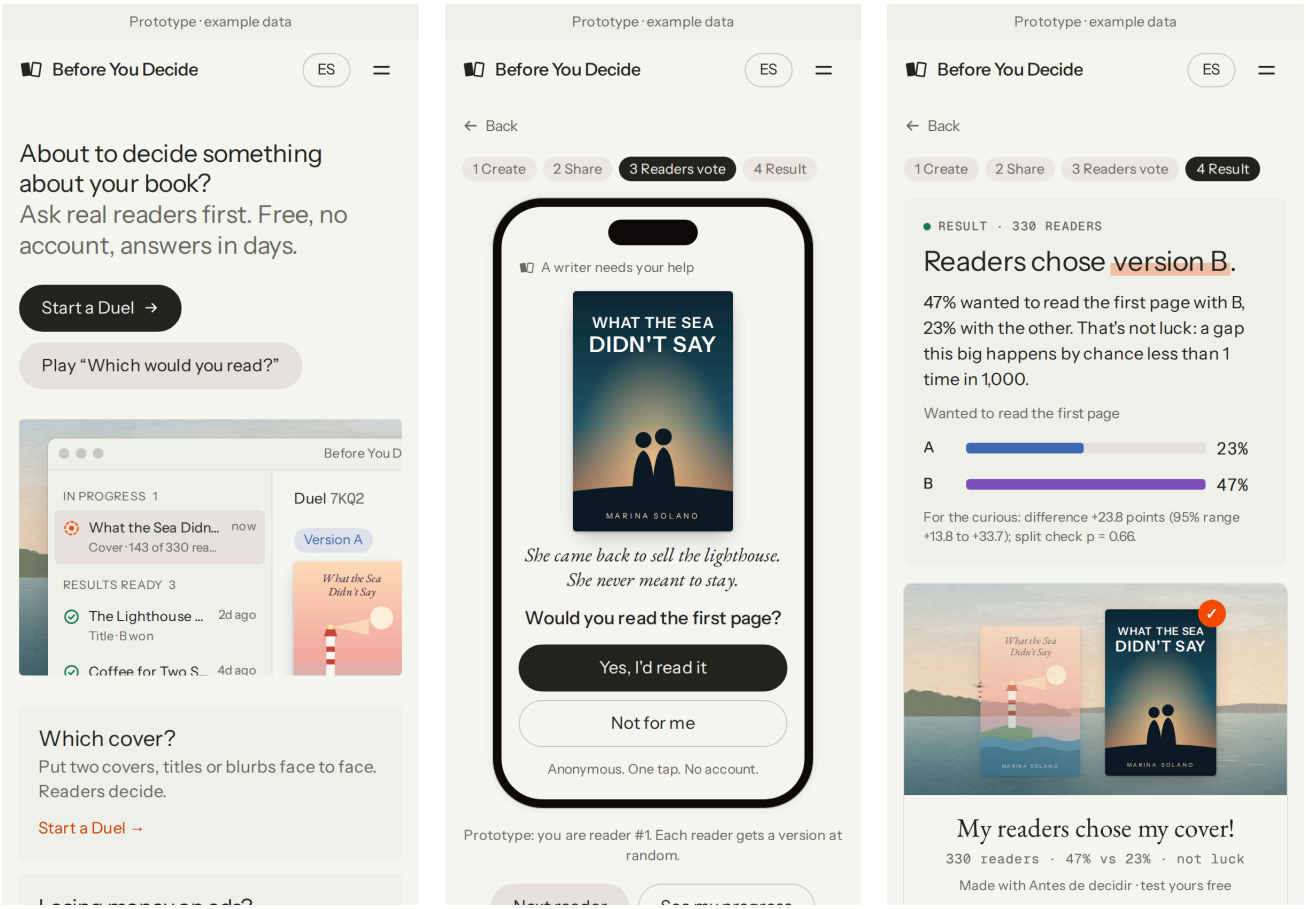


Figure 1: The big picture: what we build and who benefits. Kafka touches only the simulated store; real votes and open data arrive as files.

- **The problem.** Writers decide on covers, titles, blurbs and ads with little to go on.
  - A poll among friends is not a fair test.
  - Paid panels cost about $1 per answer.
  - An ad test costs money and a week.
  - When sales move after a change, nobody can tell a real effect from luck (section E).
- **What we build:**
  1. **Antes de decidir · Before You Decide**, free, in Spanish and English, with no account:
     - **Duel**: two versions, one link, real readers, an honest result in days;
     - **Which would you read?**: a game that brings readers to Duels;
     - **Ad Doctor**: drop your Amazon Ads report and see what to stop, keep, wait on or treat as luck. The file never leaves the browser;
     - **Real or luck?**: two sales numbers and an honest answer;
     - **the library**: what worked for writers like you, pooled from Duels and shared ad tests, once a group has 5 results;
     - inside: the ad-test planner and reader, *Real lessons*, and *Behind the scenes: our test store*.
  2. **The platform, at $0 on two clouds.**
     - Simulated shoppers buy real public-domain titles, and Kafka carries their events inside the daily GitHub Actions run.
     - Real Duel votes, stored in DynamoDB on AWS, leave once a day as one file.

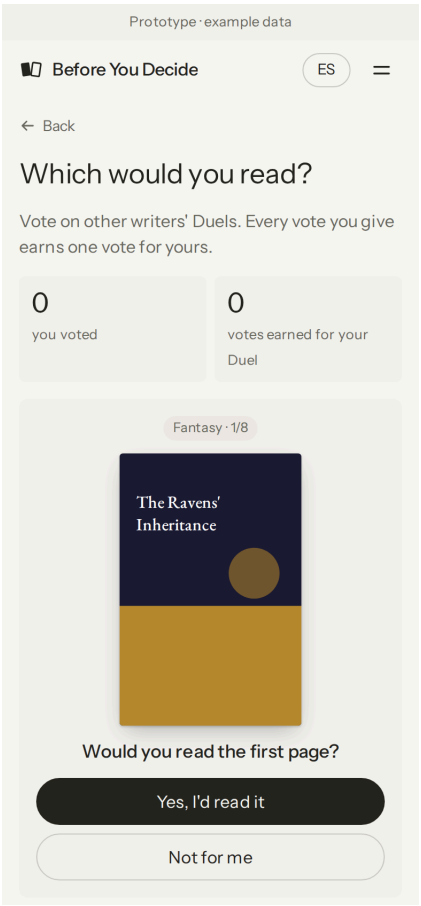
    - Every file lands in Amazon S3 first, then moves into Databricks Free Edition, where a Lakeflow pipeline cleans everything into Delta tables and the statistics notebooks decide.
    - The site, on AWS, refreshes every day without the laptop.
  3. **Lessons from real experiments.** 4,873 real headline A/B tests give realistic expectations and three plain lessons.
  4. **For nerds**: the architecture map and the stack, for recruiters and developers.
- **Who benefits:**
  - writers get answers for free and see what they saved;
  - readers discover books and get first pages;
  - you get a product people use and a portfolio of end-to-end data engineering on Databricks and AWS, and experimentation, with an answer ready for every technology on it (§3.0).

B. What you'll have at the end

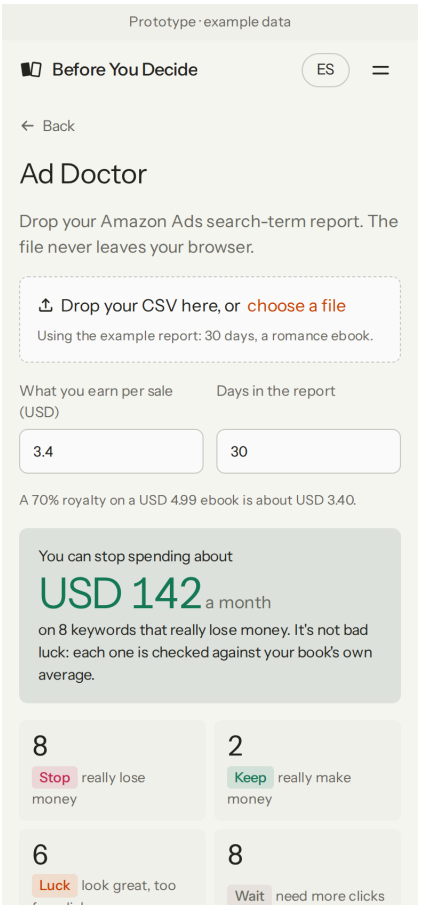


Home: three doors, one per pain | A Duel: one version, one tap | The reveal and a post to share

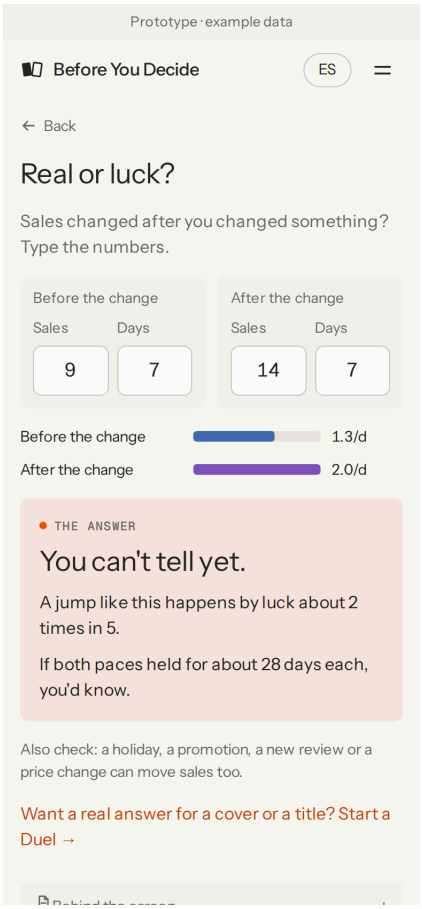
Figure 2: The writer's screens are the same as in v3.0 (the demo; example data). Only the machinery behind them changed.



Which would you read? Vote to earn votes



Ad Doctor: what to stop paying for



Real or luck? Two numbers, an honest answer

Figure 3: The game, the Ad Doctor and Real or luck? (the demo; example data).

1. **A public website** behind Amazon CloudFront (an address under `cloudfront.net`, or your own domain later):
   - home: three doors (a Duel, the Ad Doctor, Real or luck?);
   - the Duel: create it in 30 seconds, share one link, readers vote, the writer follows progress without seeing the winner, then the reveal and a ready post for social media;
   - *Which would you read?*, the game;
   - the library: what worked for writers like you. On launch day it is empty, and it says so (section C);
   - inside: the ad-test planner and reader, *Real lessons*, *Behind the scenes: our test store*, and *For nerds*
   - on every page, a badge on each number says where it comes from.
2. **A serverless Duel service.** A Lambda function behind CloudFront receives Duels, votes and opt-in results; DynamoDB stores them as counts; covers live in S3; a Step Functions state machine closes each Duel on time and deletes its covers 30 days after the end.
3. **A daily run that needs no laptop.** One GitHub Actions workflow, every morning:
   - simulates one day of shopping through Kafka and lands the files in S3;
   - asks a Lambda to copy yesterday's Duel votes and shared results out of DynamoDB as files;
   - pushes the new files into Databricks and starts the daily Job there: the Lakeflow pipeline (Bronze → Silver → Gold) with data-quality expectations, then the statistics notebooks;
   - publishes the site's data, library included, to S3 behind CloudFront.
4. **A governed lakehouse.** Bronze, Silver and 20 Gold views in Delta Lake on Databricks, governed by Unity Catalog: grants, column masks, personal-data tags, lineage, hashed keys and masked display columns. On AWS, least-privilege IAM roles.
5. **Proof the statistics are right:**
   - the analysis recovers the effects we planted in the simulated store;
   - fake A/B splits give false alarms at the expected 5% rate;
   - a broken split is caught, in the store and in every Duel.
6. **Lab (practice for the portfolio, clearly labelled):** a Spark performance lab on AWS Glue, paid by the sign-up credits, to use the Spark UI that Free Edition lacks; and Delta maintenance and rollback demos.

7. **A public repo** with tests and green CI, infrastructure as code (AWS CDK and Databricks bundles), a runbook, design decisions (ADRs), one-page decision memos, a README a recruiter can read in 5 minutes, and your interview answers (§3.0).

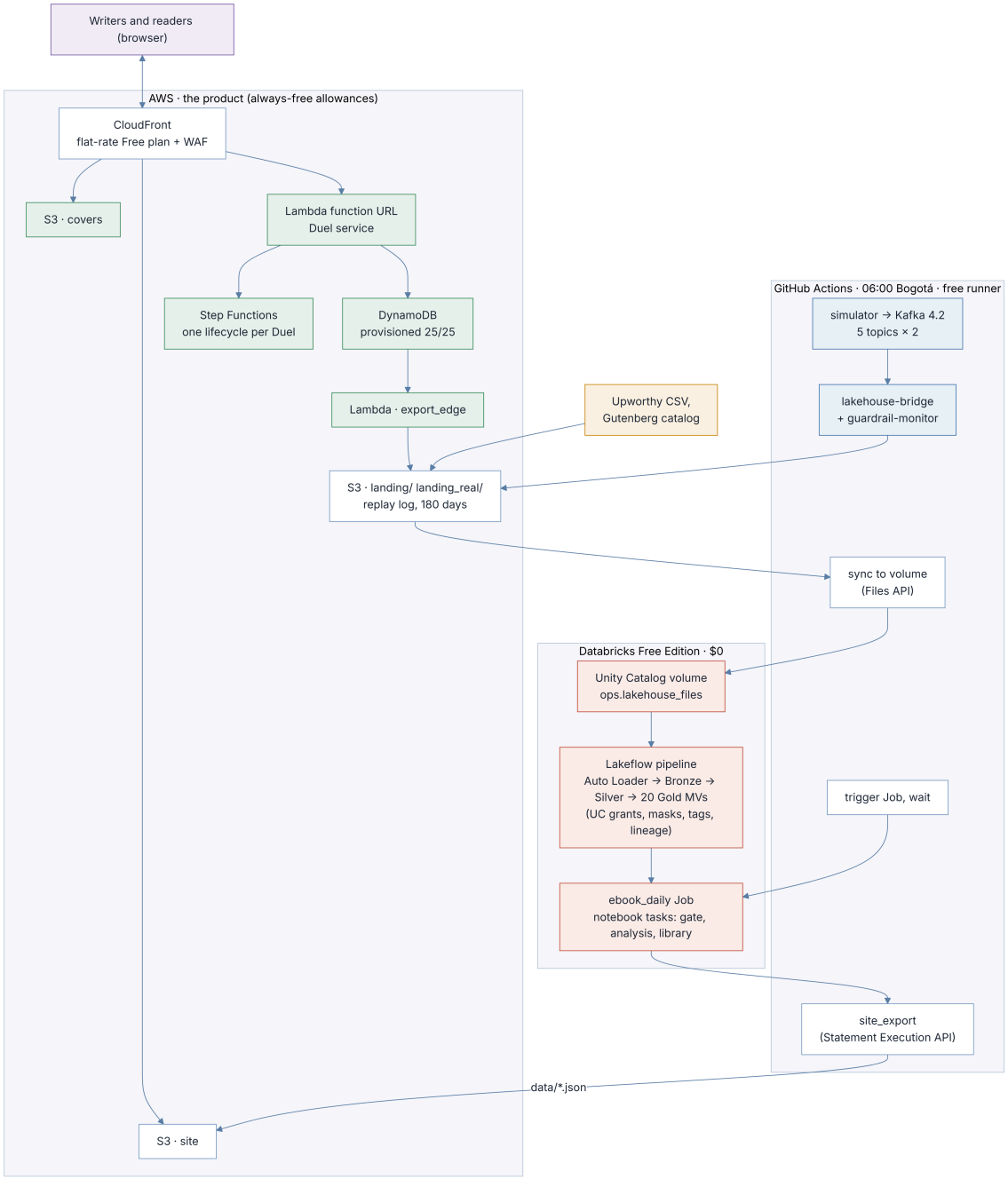


Figure 4: The hybrid architecture map, which replaces the v3.0 For nerds map: GitHub Actions, AWS and Databricks Free Edition. Three kinds of data in three lanes: the simulated store (blue), real Duel votes (green) and open data (amber).

C. Where the numbers come from

> **In one sentence:** every number on the site wears a badge that says where it comes from, and numbers from real people appear only when a group has 5 results.

Badge	What it is	Where you see it	On launch day
Shared by writers	Real results from real people: Duels that readers voted on, and ad tests writers already ran. The writer chose to add each one.	The library, and the "compared with writers like you" lines	Empty. It fills as Duels end. Each group needs 5 results.
Real public tests	4,873 real A/B tests of news headlines, published as open data by Upworthy (2013–2015, CC BY 4.0)	*Real lessons*, and the tools' realistic expectations	Ready. Analyzed in Phase 6. News headlines, not book sales.
Simulated	Made up on purpose by our program: 70,000 fictional shoppers in a fictional store	*Behind the scenes: our test store*, and the demo	Running every day. Never mixed into the library (D26).

How a group fills up (for example, *cover changes · romance · Spanish*):

When	Results	What the writer sees
Launch day	○○○○○ 0 of 5	"Not enough results yet: be one of the first." Plus lessons from the real headline tests.
Some weeks later	●●●○○ 3 of 5	The same message, with the counter moving. Each Duel counts.
5 results in	●●●●● 5 of 5	The numbers appear: how often the change won, the typical change, one dot per result.

- The minimum of 5 applies to everything built from real results: each group, each idea ("worked in W of N"), and the "compared with writers like you" lines (D26).
- **What "won" means.**
  - In a Duel: more readers wanted to read the first page, in a fair test.
  - In an ad test: more people clicked.
  - Neither proves more sales, and the card says so.
- The Duel, the Ad Doctor, Real or luck?, the planner and *Real lessons* all work from day 1. None of them needs the library.
- In the demo, all numbers are examples.

Founding writers. To fill the first groups, we invite Spanish-language indie writers to run the first Duels. Writers who already run ads can also type in tests they ran before.

- First goal: 5 results in the most common groups, such as *romance · Spanish · cover*.
- A public counter on the home page shows "N results so far".
- The invitation line: "Ask your readers. Share what you learned. Decide with data, not luck." In Spanish: "Pregúntales a tus lectores. Comparte lo que aprendiste. Decide con datos, no con suerte."
- No analytics, by design. We count Duels and shared results per week.
- The promise: better decisions and fewer guesses, never more sales.

D. The journey of one purchase, and of one vote

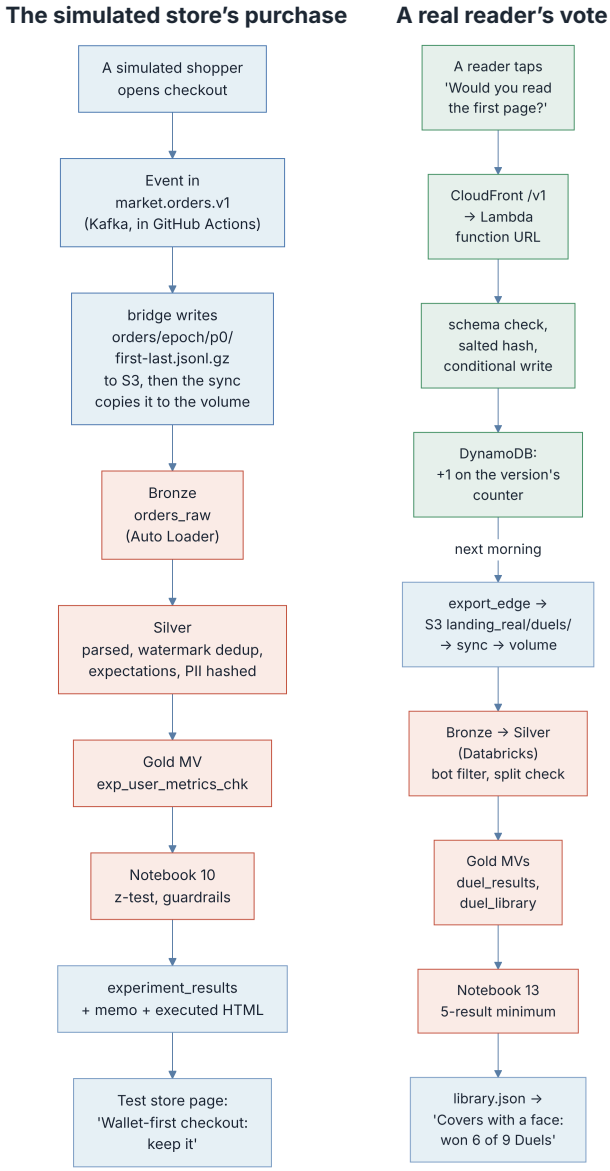


Figure 5: Left: the journey of one purchase in the simulated store, from click to decision. Right: the journey of one Duel vote, from a reader's tap to the library.

- **Kafka is like a post office for the busy store.** It keeps every letter in order, and several readers collect the same letters independently. It opens for the store's day inside the GitHub Actions run and closes when the day is delivered.
- **The votes don't need it.** A few votes a day travel together in one envelope: a daily file out of DynamoDB.
- **S3 is the archive room; Databricks is the office; Delta Lake is the filing system; Unity Catalog is the front desk.** Every file is kept in S3 and a copy goes to Databricks. Delta turns files into tables you can update and roll back, and Unity Catalog decides who may open each drawer.
- **Bronze, Silver, Gold:**
  - Bronze holds the original letters, untouched.
  - Silver holds the checked and cleaned copies, with personal data hidden.
  - Gold holds the summaries people actually read.
- **Rules written before the test** work like a referee who sets the rules before the game. Nobody can move the goalposts after seeing the score.

E. Why small sellers struggle with A/B tests

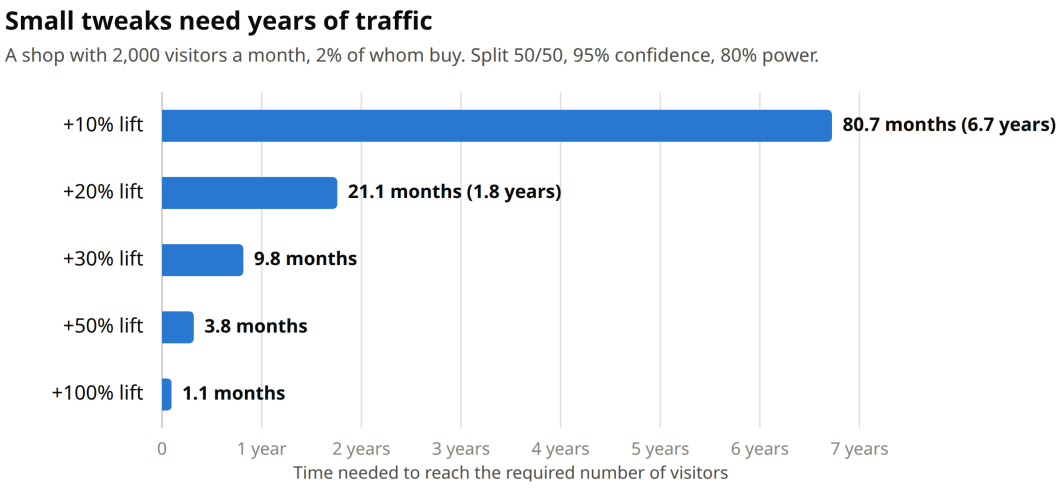


Figure 6: Months of traffic a small seller needs to detect a lift (2,000 visitors/month, 2% buy).

The chart assumes 50/50 split, α = 0.05 two-sided and 80% power. Visitors needed per variant:

Lift	Per variant	Months at 2,000/month
+10%	80,682	80.7
+20%	21,109	21.1
+30%	9,798	9.8
+50%	3,826	3.8
+100%	1,141	1.1

- **What this means:** a small seller can't test small tweaks on a sales page. A "win" after a lucky week is usually noise.
- **What helps:**
  - ask readers directly, with a fair test. In a Duel each reader sees one version and taps once. A bold difference (30% → 45% of readers who want the first page) needs 330 readers, which a writer's followers and the game can bring in days;
  - test in ads, where traffic is fast. Two ads that differ only in the cover need about 16,500 views each to show a bold difference (+30% more clicks): about 4 days at 5,000 views a day per ad. We still run every ad test for a full week, with the same budget spread over the 7 days, because ads behave differently on weekdays and weekends (D29);
  - test big changes: covers, offers, bundles, how the price is presented;
  - treat clicks and votes as a fast first signal, then check sales afterwards (Real or luck? helps);
  - start from realistic expectations taken from real tests, so a lucky streak is not trusted;
  - pool what many writers learned. One writer's test is small, but many Duels show which changes usually win for a genre.

What exists today, and what it lacks (checked 2026-10-16):

Option	Cost	Fair test?	What it lacks
Facebook or Instagram poll	Free	No	Both covers side by side, visible votes, mostly friends
Rate Book Cover (stars from strangers)	Free, paid extras	No	Star ratings of one cover, not two versions; voters aren't your readers
PickFu	About $1 per answer (about $50 for 50 people), plus about $0.40 per answer per audience filter	Panel	Paid per poll; an English-first panel
A/B test in Meta or BookBub ads	Ad money	Yes	Costs money and a week, and needs ad skills
Duel	Free	Yes	Needs voters: the writer's followers plus the game

F. Who does what, and when

When	You do	I do
Now	Read this PDF. Ask for changes or say “start Phase 0”.	Revise and re-render on request
Phase 0	OK the repo changes and fix or remove the landing page’s “Ask” placeholders (S0.1). Sign up for AWS on the Free plan, turn on MFA for the root user, create one admin user for yourself, and log in to Databricks from your laptop, about 25 minutes (S0.2). Run about 6 sudo lines for Docker (local Kafka).	Repo, tools, CDK bootstrap, the GitHub OIDC roles, the $0 budget alarms, the Databricks bundle skeleton, probes on both platforms, evidence
Phase 1	Upgrade the AWS account to the paid plan, then OK the first public deploy of the site and the Duel service (S1.1). Then show the Duel to 5 writers.	The lifesaver launch: site, Duel service, Duel, Real or luck?, home
End of every phase	Run the commands I give and paste the output	Code, tests and docs. Then I wait for your results.
Phase 7	Approve which Duels enter the game, a few minutes a week at first	The game, the Ad Doctor, the library from Duels and the other pages
Phase 8	OK the daily schedule and the daily public refresh (S8.1)	The daily workflow, the Databricks Job, the weekly maintenance, alarms
End	Review the site, the memos and the report. Practise §3.0 aloud. Share the “Founding writers” invitation with writer groups if you want.	Final docs, the launch note and the clean-clone dry run

Accounts.

- **New:** AWS, on the Free plan ($100 in credits at sign-up, up to $100 more by completing activities): `https://portal.aws.amazon.com/billing/signup` . Whether the sign-up asks for a card is noted at S0.2 (O27); the paid plan at S1.1 does need one.
- **Already have:** Databricks Free Edition (it runs the lakehouse; no payment method) and GitHub (Pipe199x).
- **Not needed any more:** Aiven, Cloudflare. Also not needed: Microsoft/Power BI, Docker Hub accounts, npm, PyPI, Vercel, Supabase.

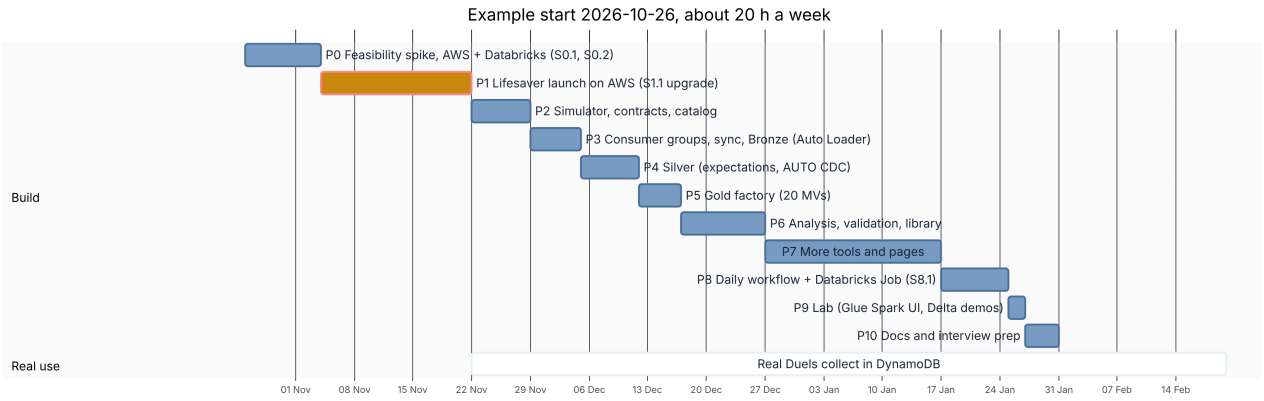


Figure 7: Phases at about 20 h/week (example start date; STOP points add calendar time).

G. What it costs: $0 by design, with one honest footnote

The rule of this design: the data runs on Databricks Free Edition, which has no payment method and can't bill. Nothing runs on AWS unless it has a monthly free allowance that our use stays inside. Glue jobs, Athena, API Gateway, MSK, Kinesis, NAT gateways and anything always-on are kept out of the daily path. A CI check fails any change that adds one of them to a production stack (D53).

The honest footnote. $0 forever is not possible on AWS, by a few cents: S3 requests (each file write and listing) have no free allowance. Our 15,000–60,000 requests a month cost about $0.10–0.30. In year 1 the sign-up credits pay them, so you pay $0. After that, expect a few cents a month, which a $0.01 budget alert reports. Phase 0 measures the real number (O31).

The monthly picture once the site is live (after the S1.1 upgrade; US East; allowances checked 2026-10-19; §4 has the sources):

What	How it is used	Free allowance it stays inside	Cost
Databricks Free Edition	The Lakeflow pipeline, the daily Job and notebooks, Unity Catalog, the SQL warehouse	Free Edition: serverless, a daily fair-use quota, no payment method	$0.00
Amazon S3 storage	Landing files (the replay log), artifacts and covers: under 5 GB	CloudFront's free plan gives S3 storage credits for 5 GB, for any bucket in the account	$0.00
Amazon S3 requests	File writes and listings by the bridge, the sync, the export and the site	None	≈ $0.10–0.30 (credits in year 1)
Data transfer out	The daily sync to Databricks: well under 1 GB a month	100 GB a month out to the internet	$0.00
AWS Lambda	The Duel service, the daily export, small scheduled jobs	1 M requests and 400,000 GB-seconds a month	$0.00
Amazon DynamoDB	Duels, votes, shares; provisioned 25 writes and 25 reads per second	25 GB, 25 WCU and 25 RCU, provisioned mode	$0.00
AWS Step Functions	One Duel lifecycle per Duel (about 6 transitions each)	4,000 transitions a month (≈ 650 Duels)	$0.00
Amazon EventBridge Scheduler	The daily salt rotation	14 M invocations a month	$0.00
Amazon CloudFront	Site, covers and the Duel API from one origin, with basic WAF	Flat-rate Free plan: 1 M requests, 100 GB, no overage charges	$0.00
CloudWatch, SNS, Parameter Store, Budgets, CloudFormation	Logs, alarms, e-mails, secrets, budgets, deploys	Their free tiers, or no charge	$0.00
GitHub Actions	Kafka, the export and sync, triggering the Databricks Job, deploys	Free for public repos (standard runners)	$0.00
Total			**$0.00 in year 1; ≈ $0.10–0.30 a month after**

- **While you build (before S1.1):** the account is on the Free plan, so nothing can be charged; the credits pay for everything, including the Glue Lab (capped at $5 of credits, D61).
- **Why upgrade at all?** On the Free plan, AWS closes the account after 6 months or when the credits run out, and CloudFront's flat-rate plans aren't offered to Free plan accounts. Upgrading costs nothing by itself; remaining credits keep paying the S3 requests (D54).
- **Guardrails (D53):** one budget alerts on any out-of-pocket cost above $0.01; another alerts when gross usage (before credits) passes $1 a month, and at $5 it blocks the pipeline and edge roles; a CI check refuses any paid service in production; DynamoDB on fixed provisioned capacity (it slows down instead of billing more); CloudFront's flat-rate plan has no overage; every Lambda has a concurrency cap; logs expire after 14 days.

What I said no to:

Option	Why not
AWS Glue jobs, daily	No free allowance: $0.44 per DPU-hour (Glue 6.0 30% less). Databricks Free Edition runs the Spark at $0; Glue stays as a Lab on credits (D40, D61).
Amazon Athena	No free allowance: $5 per TB scanned, and the tables live in Databricks. The Free Edition SQL warehouse reads Gold.
Databricks on AWS (paid)	Billed per DBU plus AWS compute after a 14-day trial. Free Edition does our job; paid is the upgrade if the project goes commercial.
Amazon API Gateway	Its 1 M free calls a month last only 12 months. A Lambda function URL behind CloudFront has no extra charge (D49).
Amazon MSK, Kinesis, Firehose	Paid-plan only on the Free plan; MSK Serverless costs $0.75 per cluster-hour, about $548 a month (D52).
AWS Lake Formation	Free, but it guards Athena and Glue, and our tables live in Databricks. Unity Catalog does the job (D45).
DynamoDB on-demand	The free allowance covers provisioned capacity only (D49).
Amazon EMR, Redshift, EC2, NAT gateway	Paid-plan only, or always-on cost (§4.1).
AWS Secrets Manager	$0.40 per secret a month; Parameter Store and the Databricks secret scope are free (D45).
Power BI	You chose React. Sharing would need Pro at $14/user/month.

Part 2 · Technical plan

Sections 1–9 below are the nine contents you asked for in the original brief, rebuilt as a $0 hybrid: Databricks Free Edition for the data, AWS's free allowances for the product.

1. Executive summary and scope

1.1 Summary

For writers and readers (real people). A public React site, served from Amazon S3 through Amazon CloudFront, hosts *Antes de decidir · Before You Decide*, in Spanish and English, with no account. Its home page has three doors:

- **Duel**, the only real experiment.
  - A writer uploads two covers, or two titles, blurbs or book ideas, and shares one link.
  - Each reader sees one version, picked by a hash of the Duel and the visitor, and taps once: "Would you read the first page?"
  - CloudFront passes the vote to an AWS Lambda function URL, which records it in Amazon DynamoDB, as counts only. Covers go straight to a private S3 bucket and are served by CloudFront.
  - The result opens at a target fixed before the start: 330 readers or 14 days. A split check (SRM) and a two-proportion z-test give one of four plain endings (D34). An AWS Step Functions state machine closes the Duel on day 14 if it hasn't ended, and deletes its covers 30 days after the end (D60).
- **Ad Doctor.** It reads an Amazon Ads search-term report in the browser and gives each keyword Stop, Keep, Luck or Wait, measured against the book's own average (a beta-binomial model). Nothing is uploaded.
- **Real or luck?** An exact test on two sales counts, adjusted for the number of days, in the browser.

Around the three doors:

- *Which would you read?*: vote on other writers' Duels to earn votes for yours;
- the library of what worked for writers like you, built from Duel results and shared ad tests (opt-in, 5 results per group);
- the ad-test planner and reader, inside;
- *Real lessons*
- *Behind the scenes: our test store*
- *For nerds*: the architecture map and the stack.

The platform, behind the screen. A simulator built on Faker `es_CO` and `confluent-kafka` plays 70,000 synthetic shoppers over 92 simulated days (2026-06-22 → 2026-09-21). They shop a fictional Latin American ebook marketplace whose catalog is real: about 2,000 public-domain titles from Project Gutenberg's offline catalog (CC0 metadata), with demand weighted by real 30-day download counts. The store has no website: it exists only as this simulator and as one inside page that shows its decisions.

Three concurrent experiments run in the store. The simulator injects known true effects and scheduled faults:

- EXP-CHK-001, wallet-first checkout;
- EXP-PDP-002, new cover + subtitle;
- EXP-BUN-003, a 3-book bundle banner.

Every morning, one GitHub Actions workflow (free for public repos) runs the part that needs Kafka and moves the files (D46):

1. a Kafka 4.2 service container starts; the simulator plays one day into 5 topics × 2 partitions; two consumer groups read it: `lakehouse-bridge`, which lands idempotently named files in S3, and `guardrail-monitor`, which raises hourly alerts (D52);
2. the workflow invokes a Lambda, `export_edge`, that copies yesterday's new Duels, votes, shared results and withdrawals out of DynamoDB into S3 as JSONL (D30);
3. a sync step uploads only the new files from S3 into the Unity Catalog volume `ops.lakehouse_files` with the Databricks Files API, under the same names. S3 stays the durable replay log outside Databricks (D62);
4. it triggers the Databricks Job `ebook_daily` by API and waits;
5. `site_export` reads Gold through the Statement Execution API, validates small JSON files against schemas and writes them to the site bucket; CloudFront serves them.

GitHub signs in to AWS through OIDC, so no AWS key exists anywhere; Databricks access uses the least long-lived credential Free Edition allows (D48).

In Databricks Free Edition (serverless only, $0, no payment method), one triggered Lakeflow Spark Declarative Pipeline builds everything (D39–D44):

- **Bronze**: raw streaming tables loaded by Auto Loader from the volume. Five come from Kafka files and five from real-data files: the catalog, Upworthy, shared results, Duels and Duel votes.

- **Silver**: typed, deduplicated (1 h watermark, D10) and quarantined streaming tables with expectations. It also holds the Duel votes (double votes and bots out, a split check per Duel) and an AUTO CDC SCD2 users table with hashed and masked PII.
- **Gold**: 20 metadata-driven materialized views (D57), including Duel results and the Duel library.

The `ebook_daily` Job then runs, as notebook tasks on serverless (D51):

1. checks of validity (SRM, A/A, invariants), stopping the run before publishing if a blocking check fails;
2. the pre-registered tests (z-test, Welch/bootstrap, CUPED, Holm/BH, fixed horizon);
3. `MERGE` into `gold.experiment_results`, and one-page memos;
4. proofs against the injected truth;
5. the library rebuild from Duel results and shared ad tests (`13_community_summary`).

A real-data notebook fits a random-effects model to 4,873 real Upworthy headline tests. It publishes the lessons and a realistic prior. Unity Catalog grants, masks, tags and lineage govern every table (D45). AWS CDK deploys the AWS side; Declarative Automation Bundles deploy the Databricks side (D47).

Lab (practice for the portfolio, labelled as such; D31, D55, D61). Paid by the AWS sign-up credits, never in the daily path:

- a Spark UI performance lab on AWS Glue 6.0: 20 M generated product views where one bestseller holds 30% of them, fixed with a broadcast join and a skew fix, measured in Glue's serverless Spark UI (Free Edition serverless has no Spark UI);
- `OPTIMIZE`, `VACUUM`, `DESCRIBE HISTORY` and `RESTORE` demos on Delta.

§3.0 says plainly what each piece is really for, and how to explain it in an interview.

1.2 In scope

- Brief items 1–10, adapted:
  - the producer and contracts;
  - both consumer groups;
  - Bronze, Silver, Gold on Databricks;
  - analysis and proofs;
  - performance and observability (query profile, pipeline event log, Glue Spark UI in the Lab, CloudWatch);
  - orchestration and CI/CD (GitHub Actions, the Lakeflow Job, Step Functions for Duels, CDK and Bundles);
  - compliance (Unity Catalog, IAM, Ley 1581 / 1266).
- Deliverables 0–6.
- From v2–v3: the public website and the writer tools; the real-data track; the laptop-free daily run; the community library and opt-in anonymous sharing (Q14–Q16); round 5's clarity changes C1–C12 (§2.8); the Duel (L1), the game (L2), the Ad Doctor (L3), Real or luck? (L4), the three-door home (L5), the library fed by Duels (L6), real votes through the platform (L7), lifesaver first (L8), the warmer voice (L9), the new look (L11), For nerds (N1) and the real-use verdicts (N2).
- New in v4.0 (round 7, §2.10): the Duel service and the site on AWS's free allowances; the Duel lifecycle state machine; S3 as the replay log; serverless only; Kafka without Aiven; $0 guardrails; interview answers (§3.0).

1.3 Out of scope

Item	Why
Real customers and real PII in the store	Synthetic data only. Free Edition also reserves the right to train on workspace data.
Sending a writer's numbers anywhere without their OK	The Ad Doctor, Real or luck? and the planner compute in the browser. Only Duel votes (as counts), the writer's own Duel and results a writer chooses to share leave it (D24, D37).
Writer accounts, logins, free-text comments	A Duel lives on its private link; sharing is anonymous (D25).
A live, real-time library	The library refreshes every night with the daily run. The site stays static (D12).
Showing a Duel's winner before the end	Progress shows counts and the split check, never the winner (D34).

Storing IP addresses	One vote per visitor and the share cap use a hash with a salt that is destroyed every day (D25, D56).
Mixing simulated results into the real library	The demo uses example data. The real site shows only real results (D26).
Routing Duel votes through Kafka	A few votes a day fit in one daily file; Kafka for votes would need an always-on broker (D30, Q22).
Showing different prices to different people	Ethically and legally sensitive (D20).
Paid Databricks, job clusters of any kind, a local Spark cluster	$0 and serverless only (D53, D55).
AWS services without a monthly free allowance in production: Glue jobs, Athena, API Gateway (after 12 months), MSK, Kinesis, EMR, Redshift, NAT gateways	The $0 rule (D53). Glue appears only in the Lab, on credits (D61).
Aiven, Cloudflare, GitHub Pages as the final host	Replaced by GitHub Actions Kafka (D52) and AWS (Q19). The current landing page points to CloudFront from S1.1.
Power BI, Amazon QuickSight, AI/BI dashboards	The React site is the BI layer (Q10).
Sequential testing (mSPRT)	Fixed horizon (Q6), for the store and for Duels.
Avro + Schema Registry, Kafka Connect	ADR-004.
Book texts and real cover images in the store	The store uses only CC0 metadata and generated SVG covers. Writers' Duel covers are deleted 30 days after the Duel (D35).
The Duel "for everyone"	L10: not now. Writers first.
SLAs, HA, multi-region, ML models	Portfolio scope.
Test guides for newsletters, own sales pages and big stores	Not chosen in Q13.
A public "See an A/B test from inside" page	C13, skipped in round 5 (§2.8).

1.4 Additions and deviations from the brief

None of these is silent. Each is explained.

ID	Change	Reason
A1	The topics are named `market.*` instead of `fintech.*`. Still exactly 5 topics × 2 partitions.	The domain changed (Q8).
A2	Validation fixtures EXP-AA-000 (A/A) and EXP-SRM-900, which drops 5% of treatment exposure logs.	The A/A and SRM proofs run on real pipeline data.
A3	Landing path `topic/epoch/partition/firstOffset-lastOffset.jsonl.gz`.	A CI Kafka is created per run, so offsets restart. The epoch keeps file names unique.
A4	Bronze adds `source_epoch`, `_source_file` and `_ingested_at`.	Lineage stays auditable.
A5	`schema_version` lives on the envelope. v2 adds an optional `network_type` from 2026-07-20.	Lets `from_json` parse each version cleanly.
A6	`gold.experiment_results` and `gold.real_prior` are Delta tables written by notebooks, not MVs.	A notebook can't write an MV.
A7	Two landing places: the S3 bucket `ebook-ab-lake-<account>` (`landing/`, `landing_real/`, `artifacts/`), the durable replay log; and the volume `ops.lakehouse_files` (`landing/`, `landing_real/`), the copy Auto Loader reads.	D62.
A8	Gold has 20 views instead of 41, and every view names its consumer.	Your point 1; v3.0 added 2 Duel views.
A9	File-based Bronze sources (catalog, Upworthy) sit beside the 5 Kafka sources.	Real data arrives as files (Q9).
A10	Phases are numbered 0–10.	Unchanged from v3.0.
A11	Catalogs `ebook_ab` / `ebook_ab_dev`; Python package `ebook_ab`; job and pipeline names use the `ebook_` prefix.	The domain changed.
A12	The "job cluster" requirement is dropped: serverless only (D55). The Glue Lab shows a managed Spark run with its Spark UI.	Your decision 2.
A13	Bayesian readouts appear only in the public tools. Platform decisions and Duels stay frequentist, fixed-horizon and pre-registered.	Q6 still holds for the platform.

A14	The daily schedule lives in GitHub Actions, which triggers the Job `ebook_daily` by API. The bundle keeps a paused 06:30 Bogotá schedule as a fallback. The pipeline stays triggered, never continuous.	Data must be produced and landed before the Job runs, and that happens in the GitHub run.
A15	A file-based Bronze source, `community_raw`, filled by the daily DynamoDB export.	Shared results arrive through the API, not Kafka.
A16	`gold.community_summary` is a Delta table written by notebook `13_community_summary`.	The random-effects summary runs in Python, like A6.
A17	The Duel service lives in `edge/`: Lambda function URLs (TypeScript) + DynamoDB + an S3 covers bucket + the Duel lifecycle state machine, deployed by CDK.	A static site can't receive data (D49, D60).
A18	v3.0's phase numbering stays: Phase 1 is the lifesaver launch.	Real writers use the Duel while the pipeline is built (L8).
A19	File-based Bronze sources `duels_raw` and `duel_votes_raw`, filled by the daily DynamoDB export.	Duel votes go as files (D30).
A20	Gold includes `duel_results` and `duel_library`.	L7.
A21	The Duel service: create, read, vote, progress, result, game queue, report, withdraw and admin routes.	L1, L2.
A22	Practice-only parts are labelled Lab: the Glue Spark UI lab and the Delta `OPTIMIZE` / `VACUUM` / `RESTORE` demos.	D31, D55, D61.
A23	Kafka lives off AWS and off Aiven: local Docker and the GitHub Actions service container. No Path A. The backfill runs as a manual GitHub workflow.	D52.
A24	The product side moves to AWS: Lambda, DynamoDB, Step Functions, S3, CloudFront replace the Cloudflare Worker, D1 and GitHub Pages.	AWS skills on a real, used service, at $0 (Q19, Q21, Q23).

2. Decisions log

Rounds 1–6 are the record of how the plan grew. They stay as they were, with a note wherever v4.0 changes a consequence. Round 7 (§2.10) and the new lead decisions (§2.11) are the AWS redesign.

2.1 Round 1 (plan v1 interview, 2026-10-15)

#	Question	Options offered (recommended first)	Your answer	Consequence (v4.0 status)
Q1	Which accounts do you already have?	Databricks Free Edition · LinkedIn-verified · Aiven account · Aiven free Kafka running	Databricks Free Edition only	v4.0: Free Edition runs the lakehouse (Q23); AWS is added for the product (Q20).
Q2	Hours per week?	10 h (Rec.) · 5 h · 20 h	20 h/week	240 h in v4.0 → about 12–14 weeks.
Q3	Repo name?	fintech-ab-platform (Rec.) · fintech-ab-lakehouse · colombia-fintech-experimentation	fintech-ab-platform	Changed to ebook-ab-platform because the domain changed (D17).
Q4	Plan language?	English (Rec.) · Spanish · English + Spanish summary	English	Docs in English. The public site is Spanish + English.
Q5	Docker runtime?	Docker Engine in WSL2 (Rec.) · Docker Desktop	Docker Engine in WSL2	v4.0: runs only the local Kafka (D55).
Q6	Peeking policy?	Fixed horizon only (Rec.) · Add mSPRT interim looks	Fixed horizon only	One readout per experiment at its cutoff.
Q7	Injected truth story?	Mixed outcomes (Rec.) · All three ship · Mixed with a true null	Mixed outcomes	Expected verdicts: CHK ship; PDP don't ship (buyers up, refunds way up); BUN ship.

2.2 Your 8 points on v1, and what changed

#	Your point	What changed (v4.0 status)
0	Ebook sellers instead of fintech?	The domain is a marketplace with a payments checkout (Q8).
1	41 views were only an example	Gold has 20 tables, each with a named consumer (A8, A20).
2	Real monetary impact; real data?	A free seller tool, real-data lessons, and the simulation kept as proof (Q9).
3	Job clusters instead of serverless	v4.0: serverless only, by your decision 2 (D55): Databricks serverless for the pipeline; a Glue Lab on credits gives the Spark UI.
4	Easy, visual, interactive; what we achieve	Part 1 and the demo (`docs/plan/DEMO.html`).
5	React or Angular, not Power BI	React + TypeScript (Q10); since v4.0 served from S3 + CloudFront.
6	Local or AWS?	v2–v3 rejected AWS to stay at $0. v4.0 adds AWS for the product, inside its free allowances, with Databricks Free Edition for the data (round 7).
7	Why Docker?	v4.0: local Kafka only (the Spark cluster is gone, D55).
8	Which accounts?	v4.0: Databricks Free Edition (you have it), a new AWS account, and GitHub.

2.3 Round 2 (2026-10-15)

#	Question	Options offered (recommended first)	Your answer	Consequence (v4.0 status)
Q8	Which domain should the platform serve?	Ebook marketplace + payments (Rec.) · Ebook sellers, content only · Keep the Colombian fintech	Ebook marketplace + payments	The checkout keeps payment methods, fraud, refunds and chargebacks as guardrails.
Q9	How should the project create real-world impact?	Tool + real data + simulation (Rec.) · Tool + simulation · Simulation only	Tool + real data + simulation	§3.12 (real data) and §3.13 (tool).
Q10	Which frontend stack and hosting?	React + TS on GitHub Pages (Rec.) · Angular · React as a Databricks App	React + TS on GitHub Pages	v4.0: same React site, hosted on S3 + CloudFront (Q19, D50).

Q11	Job clusters: keep $0 or pay?	$0: local job cluster (Rec.) · Pay for real job clusters · $0: serverless only	$0: local job cluster	v4.0: superseded by your decision 2, serverless only (D55).

2.4 Verified instead of asked

- From 2026-10-15 (still valid): repo name `ebook-ab-platform` is free on GitHub; npm versions are re-pinned at Phase 1 start; no brand name for the marketplace (D18).
- New, 2026-10-19 (official AWS pages, §4):
  - On the new sign-up's Free plan, AWS Glue, Amazon Athena, AWS Lake Formation, Amazon S3, AWS Lambda, Amazon API Gateway, Amazon DynamoDB, AWS Step Functions, Amazon EventBridge, Amazon CloudFront, Amazon CloudWatch, Amazon ECR, Amazon SNS, AWS Systems Manager, AWS Budgets and AWS CloudFormation are supported. Amazon MSK, Kinesis Data Streams, Data Firehose, Amazon EMR and Amazon Redshift need the paid plan. IAM Identity Center is not supported.
  - Monthly free allowances that don't expire with the Free plan: Lambda (1 M requests, 400,000 GB-seconds), DynamoDB (25 GB, 25 WCU, 25 RCU, provisioned mode only), Step Functions (4,000 state transitions, "available to both existing and new AWS customers indefinitely"), EventBridge Scheduler (14 M invocations), the Glue Data Catalog (1 M objects, 1 M requests), CloudWatch (5 GB of logs, 10 alarms), AWS Budgets (budgets free; two action budgets free).
  - CloudFront's flat-rate Free plan: $0 a month with no overage charges; 1 M requests and 100 GB a month; basic WAF with 5 rules; and S3 storage credits for 5 GB that "offset any S3 Standard storage costs in your AWS account", whether or not the bucket is a CloudFront origin. Up to 3 Free plans per account. Accounts on the AWS Free plan aren't eligible for flat-rate plans.
  - No monthly free allowance: AWS Glue jobs ($0.44 per DPU-hour list), Amazon Athena ($5 per TB scanned), S3 requests. API Gateway's 1 M free calls a month last 12 months.
  - AWS Glue 6.0 (generally available 2026-08-21) runs Spark 4.1.1, Python 3.13, Java 17, Scala 2.13, Iceberg 1.11.0 and Delta Lake 4.2.0, enables ANSI mode by default, and is 30% cheaper than earlier versions. Athena reads Delta tables registered in the Glue Data Catalog (reader version ≤ 3) but can't write or time-travel them. Glue's serverless Spark UI works for Glue 3.0+ job runs.
  - Databricks Free Edition (limitations page, updated 2026-09-29): serverless compute only; one SQL warehouse (2X-Small); at most 5 concurrent job tasks; one active pipeline per pipeline type; outbound internet only to trusted domains; custom workspace storage locations unsupported; no commercial use; a daily fair-use quota that stops compute for the rest of the day.
  - Databricks documents workload identity federation for GitHub Actions (OIDC with a service principal federation policy); whether Free Edition allows it is O34.
  - A Lambda function URL can sit behind CloudFront with origin access control; browsers must then send the SHA-256 of each POST body in the `x-amz-content-sha256` header.

2.5 Lead decisions D1–D29 (my calls; override them at review)

ID	Decision (v4.0 status in *italics*)	Rationale
D1	The repo lives at `D:\abtesting`. Heavy local data lives on WSL ext4.	Windows tools need Windows paths; Linux I/O stays on ext4.
D2	Python 3.12 via uv, with one lockfile.	Matches Databricks serverless environment v6 (Python 3.12.3).
D3	*Amended by D40:* local PySpark in local mode for unit tests only, pinned to the Spark line of Databricks serverless; serverless itself is versionless.	Tests stay close to production.
D4	Kafka 4.2 everywhere (`apache/kafka:4.2.2`), locally and in the GitHub Actions service container.	No version drift.
D5	*Superseded by D52:* one ingestion path, the bridge to S3.	Path A needed Aiven and Databricks.
D6	JSON + JSON Schema (draft 2020-12) contracts in Git.	ADR-004.
D7	Primary metrics are user-level. Ratio metrics use the delta method.	Users are the randomization unit.
D8	N = 70,000 shoppers over 92 days.	Sized by EXP-PDP-002, the binding constraint (§3.19).
D9	Producer: idempotence, `acks=all`, zstd. *v4.0: the 50 KiB/s throttle is dropped with Aiven.* Kafka timestamps are send time.	Durability; no free-tier bandwidth cap any more.

D10	Silver dedup is a 1 h watermark on event_ts + `dropDuplicatesWithinWatermark(["event_id"])`.	2× the maximum injected lateness.
D11	Keyed SHA-256 hashes (a pepper in the Databricks secret scope) make the join keys. Unity Catalog masks hide display columns. Gold has no PII.	Ley 1581 minimization.
D12	The React site is static: Gold → JSON → S3 → CloudFront. No AWS credential ever reaches a browser.	Cheap, public, secure.
D13	Primary: two-sided α = 0.05. Secondaries: Holm. Segments: BH q = 0.10. Guardrails: one-sided α = 0.05.	Unchanged.
D14	Catalogs `ebook_ab_dev` / `ebook_ab`, chosen by a bundle variable. Fallback: `workspace`.	
D15	Job failure e-mails go to `${workspace.current_user.userName}`; AWS alarms go to an SNS topic whose address you type at deploy time.	Keeps your address out of the repo.
D16	MIT license, `.env` git-ignored, secret scanning + push protection on.	
D17	The repo is `ebook-ab-platform`.	The domain changed.
D18	No brand name: the site calls it “a fictional Latin American ebook marketplace” and shows a banner.	Avoids clashing with real brands.
D19	The daily laptop-free run uses an ephemeral Kafka service container in GitHub Actions, with one epoch per run, plus the bridge. *v4.0: the backfill too (D52).*	A CI Kafka always works and costs nothing.
D20	Experiment ethics: no individual price differences, no fake urgency or scarcity, presentation-only changes, invariants that prove prices are identical across arms.	Fairness and consumer trust (O12).
D21	Real data comes from the Upworthy exploratory sample only. Gutenberg metadata only.	Follows each source’s guidance and license.
D22	The public site sets no cookies, has no analytics and makes no third-party requests. Tool math runs in the browser. Amended by D24 and D37.	Privacy by design.
D23	*Superseded by D49:* Duels, votes and shared results live in DynamoDB behind a Lambda function URL and CloudFront.	AWS (Q19), $0 (Q21).
D24	The only request that leaves a visitor’s browser outside Duels is the opt-in Share: one POST to our own API, after the writer reviews every field and presses Share.	Keeps D22’s promise with one voluntary exception.
D25	Anti-abuse without accounts: JSON Schema validation, fixed lists, number ranges, clicks ≤ views, fair tests only (views ratio < 1.5), duplicate payloads rejected, a cap of 5 shares per visitor a day, outlier flags, robust statistics. The cap and one vote per visitor per Duel use a daily-salted hash of the IP address (D56). Silver drops double votes and bot-like bursts.	Spam and poisoning are the main risks of open sharing (R34).
D26	Cold start: the real site shows only real shared results. Fewer than 5 results → “Not enough results yet: be one of the first” and an “x of 5” counter. The same minimum applies to rankings, idea tags and comparisons. Simulated data never enters the real library.	Honesty (R35).
D27	The library is published as open data (CC BY 4.0) in `library.json`: counts, summaries, idea tags and rounded lifts. Raw records are never published. A receipt code withdraws a shared result.	Gives back; protects small groups (R37).
D28	Every number block wears a source badge: Shared by writers, Real public tests or Simulated.	A visitor must never mistake a simulated number for a real one (C1).
D29	Every ad test is planned for at least 7 days, with the same budget spread over the week.	Weekdays and weekends behave differently (C12).

2.6 Round 3 (plan v2.1, 2026-10-15)

#	Question	Options offered (recommended first)	Your answer	Consequence

Q12	How should the product be clearer for a writer?	Writer-first redesign (Rec.) · One walkthrough only · The explanation is enough	Writer-first redesign	The seller tool becomes a guided assistant with a plain verdict (§3.13).
Q13	Where do the writers in mind sell or promote?	Ads · Email newsletter · Own sales page · Big stores	Ads	The tool plans and reads ad tests. Other channels are out of scope.

2.7 Round 4 (plan v2.2, 2026-10-15)

You asked how writers would come back, and where the magic is when ad platforms already pick a winner. My answer: one writer's numbers are too small, but many writers' results together are enough. The tool pools what many small writers learned, by genre, book language and type of change.

#	Question	Options offered (recommended first)	Your answer	Consequence
Q14	Which hooks should bring writers back?	Before you decide + journal · Real or luck? · Community library · Weekly check-in	Community library	The library sits inside the writer tool (§3.13).
Q15	Should writers be able to share results anonymously?	Yes, opt-in and anonymous (Rec.) · No	Yes, opt-in and anonymous	A storage service is needed. v4.0: DynamoDB behind the Duel service (D49). Rules D24–D27.
Q16	What should the writer app be called?	Antes de decidir · Before You Decide (Rec.) · others	Antes de decidir · Before You Decide	The name is the habit: open it before any decision.

2.8 Round 5 (plan v2.3, 2026-10-15)

You read v2.2 and the demo and asked nine questions; the answers (`docs/plan/answers/ANSWERS-v2.2.pdf`) proposed 13 changes, and you chose "apply C1–C12, skip C13".

#	Change	Where in v4.0
C1	Badges for the three kinds of numbers	Section C, D28, §3.13
C2	The library's launch-day state and "x of 5" counter	Section C, D26, §3.13
C3	An idea is listed only after 5 tests used it, as "worked in W of N"	D26, §3.12, §3.13
C4	The library card says what "worked" means: more clicks or more readers, not sales	Section C, §3.13
C5	A "?" on each ad number: where to find it on each ad platform	§3.13, O20
C6	The sales page folded into "Why not test on my book's page?"	§3.13
C7	The "compared with writers like you" line needs 5 results	D26, §3.13
C8	Real lessons as three lessons, conclusion first	§3.13
C9	Marketplace experiments becomes Test store, under "Behind the scenes"	§3.13
C10	New risk R38 (writers share mostly wins): Share offered for every fair verdict	§3.13, §6
C11	The "Founding writers" launch plan, plus a public counter	Section C, Phase 1
C12	Every ad test runs at least 7 days	Section E, D29
C13	Optional "See an A/B test from inside" page	Skipped (§1.3)

2.9 Round 6 (plan v3.0, 2026-10-17)

You asked how this could be a lifesaver for tons of people. The answer (`docs/plan/answers/LIFESAVER.pdf`) said A/B testing is the engine, never the product, and proposed L1–L11. You took L1–L9 and L11, asked for a For nerds tab, the real use of each technology, and one plan.

#	Change	Where in v4.0
L1	Duel: cover, title, blurb or book idea; one link, one version per person, progress without peeking, the reveal	Sections A–D, §3.13, D34–D37, Phase 1
L2	Which would you read? A game where voting earns votes	§3.13, D36, Phase 7
L3	Ad Doctor: Amazon Ads reports read in the browser	§3.13, Phase 7
L4	Real or luck?: two numbers, an honest answer	§3.13, Phase 1
L5	A home page with three doors	Sections A–B, §3.13, Phase 1

L6	The library fills itself from Duels	Section C, §3.12, §3.13, Phase 7
L7	Real votes through the platform: Bronze/Silver/Gold, an SRM check per Duel, a bot filter, 2 Gold tables	§3.2, §3.6, §3.9, §7
L8	Lifesaver first: ship L1, L4 and L5 right after Phase 0	§5, Phase 1
L9	A warmer voice	§3.13
L10	Later: the Duel “for everyone”	Not now (§1.3)
L11	The new look in the style of cursor.com	§3.13, Phase 1
N1	For nerds: the architecture map and the stack	Section B, §3.13, Phase 7
N2	The real use of each piece, practice parts labelled Lab	v4.0: §3.0 rewritten as interview answers

Lead decisions added in v3.0, with their v4.0 status:

ID	Decision (v4.0 status in *italics*)	Rationale
D30	Duel votes travel as files, not through Kafka. The daily run exports new Duels, votes, shared results and withdrawals *from DynamoDB* as JSONL to `landing_real/duels/` and `landing_real/community/`. Kafka carries only the simulated store.	A few votes a day fit in one file; Kafka’s strengths buy nothing here.
D31	Practice-only parts stay, labelled Lab. *v4.0: the Lab is the Glue Spark UI lab (sign-up credits) and the Delta maintenance and rollback demos (D55, D58, D61).*	Saying so is honest (§3.0).
D32	Round 6 takes L1–L9 and L11. L10 waits.	
D33	The plan stays in English (Q4). I summarize each version in Spanish in the chat if you ask.	Round 1 decision.
D34	Duel rules, fixed before it starts: 330 readers or 14 days; the result opens only at the end; SRM $p < 0.001$ → “Not fair yet”; a gap under 5 points → “Too close to call”; otherwise a two-proportion z-test at 95% names the winner or says “Not sure yet: about N more readers”. One vote per visitor per Duel; the writer’s own device is not counted.	Nobody moves the goalposts after seeing the score.
D35	*Amended by D49:* covers are shrunk in the browser to at most 300 KB (WebP, JPEG fallback), uploaded to a private S3 bucket and deleted 30 days after the Duel ends by the Duel lifecycle state machine (D60).	S3 removes v3.0’s 60 KB squeeze.
D36	Moderation: unlisted links; you approve which Duels enter the game with `ebook_ab.tools.moderate`; a report button on every Duel. *v4.0: admin routes use IAM authorization, not a shared token (D59).*	People upload images.
D37	D22 and D24 amended: a writer’s Duel and readers’ votes (counts only) also leave the browser. No cookies, no names.	A Duel can’t work without votes leaving the reader’s phone.

2.10 Round 7 (plan v4.0, 2026-10-19)

You made two decisions and asked for one rewrite:

1. “I will use AWS no matter in 6 months I should pay”: redo the whole plan around an AWS architecture.
2. “We will use only serverless since job cluster cost money.”
3. Redo page 26 (§3.0) as the best way to answer, with a real example, when someone asks how you used a technology in this project and why it was necessary.
4. Later the same day: “I want Delta tables” (D39), and a question, “can we use Kafka in the Duel? … maybe I do not want [the test store] anymore” (Q22).
5. Then: “and databricks???????”. Databricks had left the plan through Q17 and Q21; you chose to bring it back as a $0 hybrid (Q23).

Checking the AWS facts raised four choices, which I asked you on 2026-10-19. Then, seeing that the first draft would cost about $8 a month after the credits, you asked: “The plan should stay at $0 cost. Is there no $0 path in AWS?” My answer: not for Glue and Athena, which have no free allowance, but yes for a design built only on AWS’s monthly free allowances, with Spark running in the free GitHub Actions run. That became Q21. When you asked about Databricks, the hybrid (Q23) put Spark back on Databricks Free Edition, still at $0, and kept AWS for the product.

#	Question	Options offered (recommended first)	Your answer	Consequence
Q17	What runs the lakehouse on AWS?	AWS-native: Glue + Iceberg + Athena + Lake Formation (Rec.) · Databricks on AWS (paid) · Keep Free Edition 1. AWS	AWS-native	A lakehouse on S3 with the Glue Data Catalog. Amended by Q21 (Glue jobs and Athena move to the Lab) and by your decision 4 (Delta instead of Iceberg); *superseded by Q23* (the lakehouse returns to Databricks Free Edition).
Q18	Where should Kafka run?	A Kafka broker on Fargate per run (Rec.) · GitHub Actions, as now · Amazon MSK	GitHub Actions, as now	The CI Kafka stays; the bridge writes to S3 through an OIDC role. Aiven and Path A leave (D52).
Q19	Should the Duel service and the website move to AWS too?	Move both (Rec.) · Keep both off AWS · Move the Duel service only	Move both	Lambda + DynamoDB + S3 covers; the site on S3 + CloudFront (D49, D50).
Q20	Do you already have an AWS account?	No, I'll sign up · Yes, on the Free plan · Yes, older or paid	No, I'll sign up	Phase 0 creates a new account on the Free plan; S1.1 upgrades it before launch (D54).
Q21	Which $0 version should the plan become?	AWS always-free (Rec.) · Back to v3.0, $0 forever · Hybrid: v3.0 + AWS edge · Free plan only, 6 months	AWS always-free	Only services with a monthly free allowance in production on AWS; Glue as a Lab on credits; S3 requests (cents) are the one cost (section G, D53). *Amended by Q23:* Spark moves from GitHub Actions back to Databricks Free Edition.
Q22	(After Delta and "can we use Kafka in the Duel?") What should happen to the simulated store and Kafka?	Keep store + Kafka (Rec.) · Keep store, add DynamoDB Streams · Drop store and Kafka · Drop store, Kafka for votes	Keep store + Kafka	Kafka stays with the simulated store; Duel votes stay daily files (D30). Kafka for votes would need an always-on broker that isn't free and reliable, for a few hundred votes a day.
Q23	Where should Databricks fit?	Hybrid: Free Edition + AWS (Rec.) · Keep AWS-only · Databricks on AWS, paid	Hybrid: Free Edition + AWS	Databricks Free Edition runs the Delta lakehouse (Lakeflow pipeline, Auto Loader, Unity Catalog, Jobs); AWS runs the Duel service and the site; Kafka stays in GitHub Actions.

2.11 Lead decisions added in v4.0 (my calls; override them at review)

ID	Decision	Rationale
D38	AWS region: US East (N. Virginia), `us-east-1`. The Free Edition workspace keeps whatever region Databricks gave it (A1).	Every AWS service we need is there; CloudFront serves Colombia from nearby edges.
D39	Every Bronze, Silver and Gold table is Delta Lake (your decision 4), as Unity Catalog managed tables in Databricks Free Edition.	Delta is native on Databricks: MERGE, time travel, Change Data Feed and AUTO CDC need no extra setup.
D40	Spark runs on Databricks serverless (Free Edition) only, through one triggered Lakeflow Spark Declarative Pipeline and the `ebook_daily` Job. Locally, PySpark in local mode runs unit tests only.	The hybrid (Q23); serverless only (D55).
D41	Ingestion with Auto Loader from the Unity Catalog volume `ops.lakehouse_files` (`landing/`, `landing_real/`), with exactly-once file processing (`allowOverwrites=false`).	Free Edition's outbound internet only reaches trusted domains and its docs don't promise access to your own S3, so files are pushed into the volume (D62).
D42	*Retired with the AWS-only design:* Silver dedup is back to D10 (watermark + `dropDuplicatesWithinWatermark`).	Declarative pipelines handle it.

D43	`users_scd2` with `create_auto_cdc_flow(stored_as_scd_type=2)`; withdrawals and Duel updates with AUTO CDC as deletes / SCD type 1.	Native on Databricks.
D44	Data quality with pipeline expectations: `expect_or_fail` (known topic, no clear PII), `expect_or_drop` with a quarantine flow, `expect` (warn).	Native, visible in the pipeline UI and event log.
D45	Governance in two places. Data: Unity Catalog grants (`bi_readers` on Gold only, `pii_readers` can unmask), column masks, tags (`pii`, `ley_1581`, `ley_1266`, `purpose`) and lineage. AWS: one IAM role per job, least privilege. Secrets: the pepper in the Databricks secret scope; the daily salt in Parameter Store.	Each platform guards what it holds.
D46	Orchestration: one GitHub Actions workflow every morning runs Kafka, the bridge, `export_edge` and the sync to the volume, then triggers the Databricks Job `ebook_daily` by API, waits, runs `site_export` and publishes. Step Functions runs each Duel's lifecycle (D60).	Kafka lives in GitHub (Q18); the data lives in Databricks; the product lives in AWS.
D47	Infrastructure as code twice: AWS CDK (Python) for AWS (`foundation`, `edge`, `site`, `ci`, `lab` stacks) and Declarative Automation Bundles (`databricks.yml`) for Databricks (pipeline, Job, schemas, volume, secret scope).	Each platform's native tool.
D48	CI identities: GitHub signs in to AWS with OIDC (roles `gha-ci`, `gha-deploy`, `gha-pipeline`; no AWS keys). Databricks: workload identity federation for GitHub Actions with a service principal, if Free Edition allows it (O34); else a service principal OAuth secret; else a personal access token of at most 90 days in GitHub secrets.	No long-lived AWS secret; the least long-lived Databricks secret Free Edition allows.
D49	The Duel service: Lambda functions in TypeScript (Node.js LTS) as function URLs with `AWS_IAM` auth, reached only through CloudFront with origin access control (the browser sends the body's SHA-256 in `x-amz-content-sha256`); DynamoDB in provisioned mode inside the free allowance (table 20/20, index 5/5, auto scaling off); covers in a private S3 bucket by presigned POST (≤ 300 KB). Fallback if the flat-rate plan refuses a Lambda origin: an API Gateway HTTP API.	No API Gateway charge, no on-demand charge, one public door.
D50	The site in a private S3 bucket behind CloudFront (origin access control) on the flat-rate Free plan; CloudFront routes `/v1/*` to the Duel Lambda and `/covers/*` to the covers bucket; a WAF rate rule on `/v1/*`. The current GitHub Pages landing page points there from S1.1.	One origin, no CORS, no overage charges.
D51	Notebooks run as tasks of the `ebook_daily` Job on serverless. After each run, the workflow exports the executed notebooks as HTML through the Jobs API (`runs/export`) into `s3://…/artifacts/runs/`; 10–12 are published on the site, 13 and 20 stay private. A failed blocking check fails the Job, so the workflow stops before `site_export`.	Databricks keeps job runs for 60 days; S3 keeps the record. The gate protects writers from wrong numbers.
D52	Kafka stays off AWS (Q18) and off Aiven: Docker on your laptop for development; a GitHub Actions service container for CI, the backfill (a manual workflow) and the daily run. Path A is dropped.	MSK and Kinesis are paid-plan only; a CI Kafka is free and always works.
D53	The $0 guardrails. AWS: a CI check fails if a production stack adds a resource type outside the free-allowance list; two budgets (out-of-pocket above $0.01; gross above $1 a month); a $5 budget action that denies the pipeline and edge roles; Lambda concurrency caps; logs kept 14 days. Databricks: Free Edition has no payment method, so it cannot bill; heavy runs are split to respect its daily quota (R2).	Paid services must be impossible to add by accident.
D54	Stay on the AWS Free plan while building; upgrade to the paid plan at S1.1, right before the first public deploy.	The Free plan closes the account after 6 months or when credits run out, and CloudFront flat-rate plans aren't available to Free plan accounts. Remaining credits keep paying S3 requests.
D55	Serverless only: Databricks serverless for the pipeline, Lambda and Step Functions for the product. The local	Your decision 2. Free Edition serverless has no Spark UI; Glue's serverless Spark UI does.

	Docker Spark cluster, the job-cluster target and the plain Structured Streaming track are removed. The Spark UI lab runs on AWS Glue, paid by the sign-up credits (D61).	
D56	Visitor hash: `HMAC(daily salt, date + IP + Duel)`. An EventBridge Scheduler rule calls a Lambda at 00:00 UTC that writes a new random salt and deletes the old one. Dedup items carry a 24-hour TTL and are filtered out on read once expired.	DynamoDB deletes expired items “within a few days”; destroying the salt makes leftover hashes impossible to link to an IP.
D57	Gold is 20 materialized views in the pipeline, generated from `config/gold_views.yml` by the Gold factory (`@dp.materialized_view` per spec).	As in v3.0: metadata-driven, each view names its consumer.
D58	`OPTIMIZE` and `VACUUM` on the regular Delta tables (`experiment_results`) stay Lab demos at our size; materialized views reject them.	Honest about scale (§3.0).
D59	Admin and moderation use a separate admin Lambda function URL with `AWS_IAM` auth, outside CloudFront; `ebook_ab.tools.moderate` signs requests with your AWS credentials.	No shared token to leak.
D60	The Duel lifecycle: creating a Duel starts a Step Functions Standard execution: wait until the end time (14 days) → close the Duel if still open → wait until 30 days after the actual end → delete the covers → done. About 6 state transitions per Duel.	Durable timers per Duel with a visible history; 4,000 free transitions a month cover about 650 new Duels a month.
D61	The Lab on AWS credits: the Spark UI performance lab as an AWS Glue 6.0 job on Flex workers, generating its own 20 M rows, from the `lab` stack deployed by hand and destroyed after each session, with a $5 cap.	Glue’s serverless Spark UI, at $0 out of pocket.
D62	S3 is the durable landing zone and replay log: the bridge, `export_edge` and the open-data uploads write there first; one sync step then uploads only the new files into the Unity Catalog volume through the Databricks Files API, under the same names.	Free Edition can’t be assumed to read your S3 bucket, and it may delete an inactive account (R20); the raw history must live outside it. Same names keep the sync idempotent.

3. Architecture

The diagrams are rendered from the Mermaid sources in `docs/plan/diagrams/`. After approval, the demo's For nerds tab draws the same architecture live (Section 0, step 3).

3.0 How you used each technology, and why it was necessary (interview answers)

You asked for the best way to answer, with a real example, when someone asks how you used a technology in this project and why it was necessary. This section replaces v3.0's table. It has three parts: the answer formula, the five questions almost every interviewer asks about a project like this, and one card per technology.

> **Say these only about what you built.** The answers are written in the past tense, as you will say them after the project exists. Before an interview, replace every number with the one your own runs measured (minutes, row counts, the real AWS bill), and open the file named under "Proof in the repo" so you can show it. An answer you can't back with your own repo hurts more than no answer.

The answer formula (30–45 seconds)

1. **What it does in my project**, in one sentence.
2. **The real example**, with a number: "70,000 shoppers, 1.8 million events, 5 topics".
3. **Why it was necessary**: what breaks without it.
4. **The honest limit**: where it is bigger than I need, or what I would change at a real company's scale.

Step 4 is what separates you from candidates who repeat tutorials. Interviewers trust someone who says "at 1 GB, pandas would compute this" and then explains why they still chose Spark. This project has a second strong limit story: "I kept it at $0 on two platforms, and here is what that ruled out".

The verdict for each piece

Piece	Its real job in this project	Verdict
Kafka	The simulated store's event stream: per-shopper order, two independent consumer groups, a restart that loses and repeats nothing	Real, for the store only
Amazon S3	The durable landing zone and replay log, the artifacts, the site and the covers	Real
Databricks Free Edition + Lakeflow pipeline	Builds Bronze, Silver and Gold every morning, declaratively, on serverless	Real
Auto Loader	Reads each new file once, from the volume, with a managed checkpoint	Real
Delta Lake	AUTO CDC for SCD2 and withdrawals, MERGE for results, time travel for reproducible memos, Change Data Feed	Real
PySpark (serverless)	Parsing by schema version, hashing, dedup, the Gold SQL; statistics stop using it after aggregation	Real, bigger than needed
Pipeline expectations	Quarantine in Silver; two rules stop the run	Real
Unity Catalog	Grants, column masks, personal-data tags, lineage	Real
Notebooks (Job tasks)	A readable record of every decision, and a gate before publishing	Real
GitHub Actions + OIDC	The daily conductor (Kafka, export, sync, trigger, publish), tests, deploys; no AWS keys	Real
AWS Lambda	The Duel service, the daily export, small scheduled jobs	Real

DynamoDB	Duels and votes, one vote per visitor, counts	Real
AWS Step Functions	Each Duel's life: close on day 14, delete covers 30 days later	Real
CloudFront + S3 site	Serves the static site, covers and the Duel API from one origin	Real
AWS CDK + Databricks Bundles	Every resource on both platforms as reviewed code, with a $0 allow-list	Real
JSON contracts	One schema per event, request and site file, shared by every reader	Real
`ebook_ab.stats` + scipy	The pre-registered tests, proved against planted truth	Real
React + TypeScript	The product writers use, with in-browser math	Real
AWS Glue jobs + Spark UI	A skew lab on managed serverless Spark, where the Spark UI exists	Lab (on credits)
Delta maintenance demos	`OPTIMIZE`, `VACUUM`, `DESCRIBE HISTORY`, `RESTORE`, Z-ORDER	Lab (time travel and CDF themselves are real)

The five questions almost every interviewer asks

"Why AWS and Databricks? Isn't one enough?"

> *"Each does the part it's best at, and both had a free option I could run every day. The lakehouse runs on Databricks Free Edition: Lakeflow pipelines, Auto Loader, Delta and Unity Catalog, the stack most data-engineering teams I apply to use. The product writers touch runs on AWS: Lambda, DynamoDB, Step Functions, S3 and CloudFront, inside their permanent free allowances. The seam between them is plain files: AWS keeps the raw history in S3, and a sync step pushes new files into a Unity Catalog volume. That seam was a real design problem, because Free Edition only reaches trusted domains and can't be pointed at my own bucket."*

"Why serverless?"

> *"Because the work is small and short, and I didn't want to pay for anything that waits. The site and the Duel service are CloudFront, Lambda and DynamoDB, which cost nothing when nobody visits. The pipeline runs once a day as a triggered update on Databricks serverless, so there's no cluster to size or forget to turn off. The trade-offs are limits: Lambda stops at 15 minutes, DynamoDB's free capacity is 25 writes a second, and Free Edition has a daily compute quota and no Spark UI. I designed inside those limits on purpose."*

"Why Free Edition and not AWS Glue for Spark?"

> *"Glue has no free allowance: about forty-four cents per DPU-hour, every day. Free Edition gives me serverless Spark, Delta and a declarative pipeline at zero. What I lose is the Spark UI, so I learned that part on Glue: in a lab paid by my sign-up credits, I ran a skewed 20-million-row job on Glue 6.0 and fixed it with the Spark UI. At a company I'd run the same pipeline on paid Databricks; the code wouldn't change."*

"How much does it cost?"

> *"Zero out of pocket. Databricks Free Edition has no payment method, so it can't bill. On AWS everything runs inside the monthly free allowances except S3 requests, a few cents a month that my sign-up credits paid. A CI check fails any change that adds a paid AWS service, and two budgets e-mail me at one cent of real cost or one dollar of usage."*

"What would you change at 100 times the data?"

> *"Four things. The pipeline would move to paid Databricks, because 90 GB a day would break the Free Edition quota; the pipeline code wouldn't change, but I'd read S3 directly through an external location instead of syncing files. Kafka would move to Amazon MSK or Confluent Cloud, because the stream would run all day. I'd add the Spark UI and cluster policies back for tuning. And DynamoDB would switch to on-demand capacity, because the free 25 writes a second would become a ceiling."*

One card per technology

Kafka (in GitHub Actions and Docker)

THEY ASK *"Your votes don't even use Kafka. Why is it in this project?"*

YOU ANSWER

Kafka carries one thing: the simulated store's event stream. 70,000 shoppers produce about 1.8 million events over 92 simulated days, in 5 topics with 2 partitions each, keyed by user, so each shopper's events stay in order. Two consumer groups read the same events independently: a bridge that lands files in S3, and a guardrail monitor that raises hourly alerts on fraud, refunds and checkout latency per experiment arm. That is why Kafka was necessary: one stream, several independent readers, and a restart that loses and repeats nothing. I proved it by killing the bridge right after an upload: zero duplicates and zero offset gaps in Bronze. Real Duel votes skip Kafka on purpose: a few hundred a day fit in one file.

IF THEY PUSH **"Why not Amazon MSK?"** MSK isn't on the AWS Free plan, and MSK Serverless costs $0.75 an hour, about $548 a month, for a stream I use 20 minutes a day. So Kafka runs as a throwaway container inside the GitHub Actions run, with a new epoch each run. With a 24/7 store I'd use MSK or Confluent Cloud; the producer and consumers wouldn't change.

PROOF IN THE REPO `src/ebook_ab/bridge`, `src/ebook_ab/monitor`, the kill test in `tests/integration`, ADR-002.

Amazon S3 (landing zone and replay log)

THEY ASK *"Your tables are in Databricks. Why keep S3?"*

YOU ANSWER

Because the raw history must live somewhere I control. Every input lands in S3 first: Kafka batches from the bridge, the daily export of Duel votes from DynamoDB, the Upworthy CSV and the Gutenberg catalog. File names are deterministic (topic, epoch, partition and offset range), so a re-upload writes the same object with the same bytes. A sync step then copies only the new names into a Unity Catalog volume, where Auto Loader reads them. Free Edition can delete an inactive workspace and doesn't let me choose its storage, so S3 is my replay log: when I changed a Silver rule, I rebuilt everything from S3 in dev.

IF THEY PUSH **"What does it cost?"** Storage is far under 5 GB, which CloudFront's free plan credits. Requests are the one thing AWS never gives free: a few cents a month. Large files keep them low.

PROOF IN THE REPO `infra/stacks/foundation.py` (bucket, lifecycle rules), `src/ebook_ab/tools/sync_volume.py`, §3.3.

Databricks Free Edition and the Lakeflow pipeline

THEY ASK *"How did you build the lakehouse?"*

YOU ANSWER

As one Lakeflow Spark Declarative Pipeline on Databricks Free Edition, serverless, triggered once a day. I declare what each table is, and the pipeline works out the order, the checkpoints and the retries. Bronze is ten streaming tables loaded by Auto Loader. Silver is typed, deduplicated with a one-hour watermark, and quarantined, with an AUTO CDC flow that keeps a type-2 history of each shopper. Gold is 20 materialized views generated from one YAML file, each naming who uses it. A Databricks Job runs the pipeline and then the analysis notebooks, and GitHub Actions triggers that Job by API after the data has landed.

IF THEY PUSH **"Free Edition in production?"** It's non-commercial and has a daily quota, which fits a portfolio, and I say so. I measured the daily run against the quota and split heavy proofs into a weekly run. The real limit I hit was networking: Free Edition only reaches trusted domains, which is why files are pushed into a volume instead of read from S3.

PROOF IN THE REPO `pipelines/ebook/`, `config/gold_views.yml`, `resources/ebook_daily.job.yml`, ADR-003.

Auto Loader

THEY ASK *"How do you make sure each file is processed exactly once?"*

YOU ANSWER

Auto Loader reads the volume's landing folders and records every file it has processed in the pipeline's checkpoint. Each morning's update picks up only the new files and stops. Two choices make it safe: file names never repeat, because they carry the Kafka epoch and offsets, and overwrites are off, so a file can't be processed twice under the same name. Unexpected fields go to a rescued-data column instead of disappearing. I tested it by running the sync and the pipeline twice: the second pass uploaded zero files and added zero rows.

IF THEY PUSH **"Why not file notifications?"** They need cloud queues that Free Edition doesn't give me, and at a few hundred files a day, directory listing is fast enough.

PROOF IN THE REPO `pipelines/ebook/bronze.py`, the double-run test in `tests/integration`.

Delta Lake

THEY ASK *"Why Delta Lake, and not just Parquet files?"*

YOU ANSWER

Parquet files can't be updated safely, and I needed things plain files can't give. Change handling: an AUTO CDC flow keeps each shopper's history as SCD type 2, and a writer's withdrawal deletes their shared result. Reproducibility: every decision memo records the table versions it used, so a time-travel query reproduces its exact numbers, and the Change Data Feed on the results table shows which decisions changed between runs. Incremental processing: each layer reads the one before it as a stream, so every run handles only what's new. On Databricks all of that is native, with Unity Catalog managing the tables.

IF THEY PUSH **"Did you need OPTIMIZE and VACUUM?"** Not at my size, and the materialized views manage themselves. I ran them as a labelled demo on the results table, with DESCRIBE HISTORY and RESTORE.

PROOF IN THE REPO `pipelines/ebook/silver.py`, `notebooks/10_ab_analysis`, ADR-006.

PySpark, on Databricks serverless

THEY ASK *"Your data is 1 GB. Why Spark?"*

YOU ANSWER

Pandas would compute it, yes, and I say that first. I used Spark for what pandas doesn't have: incremental streaming with checkpoints, watermark dedup, and code that works the same at a real store's scale. The pipeline is pure PySpark and Spark SQL: parse JSON by schema version, explode order items, hash personal data, deduplicate, build Gold. Spark stops where the data gets small: after I aggregate 1.8 million events into one row per shopper, at most about 49,000 rows per experiment, numpy and scipy run the statistics. I unit-test the transforms with local PySpark on small fixtures, so I don't spend the quota on tests.

IF THEY PUSH **"How do you tune it without the Spark UI?"** On Free Edition I use the query profile and the pipeline event log. For real Spark UI work I built a lab on AWS Glue.

PROOF IN THE REPO `src/ebook_ab/transforms`, `pipelines/ebook/`, `tests/transforms`.

AWS Glue jobs and the Spark UI (Lab)

THEY ASK *"How do you debug a slow Spark job?"*

YOU ANSWER

I open the Spark UI and go to the stages first: the slowest stage, then the tasks inside it. Free Edition has no Spark UI, so I built a lab on AWS Glue, paid by my sign-up credits: 20 million generated product views where one bestseller holds 30% of them. In the Spark UI, one task ran far longer than the median: classic skew. I compared the max-to-median task time and the shuffle bytes before and after two fixes: broadcasting the small books table, and adaptive query execution's skew-join handling, with salting as a backup.

IF THEY PUSH **"Do you use Glue in production?"** No, and I label it Lab. Glue has no free allowance, and my daily job is too small to have skew. It's practice for when data grows.

PROOF IN THE REPO `perf/spark_ui_lab`, `docs/perf/spark-ui-lab.md` with screenshots, the `lab` CDK stack.

Pipeline expectations (data quality)

THEY ASK *"How do you stop bad data from reaching users?"*

YOU ANSWER

Three layers. First, pipeline expectations generated from a YAML file: event IDs not null, timestamps valid, amounts not negative, clicks not more than views. Failing rows are dropped from Silver and written to a quarantine table with the rule's name, so nothing disappears silently, and the event log counts every pass and fail. Two rules fail the update completely: an unknown topic, which means a wiring bug, and clear personal data in Silver, which means a privacy leak. Second, a reconciliation notebook checks that counts match the producer's manifest exactly. Third, a gate notebook fails the Job if a blocking check fails, so the site keeps yesterday's numbers instead of showing wrong ones.

IF THEY PUSH **"Why not Great Expectations?"** It would work, but expectations are built into the pipeline, show up in its UI, and cost nothing extra.

PROOF IN THE REPO `config/quality/*.yml`, `src/ebook_ab/quality`, `notebooks/20_dq_report`.

Unity Catalog (governance)

THEY ASK *"How did you handle personal data and permissions?"*

YOU ANSWER

The store's events carry fake personal data on purpose (email, phone, document number), so I could design as if Colombia's Ley 1581 applied. Clear values exist only in Bronze, behind Unity Catalog column masks that only a pii-readers group can lift. Silver keeps a keyed SHA-256 hash for joins and a masked column for display; the pepper lives in a Databricks secret scope. Gold has no personal data, and the bi-readers group can read only the Gold schema. Every sensitive column carries tags for its law and purpose, and lineage shows which Gold view each column feeds. On AWS, each job has its own IAM role with only what it needs.

IF THEY PUSH **"Why not Lake Formation?"** My tables live in Databricks, so Unity Catalog is the layer that actually guards them. IAM guards the AWS side: the buckets, the Duels table, the Lambdas.

PROOF IN THE REPO `resources/grants.yml`, `sql/verify/pii_checks.sql`, ADR-005.

Notebooks, as Databricks Job tasks

THEY ASK *"Why notebooks? Aren't they bad for production?"*

YOU ANSWER

Untested logic in notebooks is bad, so my notebooks contain no logic. The math lives in a tested Python library, `ebook_ab.stats`. A notebook only loads Gold, calls the library, shows the result and writes it. They run as tasks of the daily Job, right after the pipeline, and the workflow exports each executed notebook as HTML to S3, so any decision traces back to the exact run that made it, even after Databricks drops the run history at 60 days. The ones that use only simulated and public data are published on the site. And the data-quality notebook is a gate: if a check fails, it raises an error and the Job stops before publishing.

IF THEY PUSH **"Why not plain Python scripts?"** The output is the point: a reviewer can read the tables and charts each decision was based on.

PROOF IN THE REPO `notebooks/`, `resources/ebook_daily.job.yml`, `s3://…/artifacts/runs/`.

GitHub Actions and OIDC

THEY ASK *"How is the pipeline orchestrated, and where are your keys?"*

YOU ANSWER

One GitHub Actions workflow runs the morning in order: Kafka and the simulator, the bridge, a Lambda that exports yesterday's votes, the sync into the volume, then it triggers the Databricks Job by API and waits, exports the executed notebooks, and publishes the site's data to S3 and CloudFront. It's free for a public repo. There are no AWS keys: GitHub signs in with OIDC, AWS trusts GitHub's token for my repository only, and each workflow can only assume its own role. For Databricks I used the least long-lived credential Free Edition allowed. Every pull request runs the tests, a real Kafka with the bridge kill test, the bundle validation and the cost allow-list check.

IF THEY PUSH **"Why not Airflow?"** GitHub runs Kafka anyway, and Databricks Jobs runs the Spark side; a third orchestrator would be for show. Step Functions handles the one thing with long timers: each Duel's life.

PROOF IN THE REPO `.github/workflows/`, `infra/stacks/ci.py`.

AWS Lambda

THEY ASK *"What runs on Lambda, and what doesn't?"*

YOU ANSWER

Everything short and event-driven: the Duel service's handlers, the daily export from DynamoDB, the Duel lifecycle's two steps, and a scheduled job that rotates the daily salt. All of it fits the monthly free allowance: one million requests and 400,000 GB-seconds. The Duel handlers are function URLs reached only through CloudFront, so I didn't need API Gateway, whose free tier ends after a year. My rule: Spark goes to Databricks; anything under 15 minutes and light on data goes to Lambda.

IF THEY PUSH **"How do you stop someone calling the function directly?"** The function URL uses IAM auth, and only CloudFront's origin access control can sign requests to it. The browser just adds the SHA-256 of the body, which CloudFront needs to sign a POST.

PROOF IN THE REPO `edge/src`, `lambdas/`, `infra/stacks/edge.py`.

DynamoDB (the Duel service's data)

THEY ASK *"Walk me through what happens when a reader votes."*

YOU ANSWER

The phone sends one POST through CloudFront to the vote Lambda. It checks the body against the same JSON Schema the site uses. Then it computes a hash of the date, the reader's IP and the Duel, with a salt that changes every day. One DynamoDB conditional write stores that hash only if it doesn't exist yet: that is the one-vote-per-visitor rule, atomic, with no race condition. A second write adds one to that version's counter. The IP is never stored, and the salt is destroyed at midnight UTC, so old hashes can't be linked to anyone. The server also refuses to show the answers before the Duel ends, so nobody can peek.

IF THEY PUSH **"Why provisioned capacity and not on-demand?"** The free allowance is 25 writes and 25 reads a second in provisioned mode. A vote takes about three writes, so the table absorbs a burst of readers and slows down instead of billing me. At real scale I'd switch to on-demand.

PROOF IN THE REPO `edge/src/votes.ts`, `edge/test`, decisions D34, D49 and D56.

AWS Step Functions (each Duel's life)

THEY ASK *"Where did you use Step Functions?"*

YOU ANSWER

For something with long timers: each Duel's life. When a writer creates a Duel, an execution starts. It waits until the Duel's end time, 14 days; closes the Duel if 330 readers haven't already closed it; waits until 30 days after the actual end; deletes the covers; and finishes. Every Duel's history is visible, and a failed step retries without losing the timer. It uses about 6 state transitions per Duel, and 4,000 a month are free, so about 650 new Duels a month cost nothing.

IF THEY PUSH **"Why not a nightly cleanup job?"** A cleanup job would have to scan every Duel every night to find the ones that are due. The state machine knows exactly when each one is due, and its history proves the covers really were deleted, which matters for the privacy promise.

PROOF IN THE REPO `infra/statemachines/duel_lifecycle.asl.json`, the Step Functions execution history.

CloudFront and the S3 site

THEY ASK *"How is the website hosted?"*

YOU ANSWER

It's a static React site in a private S3 bucket, served by CloudFront on its free flat-rate plan: one million requests and 100 GB a month, with no overage charges. CloudFront also routes the API path to the vote Lambda and the covers path to the covers bucket, so the browser talks to one domain: no CORS, one place for a rate limit, and only one public door. The site's data is a set of small JSON files that the daily run writes from Gold, so opening a page never runs a query, and no credential ever reaches the browser.

IF THEY PUSH **"What if it gets popular?"** The statistics are precomputed, so traffic only means serving files. The flat-rate plan doesn't bill overages; past its limits I'd move to a paid plan with a fixed monthly price.

PROOF IN THE REPO `infra/stacks/site.py`, `web/`.

AWS CDK and Databricks Bundles (infrastructure as code)

THEY ASK *"How did you create the infrastructure?"*

YOU ANSWER

All of it is code, with each platform's own tool. AWS CDK in Python defines the buckets, Lambdas, DynamoDB table, state machine, CloudFront and IAM roles, in five stacks and two stages. A Databricks bundle defines the pipeline, the Job, the schemas and the volume, with dev and prod targets pointing at two catalogs. A pull request shows the CDK diff and validates the bundle before anything changes. I added one check of my own: the build fails if a production stack contains an AWS resource type outside my free-allowance list, so nobody, including me, can add a NAT gateway or a Glue job to production by accident.

IF THEY PUSH **"Why not Terraform for both?"** Terraform covers both clouds, and I'd consider it at a company. Here each native tool was simpler: CDK packages my Lambda code, and bundles deploy notebooks and pipeline code together. ADR-010 compares them.

PROOF IN THE REPO `infra/`, `databricks.yml`, `resources/`, `tools/check_free_allowlist.py`, ADR-010.

JSON contracts

THEY ASK *"How do producers and consumers agree on the data?"*

YOU ANSWER

With JSON Schema contracts in Git. Every Kafka event, every Duel request and every file the site reads has a schema. The vote Lambda validates requests against the same files the site and Silver use. Events carry a schema version, so when version 2 added a network type field, Silver parsed both versions side by side, and unknown versions went to quarantine. The nested arrays have real jobs: one order holds several books, and one shopper is in several experiments at once.

IF THEY PUSH **"Why not Avro and a schema registry?"** At this size JSON is easier to read and debug, and contracts in Git give me review and history. ADR-004 says when I'd switch.

PROOF IN THE REPO `contracts/`, `docs/examples/`, ADR-004.

The statistics library (ebook_ab.stats, scipy)

THEY ASK *"How do you know your A/B math is right?"*

YOU ANSWER

Three automated proofs. One: the simulator plants known true effects, and the analysis must find them; over 500 simulated seeds, the 95% intervals contain the truth between 93% and 97% of the time. Two: 1,000 fake A/A splits give false alarms about 5% of the time, as they should. Three: an experiment with a deliberately broken split must fail the SRM check. The TypeScript math in the browser is tested against the same vectors as the Python, generated with scipy and statsmodels.

IF THEY PUSH **"What test did you use for revenue?"** Welch's t-test with a 10,000-sample bootstrap interval, and CUPED with pre-period revenue to reduce variance, as pre-registered.

PROOF IN THE REPO `src/ebook_ab/stats`, `tests/stats`, `tests/vectors/`.

React and TypeScript

THEY ASK *"Why build a React site instead of using a BI tool?"*

YOU ANSWER

Because the users are writers, not analysts. They need one clear answer, like "Readers chose version B", not a dashboard. The site is in Spanish and English, works on a phone, and runs the Ad Doctor and Real or luck? math in the browser, so a writer's numbers never leave their phone. A BI tool would also charge per viewer to share publicly, and Free Edition can't be used commercially.

IF THEY PUSH **"How do you test it?"** Vitest for the math and the components, a headless browser test that checks no request leaves our domain, and a click-through of every screen.

PROOF IN THE REPO `web/`, `tools/check_demo.cjs`.

3.1 Ingestion: Kafka → S3 → volume → Auto Loader

v3.0 had two ingestion paths (Path A straight from Kafka, Path B through files). v4.0 has one: the bridge. Kafka lives in GitHub Actions and on your laptop, so files are the hand-over point (D52). They land first in S3, the durable replay log, and are then synced into the Unity Catalog volume that Auto Loader reads (D62).

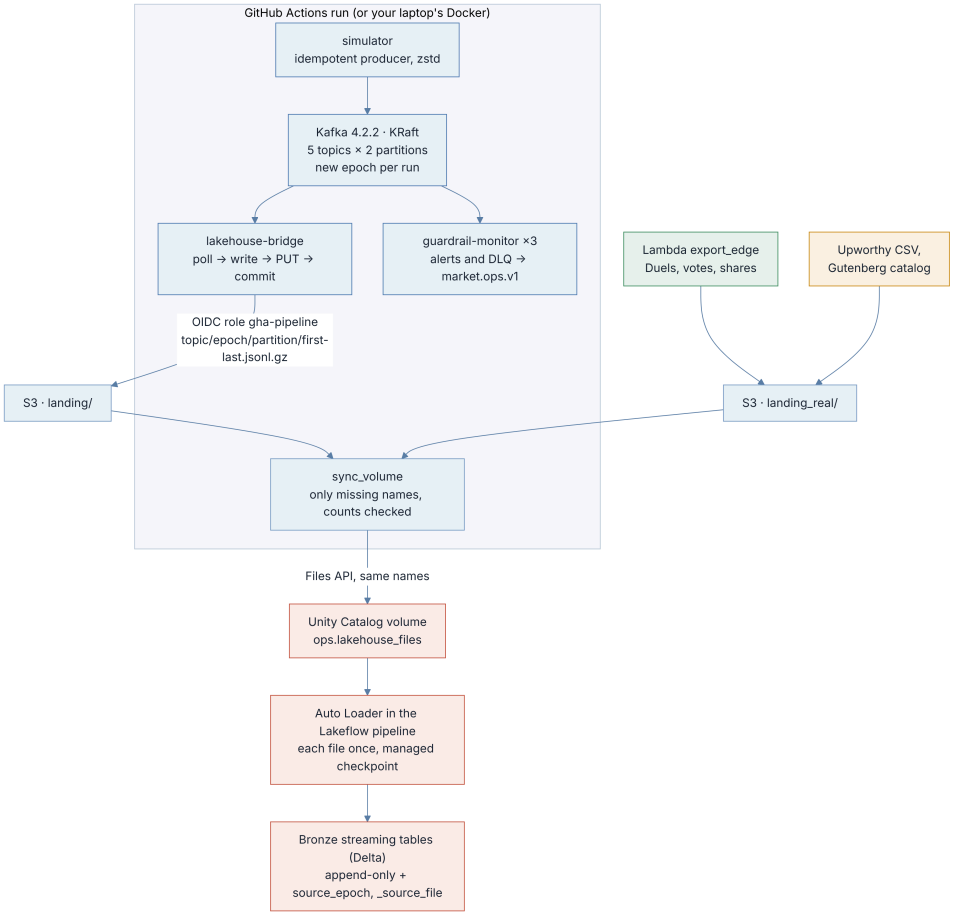


Figure 8: Ingestion: the store's Kafka runs inside the GitHub Actions run; the bridge lands files in S3; the sync step copies only the new ones into the volume; Auto Loader picks them up.

- **Where Kafka runs.**
  - Laptop: `docker/compose.yml` (Kafka 4.2.2, KRaft) for development and demos.
  - GitHub Actions: an `apache/kafka:4.2.2` service container in `ci.yml` (tests), `backfill.yml` (manual, the 92-day backfill) and `daily-e2e.yml` (one simulated day).
- **Who writes to S3.** The bridge, through the `gha-pipeline` role. On your laptop, the same bridge uses your own credentials and the `dev` prefix.
- **Files from outside the store** skip Kafka (D30): the Upworthy CSV, the Gutenberg catalog, and the daily DynamoDB export of Duels, votes and shared results. They land under `landing_real/`.

- **The sync** (`ebook_ab.tools.sync_volume`): lists yesterday's and today's S3 prefixes, lists the same folder in the volume, and uploads only the missing names with the Files API (`PUT /api/2.0/fs/files/Volumes/...`, `overwrite=false`). Same names on both sides make it idempotent: a re-run uploads nothing. After the upload it compares counts and sizes and fails the run on a mismatch.
- **Bronze picks them up** with Auto Loader inside the pipeline: each triggered update reads every new file since its last checkpoint, appends it to the matching streaming table and stops (D41). The checkpoint is managed by the pipeline, so tomorrow's run continues exactly where today's stopped.

3.2 Storage: S3 for the raw history, Unity Catalog for the tables

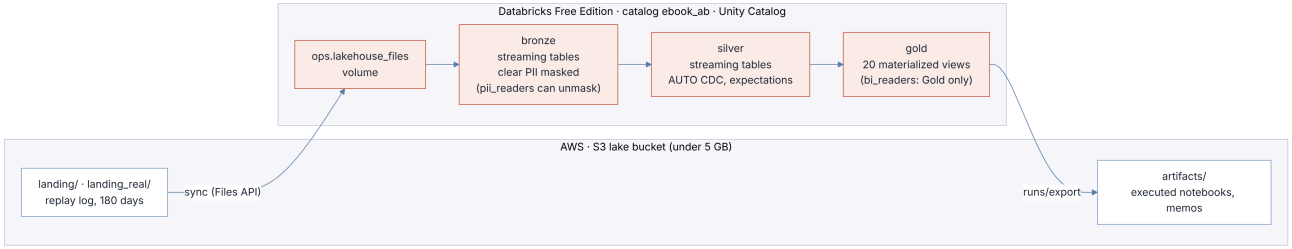


Figure 9: The lakehouse: S3 keeps the raw files and the artifacts; the volume holds the copy Auto Loader reads; Delta tables are Unity Catalog managed tables in Databricks Free Edition; Unity Catalog governs them.

Where	Holds	Lifecycle
`s3://ebook-ab-lake-<account>/landing/<stage>/`	The bridge's files: `topic/epoch/partition/first-last.jsonl.gz`	Kept 180 days (the replay log), then deleted
`s3://…/landing_real/<stage>/`	`catalog/`, `upworthy/`, `duels/`, `community/`	Kept 180 days; Upworthy and catalog forever
`s3://…/artifacts/<stage>/`	Executed notebooks (HTML, D51), memos, producer manifests, pipeline event-log extracts	Kept
Volume `<catalog>.ops.lakehouse_files`	The same `landing/` and `landing_real/` trees, written by the sync	Files older than 30 days deleted by the weekly job; S3 can always refill it
Schemas `<catalog>.bronze`, `.silver`, `.gold`, `.ops`	Unity Catalog managed Delta tables, streaming tables and materialized views	Managed by Databricks (predictive optimization where Free Edition enables it)

- **AWS storage stays under 5 GB** (well under 1 GB of landing files plus covers in their own bucket), which the CloudFront free plan's S3 credits cover (D50). The CI check in D53 fails if the lake bucket's reported size passes 4 GB.
- Block Public Access is on for the whole account. Only CloudFront reads the site and covers buckets, through origin access control. Encryption: SSE-S3, free.
- `dev` and `prod` share the bucket but never a prefix; in Databricks they are two catalogs, `ebook_ab_dev` and `ebook_ab` (A11), with separate volumes and pipelines.
- Free Edition doesn't support custom storage locations, so the tables live in Databricks' default managed storage. That is why S3 keeps the raw history: if the workspace is lost, the backfill rebuilds every table from S3 (R20).

3.3 The bridge's commit protocol: why at-least-once plus deterministic names gives zero Bronze duplicates

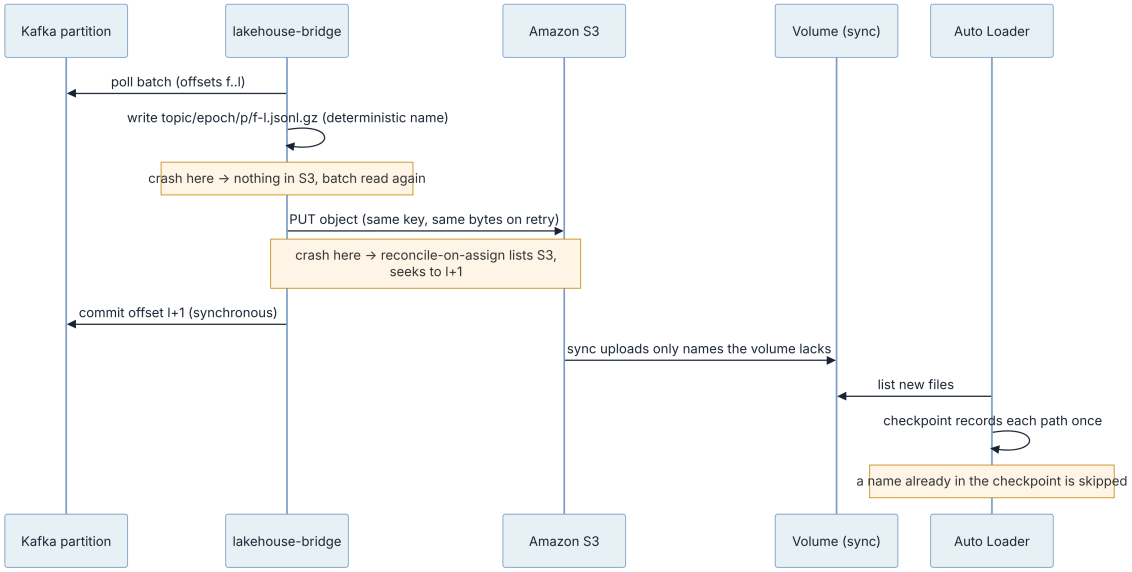


Figure 10: The bridge's commit protocol: a batch becomes one deterministic S3 object before its offsets are committed.

Argument. Each Kafka coordinate (topic, epoch, partition, offset) lives in exactly one object key, and offset ranges never overlap. The bridge can crash at three points:

- **Before upload:** nothing exists yet, so the batch is read again.
- **After upload, before commit:** on restart, *reconcile-on-assign* lists the partition's objects in S3 and seeks past the highest uploaded offset.
- **During or after commit:** normal operation.

A re-upload produces the same key with identical bytes. S3 `PUT` replaces the object as a whole, and the Bronze stream's checkpoint records each file path once, so a path is processed once.

Proof in Phase 3. Run `--crash-after-upload 3`, restart, and expect 0 duplicate coordinates and 0 offset gaps. The same test runs the sync twice and expects 0 new uploads and 0 new Bronze rows on the second pass.

3.4 Phase 0 decision rules (feasibility at $0, on both platforms)

Phase 0 proves that every piece works on the new AWS account and on Free Edition, at $0, before any phase depends on it.

Probe	What it checks	If it fails
P1	AWS: the account's plan and the console's list of allowed services include Lambda, DynamoDB, Step Functions, EventBridge Scheduler, CloudFront, S3, SNS and Systems Manager (O27)	Upgrade to the paid plan at once (D54 earlier)
P2	Databricks: create the catalog `ebook_ab_dev`, its schemas, the volume `ops.lakehouse_files`, the groups and the secret scope; a bundle deploys a pipeline and a Job (O2)	Use the workspace's default catalog with schema prefixes
P3	Databricks identity from GitHub: workload identity federation for a service principal (O34); else a service principal OAuth secret; else a PAT	The next option down (D48)
P4	The Files API from a GitHub runner uploads 100 files into the volume; a second run uploads 0; Auto Loader in a triggered pipeline reads the 100 and then 0	Upload with the Databricks CLI (`databricks fs cp`)
P5	A second pipeline (dev) beside prod, under the one-active-pipeline-per-type limit (O3); the full dev update runs inside the daily quota (O35)	Run dev and prod updates at different hours; dev on a fixture
P6	PyPI packages on serverless notebooks: scipy, statsmodels pinned in the environment (O6)	Wheel uploaded to the volume
P7	Unity Catalog: a column mask, tags, a grant to `bi_readers` on Gold only, lineage visible (O7, O8)	Masked columns written at Silver; grants only

P8	GitHub OIDC: a workflow assumes `gha-pipeline`, puts one object under `landing/` and is refused outside its prefixes	Fix the trust policy; never fall back to access keys
P9	At S1.1, right after the upgrade (flat-rate plans aren't offered on the Free plan): CloudFront flat-rate Free plan with an S3 origin, a Lambda function URL origin with origin access control (a POST with `x-amz-content-sha256`), and one WAF rate rule (O28)	API Gateway HTTP API as the origin (D49 fallback)
P10	DynamoDB in provisioned mode at 20 WCU/RCU for the table and 5 for the index; a burst of 300 votes in a minute is absorbed or throttled, never billed	Lower per-route limits
P11	A Step Functions Standard execution with a 5-minute Wait and a Lambda step; transitions counted in the console	EventBridge Scheduler one-time schedules per Duel (14 M free invocations)
P12	Cost: the probes' S3 requests and the month-to-date bill in Cost Explorer (O31)	Revisit file sizes and listings

Evidence goes to `docs/evidence/phase0/` and ADR-002, ADR-005, ADR-006 and ADR-010.

3.5 Topics, keys, contract

Topic (2 partitions; retention 72 h)	Key	Event types
`market.storefront.v1`	user_id	session_started, search_performed, product_viewed, bundle_offer_shown, added_to_cart, cart_viewed
`market.orders.v1`	user_id	checkout_started, payment_method_selected, payment_authorized, payment_failed, order_paid
`market.aftersales.v1`	user_id	book_downloaded, refund_requested, refund_issued, chargeback_opened, chargeback_resolved, marketing_opt_out
`market.experiments.v1`	user_id	assignment, exposure (both arms; control logs the counterfactual)
`market.ops.v1`	experiment_id or source topic	kind = alert or kind = dlq (flat record, no envelope)

There is no Duel topic: votes arrive as files (D30). The layout stays at 5 topics × 2 partitions; nothing limits it any more, but the monitor's rebalance demo needs exactly this shape (§3.7).

Envelope. One example; Phase 2 commits one example per topic under `docs/examples/`.


```
{"envelope_id":"0f6c…","schema_version":2,"producer":{"app":"ebook-ab-
simulator","version":"0.1.0","run_id":"bf-01","synthetic":true},
 "sent_at":"2026-10-20T14:03:11.204Z","session_id":"s-7f1e…",
 "user":{"user_id":"u_0012345","city":"Medellín","department_code":"05","segment":"avid_reader","device":"android",
        "preferred_language":"es","document_number":"99xxxxxxxx","phone":"+57 399 xxx
xxxx","email":"f.salazar@example.com","marketing_opt_in":true},
 "events":[
  {"event_id":"e-…","event_type":"checkout_started","event_ts":"2026-07-14T23:58:02.120Z","network_type":"4g",
   "experiments":[{"experiment_id":"EXP-CHK-001","variant":"treatment"},{"experiment_id":"EXP-AA-000","variant":"b"}],
   "order":{"order_id":"o-…","items":[{"line_no":1,"book_id":"pg-2000","seller_id":"s-041","price_cop":"24900.00"},
            {"line_no":2,"book_id":"pg-1342","seller_id":"s-007","price_cop":"14900.00"}],
            "bundle_id":null,"total_cop":"39800.00","currency":"COP"}},
  {"event_id":"e-…","event_type":"order_paid","event_ts":"2026-07-14T23:58:44.870Z","network_type":"4g","experiments":
[…],
   "order":{"order_id":"o-…","payment_method":"wallet","checkout_latency_ms":1850,"risk":
{"fraud_flag":false,"score":"0.0312","band":"low"}}}]}
```


Format rules:

- COP amounts are strings, later `try_cast` to `DECIMAL(18,2)`. Timestamps are ISO-8601 UTC with `Z`. Spark 4.1 runs with ANSI mode on, so every risky cast uses the `try_` form.
- Nested arrays, each with a real job: `order.items[]`, because one order holds several books (EXP-BUN-003 needs books per order); `experiments[]`, because one shopper is in several experiments at once. Silver explodes them into `order_items` and `event_experiments`.
- `events[]` batches about 4 events per message (1.8 M events in 0.42 M messages). With Aiven gone this is no longer about bandwidth; it keeps the bridge's files few and large, which keeps S3 requests (our one AWS cost) and Files API uploads low.
- Synthetic identifiers use reserved-looking ranges (O9). Book IDs are the real Gutenberg numbers.

Assignment. `bucket = int(sha256(f"{experiment_id}:{salt}:{user_id}").hexdigest()[:15], 16) % 100`. Buckets 0–49 are control and 50–99 treatment. The Spark twin is `pmod(CAST(conv(substr(sha2(concat_ws(':', exp, salt, user_id), 256), 1, 15), 16, 10) AS BIGINT), 100)`. Parity is unit-tested.

Exposure triggers: CHK: checkout opened. PDP: first view of a product page in the test set (the 200 most-viewed titles). BUN: bundle-slot evaluation on a product or cart page; control logs `counterfactual=true`. AA-000: product page view. SRM-900: checkout opened.

3.6 Datasets by layer

Bronze (streaming tables in `<catalog>.bronze`, append-only, Auto Loader):

- Kafka sources: `storefront_raw`, `orders_raw`, `aftersales_raw`, `experiments_raw`, `ops_raw`. Columns: `key, value, topic, partition, offset, kafka_timestamp, source_epoch, _source_file, _ingested_at`.
- File sources from `landing_real/`: `catalog_raw` (Gutenberg RDF extract as JSONL), `upworthy_raw` (the CSV as-is), `community_raw` (shared results and withdrawals, A15), `duels_raw` and `duel_votes_raw` (A19).
- Bronze is never fully refreshed in production; the S3 landing files are its backup (R11).

Silver (streaming tables in `<catalog>.silver`):

- Events: `storefront_events`, `order_events` (keeps items as an array), `aftersales_events`, `experiment_events`.
- Child tables: `order_items` (order_id, line_no), `event_experiments` (event_id, experiment_id).
- Dimensions: `books` (title, author, language, subjects, download weight, seller, price); `users_scd2` (AUTO CDC, SCD type 2, D43).
- Real data:
  - `real_headline_packages`: typed Upworthy rows, with the problem flag kept;
  - `community_results`: validated shared results, one row per result_id, flagged ok, outlier or duplicate; a withdrawal deletes its row (AUTO CDC with `apply_as_deletes`, D43);
  - `duels`: one row per Duel (kind, genre, book language, target, status, library opt-in, idea tag), AUTO CDC SCD type 1;
  - `duel_votes`: one row per accepted vote (Duel, version, answer, minute), after dropping double votes and bot-like bursts; the daily visitor hash is used for that and then dropped.
- Ops and quality: `ops_alerts`, `ops_dlq`, `quarantine` (coordinates, raw value, reason = the expectation's name).

Gold: 20 materialized views in `<catalog>.gold` (§7), including `duel_results` and `duel_library`, plus the private view `experiment_registry`. Delta tables written by notebooks beside them: `experiment_results`, `real_prior` (A6) and `community_summary` (A16).

Ops schema (`<catalog>.ops`): `run_log` (one row per workflow step and Job task), `producer_manifests`, `dq_reconciliation`, `validation_results`, `cost_log`, and `simulation_truth` (read only by validation; granted to nobody else).

3.7 Offsets and consumer groups

- **lakehouse-bridge:**
  - A real group: `enable.auto.commit=false`, `auto.offset.reset=earliest`, cooperative-sticky assignment.
  - Synchronous commit after a durable S3 upload, plus reconcile-on-assign (§3.3).
  - In GitHub Actions it runs until the topics are drained (end offsets reached), then exits so the workflow can continue.
- **guardrail-monitor:**
  - An independent group on the storefront, orders and aftersales topics.
  - Range assignor with 3 members and 2 partitions: one member stays idle. Killing an active member triggers a rebalance.
  - At-least-once: `enable.auto.offset.store=false`, then `store_offsets()` after processing.
  - Hourly event-time windows per experiment × variant compute fraud-flag rate, refund rate, p95 checkout latency and payment failure rate.
  - Output: `kind=alert` records with a deterministic alert_id, and `kind=dlq` records with raw base64, both to `market.ops.v1`. Per-partition lag is logged every 30 s, with a lag alert.
- **Lag:** both groups log it; the bridge's last lag line goes into the run's producer manifest, which Silver reconciles against.

3.8 Ingestion options (justified)

Option	Value	Why
Source	`cloudFiles` (Auto Loader), `cloudFiles.format` = `json` or `csv`, with an explicit schema per source	Fixed contracts; no schema inference
Path	`/Volumes/<catalog>/ops/lakehouse_files/landing/<topic>/` and `…/landing_real/<source>/`	The volume is what Free Edition can read (D41, D62)
Discovery	Directory listing (the default)	File notification mode needs cloud queues Free Edition doesn't offer; our file counts are small
`pathGlobFilter`	`*.jsonl.gz` (landing)	Ignores stray files
`cloudFiles.maxFilesPerTrigger`	1000	The 92-day backfill lands in one or two micro-batches
`cloudFiles.allowOverwrites`	`false`	A file name is processed once; the sync never overwrites
`cloudFiles.schemaEvolutionMode`	`rescue`	Unexpected fields go to `_rescued_data`, not silently dropped
Pipeline mode	Triggered, started by the Job (A14)	Processes everything new, then stops; continuous mode would burn the daily quota
Checkpoint	Managed by the pipeline	Survives between runs with no setup

3.9 Silver design

- **Parsing.**
  - `get_json_object(value, '$.schema_version')` routes v1 and v2 to their schemas, followed by `unionByName(allowMissingColumns=True)`.
  - Unknown versions and malformed JSON go to quarantine.
  - `try_cast` / `try_to_timestamp` handle amounts and timestamps (ANSI on in serverless).
  - `from_json` with explicit schemas; `VARIANT` is not used, so `site_export` and the Lab can read every column as plain types.
- **How Silver reads Bronze:** streaming tables in the same pipeline, each reading its Bronze table with `spark.readStream.table(...)`; the pipeline manages the order and the checkpoints.
- **Data quality** (pipeline expectations, D44), the rules kept in `config/quality/*.yml` and turned into decorators by `ebook_ab.quality`:

Severity	Rules	What happens
`expect_or_fail`	known_topic; no_clear_pii (no value matches an email, phone or document pattern)	The update fails before Gold; the Job fails; GitHub and Databricks e-mail you. A wiring bug or a PII leak must stop the flow.
`expect_or_drop` + quarantine flow	event_id not null, valid event_ts, valid event_type, amount ≥ 0, user_id not null; for shared results: known enums, 100 ≤ views ≤ 10,000,000, clicks ≤ views, views ratio < 1.5	Failing rows are dropped from Silver; a mirror flow with the inverted condition writes them to `silver.quarantine` with the rule's name.
`expect` (warn)	latency 50–60,000 ms, event_ts not in the future, late arrival ≤ 30 min	Kept; counted in the event log and the DQ report.

- **Dedup** (D10): `withWatermark("event_ts", "1 hour")` + `dropDuplicatesWithinWatermark(["event_id"])`. The producer only re-sends within minutes, so 1 hour is wide; the backfill is reconciled exactly against the producer manifest in `ops.dq_reconciliation`.
- **SCD2 users** (D43):
  `dp.create_auto_cdc_flow(target="users_scd2", keys=["user_id"], sequence_by=struct("observed_at","topic","offset"), stored_as_scd_type=2, track_history_column_list=[segment, city, department_code, device, preferred_language, marketing_opt_in])`
  test checks one current row per user and contiguous intervals.
- **PII** (D11, D45):
  - Keyed `sha2(concat(pepper, ':', normalized), 256)` for join keys. The pepper lives in the Databricks secret scope `ebook` (key `pii_pepper`), read with `dbutils.secrets.get`, which redacts it in output.
  - Masked display columns written at Silver (for example `f.s•••@example.com`), plus Unity Catalog column masks on Bronze's clear columns: only `pii_readers` see them unmasked.

  - Tags on every sensitive column: `pii`, `ley_1581`, `ley_1266`, `purpose` (Unity Catalog tags), listed in the data dictionary.
- **Books dimension.** Gutenberg fields plus simulated seller and price. Real downloads becomes the demand weight; it is never shown as sales.
- **Duel votes.**
  - Keep the first vote per Duel, visitor hash and day (the vote Lambda already refuses repeats; this is the second line).
  - Flag bursts: more than 20 votes an hour from one hash, or a Duel jumping far above its usual pace.
  - Check the split per Duel (SRM, χ^2 on the two versions' counts).
  - The writer's own device never sends a vote, because the reader page checks a local flag.
  - The visitor hash is not kept past Silver.

3.10 Gold factory

- `config/gold_views.yml` defines the 20 views. Each entry has: name, domain, grain, sources, consumer (required, checked in CI), description, `cluster_by`, columns, and SQL.
- `pipelines/ebook/gold_factory.py` loops over the specs and registers one `@dp.materialized_view` per entry (D57). The pipeline works out the dependency order from the reads; serverless refreshes incrementally when it can.
- Safeguards:
  - Portable SQL: the SQL stays plain Spark SQL, so CI analyzes every view on a small fixture with local PySpark.
  - Totals check: a private view `gold.check_totals` compares Gold counts with Silver counts, with `expect_or_fail` on it.
- Around the factory:
  - `tools/gen_data_dictionary.py` generates `docs/data_dictionary.md` from the YAML and the tags, and CI fails if it is stale.
  - Timestamps are converted to Bogotá time in Gold only.
  - Grants: `bi_readers` get `SELECT` on `<catalog>.gold` only (D45).

3.11 Analysis method (notebooks `10_ab_analysis`, `11_validation`; library `ebook_ab.stats`)

1. **Validity gates**, before any outcome is looked at:
   - SRM χ^2 ($p < 0.001$ = invalid);
   - exposure-based population;
   - invariants: identical risk-engine decisions by band (CHK) and identical prices shown (PDP, BUN);
   - A/A on EXP-AA-000.
2. **Tests:**
   - two-proportion z-test (pooled test, unpooled CI, delta-method relative CI);
   - Welch + 10,000-sample user-level bootstrap with a fixed seed, for COP revenue;
   - cluster bootstrap for p95 latency;
   - CUPED with pre-period covariates;
   - delta method for ratios;
   - non-inferiority via one-sided 95% bounds;
   - Holm for secondaries; BH q = 0.10 for segments.
3. **Outputs:** a `MERGE` into `gold.experiment_results` (with metric_role, p_value_adj, method, data_cutoff_ts, analysis_run_id, code_version, source_snapshot_id); memos in `docs/memos/`; the executed notebook as HTML in `artifacts/runs/<run_id>/`.
4. Only then does `11_validation` load the truth.

Where it runs. As notebook tasks of the `ebook_daily` Job on serverless (D51), after the pipeline update: Python 3.12, numpy, scipy, pandas, pyarrow and statsmodels pinned in the serverless environment (O6). Gold is read with `spark.table(...).toPandas()` (the views are small). statsmodels also runs in CI as the oracle. The heavy proofs (500-seed coverage, 1,000 re-salted A/A splits) run in CI on fixtures and in `11_validation`, vectorized with numpy; if they press on the daily quota, they move to a weekly run (R2).

Proofs:

- (a) Truth falls inside each 95% CI. A 500-seed coverage simulation must land in [0.93, 0.97].
- (b) 1,000 re-salted A/A splits give an FPR in [3.65%, 6.35%] and KS $p > 0.01$.
- (c) EXP-SRM-900's SRM $p < 0.001$; clean experiments ≥ 0.001.

Bayesian readouts are used only by the public tools (A13).

3.12 Real-data track (Q9)

- **Upworthy Research Archive** (CC BY 4.0):
  - The exploratory sample is `upworthy-archive-exploratory-packages-03.12.2020.csv`: 14,260,949 bytes, 22,666 packages, 4,873 tests (OSF listing, checked 2026-10-15).
  - Each row is a package: a headline/image bundle shown at random, with impressions and clicks. The archive holds aggregates only.
  - We exclude problem = 1 (tests from 2013-06-25 to 2014-01-10, which the archive discourages).
- **Pipeline:**
  1. `aws s3 cp` the CSV to `landing_real/<stage>/upworthy/`; the sync copies it into the volume.
  2. `bronze.upworthy_raw` (Auto Loader, CSV).
  3. `silver.real_headline_packages` (typed).
  4. Gold `real_headline_effects`: each package's log-odds ratio vs the test's earliest package, with its standard error.
  5. Gold `real_headline_tests`: a per-test summary.
- **Notebook `12_real_headlines`:**
  1. Fit a random-effects model (DerSimonian–Laird and REML) to get the mean and spread (τ) of true effects.
  2. Report the share of tests with a clear winner after Holm.
  3. Measure the winner's curse: how much the observed best lifts shrink.
  4. Compute the traffic needed to detect typical effects.
  5. Run a sensitivity analysis on impression filters (≥500 / 1,000 / 2,000).
- **Outputs:** `gold.real_prior` (μ, τ, method, filters, n_tests); `docs/reports/headline-tests.md`; the site's *Real lessons* page; the executed notebook, published.
- **Limitation, stated on the site:** news-headline click tests, not ebook purchases. The prior is the default only for title, cover and copy tests; users can switch to a neutral prior.
- **Gutenberg catalog:** per-book RDF metadata is CC0 and includes 30-day downloads. We download the offline RDF catalog once and select about 2,000 Spanish and English titles by downloads. No book texts, no cover images.
- **Writers' shared results** (D23–D27, now D49):
  - One shared record holds only: genre (a fixed list of about 10, plus "other"), book language (es, en), change (cover, title, blurb, price display, bundle), an optional idea tag from a fixed list, and views and clicks for ad A and ad B. The Lambda adds a random result_id, the date received, a sequence number and the schema version. No names, emails, book titles, free text or IP addresses.
  - Pipeline: the share route validates the record and stores it in DynamoDB (§3.13) → `export_edge` writes records and withdrawals received since the last export as JSONL to `landing_real/community/<date>.jsonl.gz` → `bronze.community_raw` → `silver.community_results` (quality rules above; duplicates and outliers flagged and kept out of summaries; withdrawals applied as deletes by AUTO CDC).
  - Notebook `13_community_summary`, for each genre × book language × change, pooling the ended Duels that writers added to the library (`gold.duel_results`) and the shared ad tests: counts of results, clear wins and losses by source; the median lift and middle half; a random-effects mean with its 95% range, pulled toward the change type's mean when a cell is small; idea tags used by at least 5 tests as "W of N"; and for cells under 5 results, a "not enough results yet" mark with the number still needed (D26). It writes `gold.community_summary`; the site export turns it into `library.json`.
  - Expected size: small. Even 10,000 shared results are a few MB in DynamoDB, far inside the 25 GB free storage.
- **Duels and their votes** (L6, L7; D30, D34–D37):
  1. The Duel service stores Duels and votes in DynamoDB (§3.13).
  2. `export_edge` writes what is new since the last export as JSONL: Duels to `landing_real/duels/duels-<date>.jsonl.gz` and votes to `landing_real/duels/votes-<date>.jsonl.gz`.
  3. `bronze.duels_raw` and `bronze.duel_votes_raw`.
  4. `silver.duels` (AUTO CDC, one row per Duel) and `silver.duel_votes` (first vote per Duel, visitor hash and day; bursts flagged and dropped; the hash dropped after use).
  5. Gold `duel_results` and `duel_library`, which feed `13_community_summary`.
  - Why files and not Kafka (D30): the votes leave DynamoDB once a day and are few, so a file per day is the simple, exactly-once path.

3.13 Website, the writer tools and the Duel service

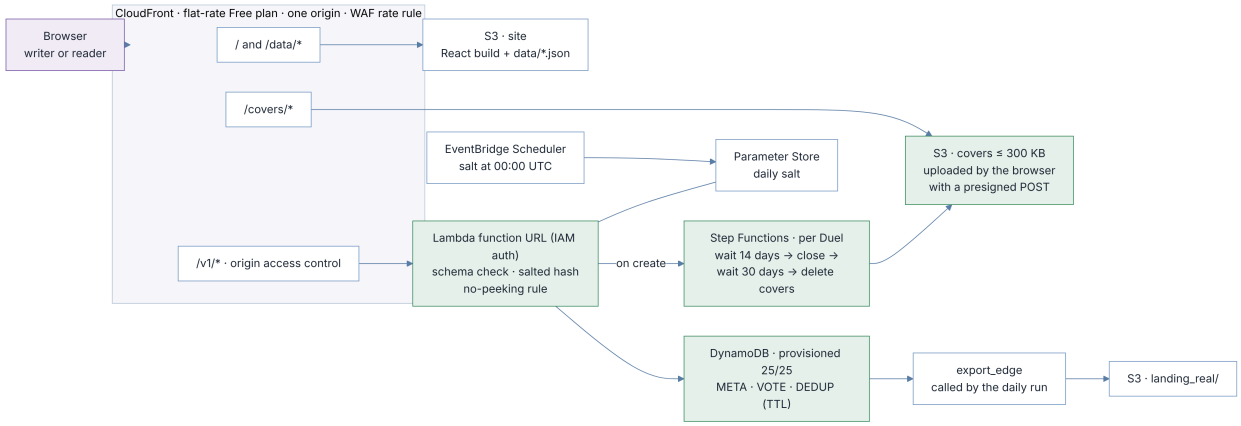


Figure 11: How the site, covers and the Duel API share one CloudFront origin, how each Duel's life is timed, and how votes reach the lakehouse once a day. No AWS credentials in the browser.

- **Stack.** Vite + React + TypeScript, Apache ECharts for charts, i18next for Spanish/English, Vitest + Testing Library for tests. All MIT or Apache-2.0.
- **The look (L11).** Warm paper colors and dark mode, regular-weight type, pill buttons, and app windows on paintings drawn in code. The fonts are free and served from our own origin: Instrument Sans, EB Garamond (book text), Geist Mono.
- **The voice (L9).** Plain, kind, direct sentences, in the writer's words. For example: "Too close to call. Both work. Pick the one you love and stop worrying."
- **Pages.** Unchanged from v3.0; the demo (`docs/plan/DEMO.html`) shows each with example data. Every number block wears a source badge (D28).
  1. **Home**: three doors (L5): "Which cover?" opens a Duel; "Losing money on ads?" opens the Ad Doctor; "Did my change work?" opens Real or luck?. A public counter: "N results so far" (C11).
  2. **Duel** (L1, D34–D37):
     - *Create*: what are you choosing (cover, title, blurb or book idea), the two versions, the genre and the book's language. Covers are shrunk in the browser to at most 300 KB and uploaded straight to S3 with a presigned POST (D35, D49). Optional: reward voters with the first page, and add the writer's own "tell me when it's out" link.
     - *Share*: one unlisted link. "The result opens at 330 readers. That's enough to see a clear difference."
     - *Readers vote*: the browser picks the version from a hash of the Duel and a random number that never leaves the device. One tap: "Would you read the first page?" The thank-you page shows the first page, the writer's link and "Do you write? Test yours free". The writer's own device is never counted.
     - *Progress*: readers so far and the fair-split check, never the winner. The Duel ends at 330 readers or after 14 days.
     - *Reveal*: one of four endings (D34): "Readers chose version B"; "Too close to call: both work"; "Not sure yet: about N more readers"; "Not fair yet". For the curious: the difference with a 95% range and the split check. A ready image to post. A check box adds the result to the library, anonymously.
     - What a Duel measures: readers' interest in the first page, in a fair test. Not sales. The page says so.
  3. **Which would you read?** (L2, D36): one random version from another writer's Duel per card; each vote counts in that Duel and earns one vote for your own. You approve which Duels enter the game. Every card has a report button.
  4. **Ad Doctor** (L3): an Amazon Ads search-term report (CSV, English or Spanish columns, commas or semicolons), what the writer earns per sale and the days in the report. Each keyword gets Stop (≥ 10 clicks, 90% sure it earns less than it costs), Keep (≥ 10 clicks, 90% sure it earns more), Luck (looks great from fewer than 15 clicks) or Wait (about N more clicks). The headline: "You can stop spending about USD X a month". Beta-binomial, each keyword starting from the book's own average as if it already had 20 clicks. Meta Ads is judged on cost per click. The file never leaves the browser.

5. **Real or luck?** (L4): sales and days, before and after a change; an exact binomial test adjusted for the days: “It’s real” (p < 0.05) or “You can’t tell yet”, with the days of the same pace that would settle it. It also lists other things that move sales, and points to a Duel for covers and titles.
6. **The library** (L6): filters by genre and book language, then “What are you about to change?”. An evidence card shows the count, how often the change won, the typical gap with its middle half, a dot strip and plain advice; ideas as “won W of N”; the ranking once each change has 5 results; launch-day and small groups show “Not enough results yet: be one of the first” (D26).
7. **Plan an ad test** and 8. **Read your ad results** (inside, kept from v2.3): the planner never plans fewer than 7 days (D29); the “?” help says where each number lives in Meta, Amazon Ads, BookBub and TikTok (C5); the reader gives Use B / Keep A / Not sure yet / Too close / Not fair yet, with details folded, “what this doesn’t tell you”, the library comparison once it has 5 results (C7), and Share for every fair verdict (C10) with a receipt code (D27).
8. **Real lessons** (C8): the conclusion first, then three lessons with “what this means for you”, from the Upworthy analysis.
9. **Behind the scenes: our test store** (C9): one plain sentence per decision, the checks on the checks, readouts, guardrails, memos, a data-health badge, “data as of”, and links to the executed notebooks 10–11 (D51).
10. **For nerds** (N1): the hybrid map of section B, a card per box (what it does, where it lives in the repo, why it is built that way, what it costs, which free allowance or free plan it stays inside), five guided stories (a reader’s vote, a simulated shopper, the real headline tests, the 06:00 run, an Ad Doctor report), filters, the stack with the verdicts of §3.0. Motion pauses on request and respects reduced motion.
11. **Methods**: the three badges, assumptions and formulas, the Duel rules (D34), how sharing and voting work, a link to the repo.

- **The Duel service** (`edge/`, A17, A21, D49, D56, D59, D60): Lambda function URLs (TypeScript, Node.js LTS) behind CloudFront + DynamoDB + the covers bucket + the Duel lifecycle state machine.
  - Routes, all under `/v1` on the site’s own domain (CloudFront → the `duel-api` function URL, signed by origin access control; the browser adds the body’s SHA-256 in `x-amz-content-sha256`):
    - `POST /v1/duels` creates a Duel, starts its lifecycle execution (D60), and returns its public link, the writer’s private link (an owner key, kept only as a hash) and two presigned POST forms for the covers;
    - `GET /v1/duels/{id}` and `GET /v1/duels/{id}/versions/{arm}` serve a Duel and one version (covers come from `/covers/…`); the browser picks the version, so its random number never leaves it;
    - `POST /v1/duels/{id}/votes` takes `{arm, answer}`; the 330th reader closes the Duel;
    - `GET /v1/duels/{id}/progress` (owner key) returns readers per version for the split check, never the answers;
    - `GET /v1/duels/{id}/result` (owner key) returns the answers only once the Duel has ended. The server enforces “no peeking”;
    - `POST /v1/duels/{id}/library` adds an ended Duel to the library (opt-in);
    - `GET /v1/game/next` serves the game’s next card, and votes given there add credit to the voter’s Duel;
    - `POST /v1/duels/{id}/report` reports a Duel;
    - `POST /v1/results` and `POST /v1/withdraw` keep the share and its receipt code;
    - admin (list, approve, remove for the game) lives on a separate `duel-admin` function URL with IAM auth, outside CloudFront (D59).
  - DynamoDB, provisioned mode inside the free allowance (table 20 WCU / 20 RCU, export index 5 / 5, auto scaling off), one table with typed keys (single-table design):
    - `DUEL#<id>` / `META`: owner key hash, kind, genre, book language, target, created, ends, status, in game, in library, idea tag, counters per version (readers, yes);
    - `DUEL#<id>` / `VOTE#<seq>`: version, answer, minute, visitor hash (removed after the export);
    - `DEDUP#<hash>`: the one-vote and rate-limit markers, with a 24-hour TTL (D56);
    - `RESULT#<id>`, `CREDIT#<duel>`, `REPORT#<id>`;
    - a sparse index on `export_day` lets `export_edge` read only yesterday’s items.
  - One vote per visitor per Duel: a conditional `PutItem` on `DEDUP#HMAC(daily salt, date + IP + Duel)` with `attribute_not_exists`, then an atomic `ADD` on the version’s counter. The IP is never stored. An EventBridge Scheduler rule calls `salt_rotate` at 00:00 UTC.

  - The Duel lifecycle (D60), Step Functions Standard: `Wait` until `ends_at` → `close_duel` (Lambda; no-op if the 330th reader already closed it) → `Wait` until `ended_at + 30 days` → `delete_covers` (Lambda) → `Succeed`. A lifecycle rule deletes any cover older than 45 days as a backstop.
  - Covers: private bucket `ebook-ab-covers-<account>`; the presigned POST enforces at most 300 KB and an image content type.
  - Limits: request bodies at most 16 KB (covers go straight to S3; the WAF inspects 16 KB on the Free plan); every body checked against `contracts/duels/*.schema.json`, the same schemas the site and Silver use; at most 5 Duels and 5 shares per visitor a day; reserved concurrency 10 on `duel-api`; a WAF rate rule on `/v1/*`; no request-body logging. When DynamoDB throttles, the site says "Lots of readers right now. Try again in a minute."
  - Deploy and tests: deployed by CDK (`edge` stack). Unit tests mock the AWS SDK (aws-sdk-client-mock); integration tests run against DynamoDB Local in Docker and then against the dev stage; the lifecycle runs in dev with 5-minute waits.
- **Stats in TypeScript** (`web/src/stats`, no dependencies): normal CDF and quantile; the z-test and its CI; sample size and days still needed; the views-ratio check; SRM χ^2; a normal–normal posterior on the log-odds ratio with closed-form expected loss; the Ad Doctor's beta-binomial and Real or luck?'s exact binomial test. They are tested against shared vectors in `tests/vectors/*.json`, which the Python library generates with scipy/statsmodels as the oracle.
- **Data for the site:**
  - `ebook_ab.site_export`, the last step of the daily workflow, runs on the GitHub runner: it reads Gold through the Databricks SQL Statement Execution API on the Free Edition SQL warehouse, validates against `contracts/site/*.schema.json`, writes `data/*.json` to the site bucket and invalidates `/data/*` in CloudFront.
  - `library.json` holds counts and summaries per group, idea tags with 5 results or more, rounded gaps for the dot strip, and the total for the public counter. Never single votes, views, clicks, dates or IDs (D27).
  - A frozen snapshot is committed in Phase 1, so the site always builds.
- **Deploy:** `site.yml` builds with Vite, runs `aws s3 sync` to the site bucket through `gha-deploy`, and invalidates CloudFront.
- **Quality:** keyboard and contrast checks; mobile first; light and dark themes; reduced motion. A headless-browser test asserts that no request leaves the site's origin (the API is on the same origin now), and that the Ad Doctor, Real or luck? and the planner send nothing. A build with no real results shows "0 of 5" everywhere. Duel service tests: bad payloads rejected; a second vote from the same visitor refused; the result route refuses before the end; the 6th Duel or share in a day refused; a withdrawal works; covers deleted by the lifecycle (fast clock in dev); the split check flags a broken split. A build-size budget (< 5 MB).

3.14 Orchestration: the daily run

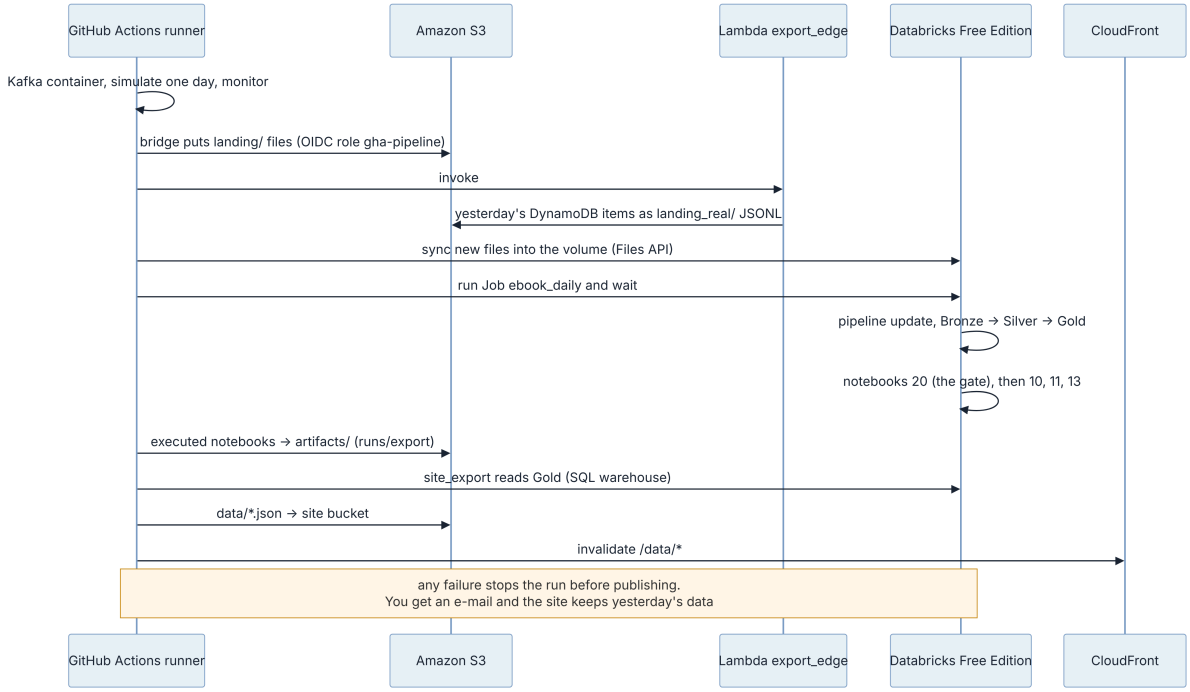


Figure 12: The 06:00 run: GitHub Actions runs Kafka, the bridge, the export and the sync, then triggers the Databricks Job and publishes the site.

GitHub Actions, `daily-e2e.yml` (cron 11:00 UTC = 06:00 Bogotá; timeout 90 min; one run at a time; OIDC role `gha-pipeline`; Databricks credential per D48):

Step	Where	What it does
1. kafka	service container	`apache/kafka:4.2.2`; `topics ensure`
2. simulate	runner	One simulated "today" (about 2,000 active shoppers, shipped variants on); the guardrail monitor consumes in parallel
3. bridge	runner → S3	Drain the topics into `landing/prod/` with epoch `gha-<run_id>`; write the producer manifest
4. export_edge	Lambda (invoked)	New Duels, votes, shared results and withdrawals from DynamoDB → `landing_real/` (D30)
5. sync	runner → volume	New S3 files → `ops.lakehouse_files` with the Files API; counts checked (D62)
6. run Job	Databricks	`databricks bundle run ebook_daily` (Jobs API `run-now`), then wait for the result
7. export notebooks	runner → S3	Executed notebooks as HTML through `runs/export` → `artifacts/runs/<run_id>/` (D51)
8. site_export	runner → S3	Gold through the Statement Execution API → `data/*.json`; CloudFront invalidation; `freshness.json`
9. log_run	runner → Databricks	One row per step in `ops.run_log`

The Databricks Job `ebook_daily` (defined in the bundle, serverless, at most 5 concurrent tasks):

Task	Type	Depends on
`pipeline_update`	Pipeline task: one triggered update of `ebook_pipeline` (Bronze → Silver → Gold, D40)	—
`nb_20_gate`	Notebook: validity gates; fails the Job if a blocking check fails	`pipeline_update`
`nb_10_analysis`, `nb_13_community`	Notebooks, in parallel	`nb_20_gate`
`nb_11_validation`	Notebook	`nb_10_analysis`
`nb_12_real`	Notebook, only when Upworthy files changed (`if/else` task on a flag)	`pipeline_update`

- A failed step or task fails the workflow; GitHub and Databricks e-mail you (D15), and the site keeps yesterday's data.
- `weekly-maintenance.yml` (Sundays): deletes volume files older than 30 days; a Cost Explorer snapshot to `ops.cost_log` (O31); pipeline event-log extract to `artifacts/`.

- **Terms and limits:** GitHub-hosted runners may be used for the project's production, testing, deployment or publication; a daily run that refreshes the published site fits, and nothing runs 24/7. Scheduled workflows can start late at busy times; ours has no deadline. Schedules in public repos are disabled after 60 days without repo activity; documented, with one-click re-enable (R29). Free Edition's daily quota: the daily run must stay well under it; Phase 3 measures it (O35).

3.15 Security and access

AWS (IAM):

Identity	Who uses it	Can do	Cannot do
Root user	Nobody, day to day	Billing and account recovery; MFA on	Everything else is done by `andres-admin`
`andres-admin` (IAM user, MFA)	You, in the console and CLI	Administer the account	— (IAM Identity Center isn't on the Free plan)
`gha-ci`	GitHub, any branch	`cdk synth`, read-only checks	Change anything
`gha-deploy`	GitHub, `main` only	`cdk deploy`, sync the site, invalidate CloudFront	Read the lake bucket
`gha-pipeline`	GitHub daily, weekly and backfill workflows	Its prefixes in the lake bucket, invoke `export_edge`, write `data/` in the site bucket	Touch the Duels table or the covers
`edge-*`	The Duel Lambdas	Read and write the Duels table and the covers bucket; read the salt; start lifecycle executions	Touch the lake bucket

Databricks (Unity Catalog):

Principal	Who	Can do	Cannot do
You (workspace admin)	Development	Everything in `ebook_ab_dev`; read `ebook_ab`	—
`sp-ebook-ci` (service principal)	GitHub workflows	Deploy the bundle, run `ebook_daily`, write the volume, read Gold through the warehouse	Read Bronze's clear columns
`bi_readers` (group)	Anyone you give Gold to	`SELECT` on `<catalog>.gold`	Silver, Bronze, any PII
`pii_readers` (group)	Audits	See Bronze's clear columns unmasked (column mask)	Change data

- cdk-nag runs on every synth; IAM Access Analyzer (free for external access) runs after each deploy.
- Secrets: the pepper in the Databricks secret scope; the daily salt in Parameter Store (SecureString, AWS-managed key). GitHub holds only role ARNs (not secret) and, at worst, one Databricks credential (D48).

3.16 Observability, performance, Delta

- **The daily run:** the GitHub Actions log of every step; the Job run page (task durations, retries); the pipeline event log (row counts, expectation results, flow durations), queried in `ops` and extracted weekly to S3; the query profile of any slow statement; `ops.run_log` as a table.
- **The edge (CloudWatch, inside the free tier):** Lambda errors and duration, DynamoDB throttles, Step Functions failures; alarms → SNS e-mail (at most 10 alarms).
- **Cost:** the two AWS budgets (D53) and `ops.cost_log`; for Databricks, the quota: run durations in `ops.run_log`.
- **Spark UI lab on Glue (Lab, D61):** 20 M generated product views where one bestseller holds 30% of them; before/after numbers for a broadcast join (books dimension), a skew fix (AQE skew join or salting) and file sizing, in `docs/perf/spark-ui-lab.md`, with screenshots of Glue's serverless Spark UI. Glue 6.0, Flex class, 10 G.1X workers for about 15 minutes ≈ $0.75 of credits per run at list price. Free Edition serverless has no Spark UI, so the daily run is tuned with the query profile instead.
- **Delta:**
  - real: AUTO CDC for SCD2 and withdrawals; `MERGE` into `experiment_results`; time travel to reproduce a memo's exact numbers (`VERSION AS OF <version>`); the Change Data Feed on `experiment_results` that shows which decisions changed between runs; streaming tables between layers;
  - Lab: `OPTIMIZE`, `VACUUM`, `DESCRIBE HISTORY` and `RESTORE` demos on `experiment_results` (D58), and Z-ORDER on the Glue perf lab's 20 M rows.

3.17 CI/CD

- **Databricks Declarative Automation Bundle** (`databricks.yml` + `resources/*.yml`): the pipeline, the Job, the schemas, the volume; targets `dev` (catalog `ebook_ab_dev`) and `prod` (`ebook_ab`) (D47).
- **AWS CDK app** (`infra/`, Python): stacks `foundation`, `edge`, `site`, `ci`, and `lab` (manual only); stages `dev` and `prod`. Lambda code is packaged as CDK assets.
- **GitHub Actions** (standard `ubuntu-latest` runners, free for public repos):
  - `ci.yml` (every PR): Python ruff, unit, stats and contract tests; Kafka integration with an `apache/kafka:4.2.2` service container and the bridge kill test; local PySpark tests for the transforms and the Gold SQL on fixtures; `databricks bundle validate`; web tsc, Vitest with the shared vectors, build, the no-third-party-requests check; edge tests with DynamoDB Local; `cdk synth` + cdk-nag; **the free-allowance allow-list check** (D53).
  - `deploy.yml`: push to `main` → `bundle deploy -t dev` and `cdk deploy` to `dev`; a tag `v*` → `prod`.
  - `site.yml`: build and publish the site.
  - `backfill.yml` (manual): the 92-day backfill through a CI Kafka, then steps 4–9 of the daily run (a full refresh only in `dev`).
  - `daily-e2e.yml` and `weekly-maintenance.yml`: §3.14.

3.18 Compliance and ethics by design

- **Ley 1581 de 2012 (personal data):**
  - Minimization: hashes and masks; Gold carries no PII.
  - Purpose limitation: `purpose` tags.
  - Least privilege: `bi_readers` see Gold only; clear values only in Bronze, behind Unity Catalog column masks that only `pii_readers` lift.
  - Audit columns: Kafka coordinates, epoch, file, `_ingested_at`, `_processed_at`; AWS CloudTrail keeps 90 days of management events at no charge; Unity Catalog lineage shows which tables feed each view.
  - International transfer: the Duel service lives in AWS US East; the lakehouse lives in Databricks Free Edition, in the region Databricks chose. The notice names Amazon Web Services, Databricks and the United States; whether more is needed is O26.
  - Free Edition's terms let Databricks use workspace data to improve its services, so only synthetic store data and anonymous, aggregate Duel and share records go there: no IPs, no names, no covers.
- **Ley 1266 de 2008** (financial habeas data, as amended by Ley 2157 de 2021): payment behavior (failed payments, fraud flags, chargebacks) is tagged `ley_1266`, with bounded, documented retention. Designed as if the law applied.
- **Consumer protection** (Ley 1480 de 2011, O12): the "7-day satisfaction guarantee" stays a simulator policy, not a legal claim, until Phase 2 reads the official text.
- **Experiment ethics:** D20.
- **Public site:** D22, D24 and D37, source badges on every number (D28) and a "Simulated" banner on the test store.
- **Duels and votes** (D34–D37, D56, D60):
  - Notice: the create page says what is stored (the two versions, genre, book language, the optional first page and link), where (AWS, United States), for how long (covers: 30 days after the end) and who can see it (people with the link).
  - Votes: anonymous. No cookies, no names. The one-vote marker is a hash made with a salt destroyed every day; the IP itself is never stored.
  - Uploaded images: the writer confirms they own the cover; a report button; your approval for the game (D36); deletion by the Duel's lifecycle, with its execution history as proof.
- **Shared results** (D24–D27): the share dialog is the notice; nothing is sent until the writer presses Share; the receipt code withdraws a result from the next nightly library; only summaries for groups of 5 or more are published. Processors named in the notice: Amazon Web Services and Databricks.
- **Licenses:** Upworthy data CC BY 4.0, with attribution; Gutenberg metadata CC0, no texts or covers; `library.json` published as CC BY 4.0 (D27).
- **Synthetic flag:** every envelope carries `producer.synthetic=true`. Duel votes are real and never carry it; Silver keeps the two apart, and the library takes only real results (D26).

3.19 Sizing and throughput

Experiment	Baseline	MDE	α / power	n per arm	Exposure probability	Users needed
EXP-CHK-001	70% paid within 30 min	+3.0 pp	0.05 two-sided / 0.80	3,554	0.25	28,430
EXP-PDP-002	4.0% purchase within 24 h	+0.6 pp (+15% rel.)	0.05 / 0.80	17,943	0.70	51,270 (binding)
EXP-BUN-003	COP 2,400 net revenue per exposed user (σ ≈ 9,600)	+15% (COP 360)	0.05 / 0.80	8,373 with CUPED ρ = 0.5 (10,159 at ρ = 0.3; 11,163 without)	0.60	27,910

- Formulas. Proportions: $n = [z_{0.975}\cdot\sqrt{(2\bar{p}\bar{q})} + z_{0.8}\cdot\sqrt{(p_1q_1+p_2q_2)}]^2 / \delta^2$. Means: $n = 2(z_{0.975}+z_{0.8})^2\sigma^2(1-\rho^2)/\delta^2$.
- N = 70,000: 51,266 × 1.35 for safety.
- Expected exposed per arm: CHK 8,750, PDP 24,500, BUN 21,000.
- Power at the expected n: primaries CHK 0.99 and PDP 0.91 at the MDE; guardrails CHK fraud 0.88, CHK chargebacks 0.98, PDP refunds ≈ 1.00, BUN any-purchase 0.98, BUN refunds ≈ 1.00.

Timeline (simulated, America/Bogota):

Window	Dates
Pre-period (CUPED)	06-22 → 07-05
Enrollment	07-06 → 08-02 (28 days)
Schema v2 switch	07-20
Cutoff: PDP	08-10 (24 h purchase + 7-day refund window)
Cutoff: BUN	08-17 (7-day revenue + 7-day refund maturity)
Cutoff: CHK (single final readout)	09-21 (chargebacks mature 45 days after the last order, + 5 days)
Live mode	From Phase 8: one simulated “today” per daily run

Duels. The target is 330 readers per Duel: 165 per version find a gap from 30% to 45% at α = 0.05 two-sided and 80% power (the prototype computes 326 and rounds up). A Duel’s covers take at most 600 KB; 3,000 Duels’ covers are about 1.8 GB, deleted 30 days after each end. A vote costs about 3 write units, so 20 WCU absorb about 6–7 votes a second, with DynamoDB’s burst capacity on top.

Volume: about 1.8 M events in about 0.42 M messages: 0.9 GB of raw JSON, 0.17 GB on the wire with zstd at 5×. The backfill through a CI Kafka takes about 5–8 minutes, like the local Docker run.

The runner’s budget: a standard runner has 4 CPUs, 16 GB of RAM and 14 GB of SSD. It only runs Kafka, the simulator, the bridge and small API calls now; the backfill’s 0.9 GB fits with room. The sync moves the day’s files (a few MB) from S3 out to Databricks, far inside the 100 GB of free data transfer out a month. The Databricks side of the run (pipeline update + notebooks) must fit the daily quota with room; Phase 3 measures it (O35). # 4. Bill of materials (Databricks and AWS allowances and prices checked 2026-10-19)

All AWS figures are US East (N. Virginia). “Monthly cost” is the steady state after the S1.1 upgrade, with the site live and the daily run on (section G).

Component	Free allowance / license	Our use	When a limit is hit	Official source	Monthly cost
Databricks Free Edition	$0, no payment method; serverless only; one 2X-Small SQL warehouse; ≤ 5 concurrent job tasks; one active pipeline per type; outbound internet to trusted domains only; no commercial use; a daily fair-use quota	The pipeline, the Job, notebooks, Unity Catalog, the volume, the warehouse for `site_export`	Compute stops for the rest of the day; nothing is billed	Databricks Free Edition limitations (updated 2026-09-29)	$0

AWS account, Free plan (until S1.1)	$100 credits at sign-up + up to $100 by activities; no charges while on the plan; only “select services”	Phases 0–1	The account closes after 6 months or when credits run out; data kept 90 days	Choosing a plan; Supported services for Sign up for AWS (new)	$0
AWS account, paid plan (from S1.1)	Remaining credits apply to future bills; monthly free allowances continue	Everything below	Normal pay-as-you-go above the allowances	Choosing a plan; Free Tier FAQs	$0
Amazon S3 storage	5 GB of S3 Standard storage credits with a CloudFront flat-rate Free plan, for any bucket in the account	< 1 GB landing + artifacts, ≤ 2 GB covers	Pay $ per GB-month above 5 GB; CI alert at 4 GB	CloudFront flat-rate plans	$0.00
Amazon S3 requests	None	15,000–60,000 PUT/LIST a month	n/a	S3 pricing (re-checked in the Pricing Calculator in Phase 0)	≈ $0.10–0.30, credits in year 1
Data transfer out to the internet	100 GB a month, all services	< 1 GB (the daily sync to Databricks, site traffic goes through CloudFront)	Billed per GB above	DynamoDB pricing page (general Free Tier note)	$0.00
AWS Lambda	1 M requests and 400,000 GB-seconds a month	Duel API, export, lifecycle steps, salt	$0.20 per M requests; $0.0000166667 per GB-s	Lambda pricing	$0.00
Lambda function URLs	No extra charge beyond Lambda	The Duel API’s door	n/a	Lambda docs	$0.00
Amazon DynamoDB	25 GB, 25 WCU, 25 RCU, 2.5 M stream reads; provisioned mode, Standard class	Table 20/20 + index 5/5, < 1 GB	Requests throttle (no bill)	DynamoDB provisioned pricing	$0.00
AWS Step Functions (Standard)	4,000 state transitions a month, for new and existing customers indefinitely	6 per Duel	$0.000025 per transition above	Step Functions pricing	$0.00
Amazon EventBridge Scheduler	14 M invocations a month	30 (daily salt)	$1.00 per M above	EventBridge pricing	$0.00
Amazon CloudFront, flat-rate Free plan	$0 a month, no overage charges: 1 M requests, 100 GB, 5 cache behaviors, WAF with 5 rules, 16 KB body inspection; up to 3 Free plans per account; not for Free plan accounts	Site, covers, `/v1/*`	Traffic far above the allowance for months may be served from fewer edges	CloudFront flat-rate plans; CloudFront pricing	$0.00
Amazon CloudWatch	5 GB of logs, 10 custom metrics, 10 alarms, 3 dashboards	Lambda logs (14 days), ≤ 10 alarms	n/a	CloudWatch pricing	$0.00
AWS Budgets	Budgets free; first two action-	2 budgets, 1 action	$0.10 a day per extra action budget	Budgets pricing	$0.00

	enabled budgets free				
Parameter Store (Systems Manager)	Standard parameters free	3 SecureStrings	n/a	Systems Manager pricing	$0.00
Amazon SNS	Free tier for e-mail notifications	A few alarm e-mails	n/a	SNS pricing	$0.00
AWS CDK, CloudFormation, IAM, CloudTrail (90-day event history)	No charge	All stacks	n/a	CDK and CloudFormation docs	$0.00
GitHub public repo + Actions	Free for public repos, standard runners	Daily (30–60 min including the wait for the Job), weekly, CI, deploys	Schedules disabled after 60 days without repo activity	GitHub plans, runners, terms	$0.00
Apache Kafka 4.2.2 image (KRaft)	Apache-2.0	Docker and the runner	Lower the heap	Kafka downloads	$0.00
Docker Engine (Ubuntu 26.04, WSL2)	Apache-2.0	Local Kafka, DynamoDB Local	Anonymous Hub pulls: 100 per 6 h per IP	Docker docs	$0.00
PySpark (local mode, unit tests only), Databricks CLI and SDK	Apache-2.0 / Databricks license	Tests, bundle deploys, Job trigger, Files API, Statement Execution API	n/a	PyPI; Databricks docs	$0.00
uv + CPython 3.12	Apache-2.0/MIT; PSF	Everything Python	n/a	github.com/astral-sh/uv	$0.00
Python libraries	confluent-kafka, Faker (es_CO), numpy, scipy, pandas, pyarrow, statsmodels, pytest, boto3, databricks-sdk, jsonschema, PyYAML, ruff: OSI licenses		n/a	PyPI	$0.00
Frontend	React, Vite, TypeScript, ECharts, i18next, Vitest, Testing Library (MIT / Apache-2.0)	The site	n/a	npm registry	$0.00
Fonts: Instrument Sans, EB Garamond, Geist Mono	SIL OFL 1.1, served from our origin		n/a	the font projects	$0.00
Upworthy Research Archive	CC BY 4.0	14.3 MB CSV	n/a	archive; OSF	$0.00
Project Gutenberg metadata	CC0 per record	2,000 titles	Use the offline catalogs	offline catalogs	$0.00
Lab only: AWS Glue 6.0 jobs	No free allowance: $0.44 per DPU-hour list (6.0 is 30% less; Flex $0.29 list)	A few sessions, ≤ $5 of credits in total (D61)	`lab` stack destroyed after each session	Glue pricing; What's New 2026-08-21	$0 out of pocket
Total					**$0 in year 1; ≈ $0.10–0.30 after**

Official sources, checked 2026-10-19:

- Databricks Free Edition: `docs.databricks.com/aws/en/getting-started/free-edition-limitations` (updated 2026-09-29); Files API `docs.databricks.com/api/workspace/files`; workload identity federation for GitHub Actions `docs.databricks.com/aws/en/dev-tools/auth/provider-github`; Jobs API `runs/export`; Lakeflow Declarative Pipelines, Auto Loader and AUTO CDC docs.
- AWS Free plan: `docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html` and `aws.amazon.com/free/free-tier-faqs/`.
- Services on the Free plan: `docs.aws.amazon.com/accounts/latest/reference/supported-services-sign-up-new.html` (written for the new sign-up experience, which is being released to a limited number of customers; P1 checks your account).
- CloudFront: `docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/flat-rate-pricing-plan.html`, `aws.amazon.com/cloudfront/pricing/`, and, for Lambda function URL origins, `docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-lambda.html`.
- `aws.amazon.com/lambda/pricing/`, `aws.amazon.com/dynamodb/pricing/provisioned/`, `docs.aws.amazon.com/amazondynamodb/latest/developerguide/howitworks-ttl.html`, `aws.amazon.com/step-functions/pricing/`, `aws.amazon.com/eventbridge/pricing/`, `aws.amazon.com/glue/pricing/`, `aws.amazon.com/athena/pricing/`, `aws.amazon.com/cloudwatch/pricing/`, `aws.amazon.com/aws-cost-management/aws-budgets/pricing/`, `aws.amazon.com/api-gateway/pricing/`, `aws.amazon.com/msk/pricing/`.
- Glue 6.0: `aws.amazon.com/about-aws/whats-new/2026/08/aws-glue-6-0-price-reduction-iceberg-v3/`, `docs.aws.amazon.com/glue/latest/dg/migrating-version-60.html`, `docs.aws.amazon.com/glue/latest/dg/monitor-spark-ui.html`.

4.1 Rejected options

Option	Facts (checked 2026-10-19)	What we lose without it
AWS Glue jobs in the daily path	No free allowance; $0.44 per DPU-hour list; ≈ $6.60 a month for a daily run.	Managed Spark on AWS every day. Free Edition does the job at $0; Glue stays a Lab (D61).
Amazon Athena + Glue Data Catalog	Athena $5 per TB scanned; the tables live in Unity Catalog, not on S3.	SQL over the tables from AWS. The Free Edition SQL warehouse does it.
Spark in GitHub Actions on S3 (v4.0's first draft)	$0, but hand-written MERGE dedup and SCD2, IAM-only governance, and Delta's single-writer rule on S3.	Nothing: Databricks gives pipelines, AUTO CDC, expectations and Unity Catalog at $0 (Q23).
Amazon API Gateway (HTTP API)	1 M free calls a month for 12 months, then $1.00 per million.	Request validation and usage plans. A Lambda function URL behind CloudFront does our job; API Gateway is the fallback (D49).
DynamoDB on-demand	The free allowance covers provisioned capacity only.	Automatic scaling for viral spikes; we throttle instead (R57).
AWS Lake Formation	Free, but it governs Athena, Glue, EMR and Redshift, and our tables live in Databricks.	Nothing: Unity Catalog grants and masks (D45).
AWS Glue Data Quality	Runs inside Glue jobs, which cost money.	Nothing: pipeline expectations (D44).
Amazon MSK	Paid-plan only on the Free plan. MSK Serverless: $0.75 per cluster-hour (≈ $548 a month).	A managed always-on Kafka.
Kafka on ECS Fargate per run	About $0.03 a day; no free allowance. You chose GitHub Actions.	Kafka inside AWS. ADR-002 keeps it as the first upgrade if GitHub's terms ever get in the way.
Amazon Kinesis, Firehose, EMR, Redshift	Paid-plan only on the Free plan, or billed per hour.	Nothing we need.
Apache Iceberg	AWS's first-class table format (Athena can write it); free to use.	Athena writes. You chose Delta (D39), native on Databricks.
AWS Secrets Manager	$0.40 per secret a month.	Automatic rotation; our salt rotates by its own Lambda.
A NAT gateway / VPC	About $0.045 an hour (≈ $33 a month) plus data.	Private networking; nothing we run needs it.
Databricks on AWS (paid)	Compute billed by Databricks per DBU plus AWS compute; a 14-day trial.	Reading S3 directly (external locations), the Spark UI, no daily quota. The first upgrade if the project goes commercial.

Cloudflare Workers + D1, GitHub Pages, Aiven	$0 forever, but you chose AWS for the product (Q17–Q21).	Not even the cents of S3 requests.
Power BI Pro / PPU	$14 / $24 per user per month to share	Not needed (React).

5. Phases

Effort: 230 h (192–268), about 12–14 weeks at 20 h a week. v3.0 was 230 h too: the hybrid adds AWS IaC, IAM, the Duel lifecycle and the S3-to-volume sync, and removes the local cluster, Aiven and the plain Structured Streaming track; Databricks keeps doing the heavy lakehouse work (AUTO CDC, expectations, Unity Catalog). Each phase ends with its acceptance checks. A STOP means I wait for your OK.

Lifesaver first (L8). The Duel, Real or luck? and the home page go live in Phase 1, right after the feasibility spike, on AWS. Real votes then collect in DynamoDB while the pipeline is built.

Phase	What	Hours	Weeks (20 h/week)
0	Feasibility spike on both platforms: AWS account, $0 guardrails, CDK bootstrap, OIDC; Free Edition catalog, volume, secret scope, bundle, CI identity; probes P1–P8, P10–P12 (S0.1, S0.2)	12–16	W1
1	Lifesaver launch: site on S3 + CloudFront, the Duel service (Lambda function URLs, DynamoDB, covers, the Duel lifecycle), the Duel, Real or luck?, home, the new look; upgrade to paid, P9, first public deploy (S1.1)	40–56	W2–W4
2	Simulator, contracts and the real catalog	13–17	W4–W5
3	Consumer groups, the S3 landing, the sync, and Bronze with Auto Loader, with the file sources and the DynamoDB export	10–14	W5–W6
4	Silver: expectations, watermark dedup, AUTO CDC SCD2, Unity Catalog masks and grants, Duel votes	14–19	W6–W7
5	Gold factory: 20 materialized views	9–12	W8
6	Analysis, validation, real-data lessons, the library; notebooks as Job tasks	18–25	W8–W9
7	More tools and pages: the game, the Ad Doctor, the library from Duels, the planner, Real lessons, the Test store, For nerds, Methods	50–70	W10–W12
8	The daily and weekly workflows, the `ebook_daily` Job, the backfill workflow, alarms (S8.1)	16–22	W13
9	Lab on credits: the Glue Spark UI lab; Delta maintenance and rollback demos	4–6	W14
10	Docs, report and interview prep (§3.0 against real numbers)	6–11	W14
	Total	**192–268**	

The weeks assume the slower end. At the faster end, everything fits in about 10 weeks.

Phase 0: Feasibility spike (12–16 h)

1. **Repo check:** the public repo `Pipe199x/ebook-ab-platform` and its Pages landing page already exist. Add `.gitignore`, MIT `LICENSE` and `tools/make_plan_md.py` if missing.
   - **STOP S0.1:** your OK before (a) changing the repo, (b) turning on secret scanning + push protection, (c) fixing the landing page's "Ask" placeholders (your text, or remove the section). The landing page stays on GitHub Pages until S1.1, then points to the CloudFront site.
2. **Local tools (no sudo):** uv → Python 3.12 → `uv sync`; AWS CLI v2 (user install); Databricks CLI; Node LTS; `npm i -D aws-cdk`; a JDK 17 for local PySpark tests.
3. **Docker Engine:** you run the apt-repo lines (I paste them) + usermod. Used for local Kafka and DynamoDB Local only.
4. **STOP S0.2, the AWS sign-up and the Databricks login.** You:
   - sign up at `portal.aws.amazon.com` and choose the **Free plan** note whether a card is asked (O27);
   - turn on MFA for the root user;
   - create `andres-admin` (IAM user with MFA, AdministratorAccess) and an access key for the CLI on your laptop only (`aws configure --profile ebook`), rotated every 90 days; IAM Identity Center, the better option, isn't on the Free plan, so we revisit it after S1.1;
   - keep region `us-east-1` (D38);
   - log in to your Databricks Free Edition workspace, run `databricks auth login` on your laptop (OAuth, no token on disk for long), and note the workspace URL and region.
5. **Guardrails before anything else (D53):** the two AWS budgets and the $5 action, an SNS topic with your e-mail, the free-allowance allow-list check in CI.
6. **CDK bootstrap** (`cdk bootstrap aws://<account>/us-east-1`) and the `foundation` and `ci` stacks: the lake bucket and lifecycle rules, the IAM roles, the GitHub OIDC provider and its three roles (D48).
7. **The bundle skeleton** (`databricks.yml`): catalog `ebook_ab_dev`, schemas, the volume `ops.lakehouse_files`, the secret scope `ebook` with the pepper, a hello-world pipeline and Job; the CI identity per D48 (O34).

8. **Probes P1–P8 and P10–P12** (§3.4), each a script in `ebook_ab.tools.probes` or a GitHub workflow, writing evidence to `docs/evidence/phase0/`. P9 waits for S1.1.
9. **Real-data probe:** Upworthy exploratory CSV (expect 14,260,949 bytes, 22,666 rows, 4,873 test IDs); Gutenberg offline RDF catalog (parse 10 records).
10. **Write ADR-002, ADR-005, ADR-006 and ADR-010**, and record the spike's S3 request count, AWS cost and Databricks run durations (O35, O31).

Acceptance:

- the AWS account is on the Free plan, MFA on, both budgets and the action in place;
- P1–P8 and P10–P12 pass, or their fallbacks are recorded;
- a GitHub workflow assumes `gha-pipeline`, lands files in S3, syncs them into the volume, triggers a Job by API, and is refused outside its prefixes;
- the allow-list check fails a test branch that adds an `AWS::Glue::Job` to a production stack;
- real-data counts match the sources;
- Cost Explorer shows $0.00 out of pocket.

Verification

```
aws sts get-caller-identity --profile ebook
aws budgets describe-budgets --account-id <id> --profile ebook | jq '.Budgets | length'      # 2
databricks bundle validate -t dev && databricks bundle deploy -t dev
gh workflow run probes.yml && gh run watch                                                   # P3, P4, P8: PASS
uv run python -m ebook_ab.tools.probes local --profile ebook                                 # P1, P2, P5-P7, P10-P12
uv run python -m ebook_ab.realdata.check                                                     # upworthy rows=22666
tests=4873
```

Phase 1: Lifesaver launch (40–56 h)

The writer-facing core goes live first (L8), so real votes collect while the pipeline is built.

- **The site base** (`web/`, §3.13): Vite + React + TypeScript, Spanish and English, the new look (L11), the warmer voice (L9), light and dark themes, the source badges (D28), and the frozen data snapshot.
- **The pages:** home with three doors (the Ad Doctor door says "Coming soon" until Phase 7); the Duel (create, share, vote, progress, reveal, share image, library check box); Real or luck?; a first Methods page.
- **The Duel service** (`edge/`, §3.13): CDK `edge` stack: the `duel-api` and `duel-admin` function URLs (IAM auth), the DynamoDB table and export index in provisioned mode, TTL, the covers bucket with presigned POST and its 45-day backstop, the Duel lifecycle state machine (D60), the salt schedule (D56); the routes to create, serve, vote, see progress, see the result, opt into the library and report. The browser computes `x-amz-content-sha256` for every POST. The game routes come in Phase 7.
- **The `site` stack:** the site bucket, CloudFront with origin access control for S3 and for the function URL, behaviors for `/v1/*` and `/covers/*`, the WAF rate rule.
- **Stats in TypeScript, first part:** the z-test and its range, SRM χ^2, the "Too close" rule and the exact binomial test, with shared vectors from scipy.
- **Checks:** the headless privacy test, keyboard and contrast checks, the < 5 MB budget, the demo's click-through test pointed at the real pages.
- **Deploy:** `site.yml` and `deploy.yml` (AWS stacks only for now). ADR-007 (static site + daily export) and ADR-009 (Duels on Lambda + DynamoDB + Step Functions; privacy; why votes reach the lakehouse as files).
- **The "Founding writers" launch note**, `docs/launch/founding-writers.md`, in Spanish and English. Posting it is your call; I don't contact anyone.
- **STOP S1.1, before the first public deploy:** you upgrade the account to the paid plan (Billing → Upgrade plan; D54) and confirm the budgets still e-mail you. Then I switch the distribution to the flat-rate Free plan, run P9, and deploy `prod`.

Acceptance:

- Vitest green; the TypeScript results agree with scipy within 1e-9 on the shared vectors;
- edge tests (mocked SDK and DynamoDB Local): bad payloads refused, a second vote from the same visitor refused, the result route refuses before the end, the 6th Duel in a day refused, a POST without the right SHA-256 refused;
- in dev, a Duel's lifecycle with 5-minute waits closes it and deletes its covers, visible in the execution history;
- each of the four endings appears on its fixture Duel, in Spanish and English;

- a test Duel on the live site gets its two links, takes votes from two phones and shows progress without the winner; then it is removed;
- calling the function URL directly, without CloudFront, is refused (403);
- 0 requests to any origin but the site's own; the build is < 5 MB; the account shows "Paid plan" and the distribution "Free (flat-rate)".

Verification

```
npm --prefix web ci && npm --prefix web test && npm --prefix web run build
npm --prefix edge ci && npm --prefix edge test
uv run cdk deploy -c stage=prod ebook-edge-prod ebook-site-prod --require-approval broadening
curl -s -o /dev/null -w "%{http_code}" -X POST <function-url>/v1/duels      # 403
node tools/check_demo.cjs --url https://<distribution>.cloudfront.net/
```

Phase 2: Simulator + contracts + real catalog (13–17 h)

- **Config:** `topics.yml`; `catalog.yml` (title selection and price model); `simulation.yml` (70k shoppers; weekly seasonality and paydays on the 15th/30th; a payment-method mix and success rates, invented and documented; risk engine, 7-day guarantee, chargeback lag of 10–60 days; faults; the truth block); `experiments.yml`.
- **Freeze the pre-registrations:** `docs/prereg/EXP-*.md` + tag `prereg-v1`, before the first full backfill.
- **Catalog builder:** Gutenberg RDF → about 2,000 es/en titles, about 150 fictional sellers; prices COP 9,900–39,900; demand ∝ downloads^α; JSONL for `landing_real/catalog/`.
- **Contracts:** envelope v1/v2, every event type per version, ops records, `contracts/community/result.schema.json`.
- **Simulator** (`src/ebook_ab/simulator`): backfill and live modes, run manifests. No throttle (Aiven is gone, D9).
- **Faults:** duplicates 1% (0–120 s), late 2% (1–30 min), malformed 0.5%, the v2 switch on 07-20, the SRM toggle (EXP-SRM-900 only), one malformed burst.
- **Infra and examples:** `docker/compose.yml` (Kafka 4.2.2, KRaft, 72 h; DynamoDB Local), `tools/topics.py`, `docs/examples/<topic>.json`.
- **Tests:** assignment parity + uniformity χ^2; fault-rate tolerances; schema validity; catalog parsing; payday seasonality sanity; a local Kafka round trip.

Acceptance: the 5 × 2 layout locally and in CI; fault rates within ±20%; treatment share 0.5 ± 0.01; idempotence + `acks=all` in the logs; a local 70k backfill finishes and prints the compression ratio.

Verification

```
docker compose -f docker/compose.yml up -d && uv run python -m ebook_ab.tools.topics ensure --target local
uv run pytest tests/unit tests/contract -q && uv run pytest tests/integration -m kafka -q
uv run python -m ebook_ab.simulator backfill --target local --users 70000 --seed 20260706
```

Phase 3: Consumer groups + S3 landing + sync + Bronze (10–14 h)

- **Code:** `src/ebook_ab/bridge` (§3.3, with `--crash-after-upload N` and `--epoch`, S3 writer and local writer); `src/ebook_ab/monitor` (§3.7); the `export_edge` Lambda (a sequence watermark per entity); `ebook_ab.tools.sync_volume` (S3 → volume, only new names, counts checked, D62).
- **`pipelines/ebook/bronze.py`:** one Auto Loader streaming table per source (5 Kafka-file sources, 5 real-data sources), explicit schemas, `allowOverwrites=false`, rescued data on (D41). ADR-001.
- **Runs and demos:** the 92-day backfill through `backfill.yml` (CI Kafka → bridge → S3 → sync → pipeline in dev); the kill test; the double sync test; the monitor demo (3 members → one idle → kill one → rebalance → alerts and DLQ), recorded locally.

Acceptance:

- the 5 Kafka Bronze tables share one schema;
- per (topic, epoch, partition): count = last − first + 1, with 0 duplicates and 0 gaps, including after the kill;
- a second sync and pipeline update with no new files upload 0 files and add 0 rows;
- file Bronze counts match the sources; the Duel votes in Bronze equal the rows in the export files;
- the backfill fits inside one day's Free Edition quota (or is split across two days, recorded, O35); its S3 request count is recorded (O31).

Verification (SQL in the Free Edition SQL editor, dev catalog)

```
SELECT topic, source_epoch, `partition`, `offset`, count(*) c FROM ebook_ab_dev.bronze.orders_raw GROUP BY ALL HAVING c
> 1;  -- 0 rows
```

```
SELECT topic, source_epoch, `partition`, max(`offset`) - min(`offset`) + 1 - count(DISTINCT `offset`) AS gaps
  FROM ebook_ab_dev.bronze.orders_raw GROUP BY ALL;
-- all 0
SELECT count(*) FROM ebook_ab_dev.bronze.upworthy_raw;
-- 22666
```

Phase 4: Silver, with Duel votes (14–19 h)

- `pipelines/ebook/silver.py`: parse/route/explode; watermark dedup (D10); `order_items`, `event_experiments`.
- `duels` (AUTO CDC, type 1) and `duel_votes` from the export files: the first vote per Duel, visitor hash and day; the bot filter; the hash dropped after use.
- `books`, `users_scd2` (AUTO CDC, type 2, D43), `real_headline_packages`, `community_results` with withdrawals as AUTO CDC deletes.
- `ebook_ab.quality` turns `config/quality/*.yml` into expectations with the three severities and the quarantine flow (D44).
- Masked display columns; the pepper from the secret scope; Unity Catalog column masks, tags and grants (D45).
- Pure transforms in `src/ebook_ab/transforms`, tested locally with `assertDataFrameEqual`. ADR-004.

Acceptance: `ops.dq_reconciliation` all PASS: exact counts; duplicates removed = injected; late events kept; quarantine counts by rule; Duel votes match the export files; v2 columns present; SCD2 has 1 current row per user and contiguous intervals; 0 clear PII in Silver; a `bi_readers` member is refused on Silver and sees Bronze's clear columns masked (P7 repeated on real tables).

Verification

```
SELECT * FROM ebook_ab_dev.ops.dq_reconciliation WHERE status <> 'PASS';                              -- 0 rows
SELECT user_id FROM ebook_ab_dev.silver.users_scd2 GROUP BY user_id HAVING count_if(__END_AT IS NULL) <> 1; -- 0 rows
SHOW GRANTS ON SCHEMA ebook_ab_dev.silver;                                                           -- no
bi_readers
```

Phase 5: Gold factory, 20 materialized views (9–12 h)

- `config/gold_views.yml` (20 entries with consumers), `pipelines/ebook/gold_factory.py`, the private registry and totals check (D57).
- `docs/data_dictionary.md` from the YAML and the tags.
- Tests: the YAML model, every view's SQL on a small fixture with local PySpark, staleness, a required consumer.

Acceptance: 20 views (exp 8, mkt 3, pay 2, cat 1, real 2, ops 2, duel 2); every view has comments and `cluster_by`; Gold totals = Silver totals; SRM flags EXP-SRM-900 only; `bi_readers` can read every Gold view.

Verification

```
SHOW TABLES IN ebook_ab_dev.gold;                                    -- 20 views (+ registry, results, prior,
summary)
SELECT * FROM ebook_ab_dev.gold.check_totals WHERE NOT ok;           -- 0 rows
```

Phase 6: Analysis, validation, real-data lessons, the library (18–25 h)

- **Code:** `ebook_ab.stats`, including `bayes` (normal–normal) and `meta` (random effects), with oracle tests, the coverage simulation and `tests/vectors/*.json`.
- **Notebooks** (`.ipynb`, outputs stripped in Git), run as tasks of `ebook_daily` with Job parameters (catalog, run_id, cutoff): `10_ab_analysis`, `11_validation`, `12_real_headlines`, `13_community_summary` (ideas need 5 tests, shown as W of N), `20_dq_report` (the gate) (D51).
- **Community and Duels in the library** (L6): fixtures in the dev catalog, flagged synthetic, never in prod (D26). The endings in `gold.duel_results` must match the API's reveal on the shared vectors (D34).
- **Outputs:** `experiment_results`, `real_prior`, `community_summary`, 3 memos, `docs/reports/headline-tests.md`, executed notebooks exported to `artifacts/` (D51).

Acceptance: decisions follow the pre-registered rules mechanically; proofs (a), (b) and (c) pass; results reproducible from a seed + code version + snapshot ID; the real-data numbers reproduce from Gold with the sensitivity table; on the fixture, `community_summary` matches hand-computed counts and medians, cells under 5 are marked, idea tags under 5 tests hidden, bad rows in quarantine; a forced quality failure fails the Job and stops the workflow before `site_export`.

Verification

```
uv run pytest tests/stats -q
databricks bundle run ebook_daily -t dev
SELECT experiment_id, metric, metric_role, variant, n, estimate, ci_low, ci_high, p_value, decision
  FROM ebook_ab_dev.gold.experiment_results ORDER BY 1,3,2,4;
SELECT check_name, passed FROM ebook_ab_dev.ops.validation_results;   -- all passed
```

Phase 7: More tools and pages (50–70 h)

- **Which would you read?** (L2, D36): the game route, vote credits on the Duel's private link, the report button, and `ebook_ab.tools.moderate` (list, approve, remove) through the IAM-authorized admin function (D59).
- **Ad Doctor** (L3): the CSV reader (English or Spanish columns, commas or semicolons), beta-binomial verdicts, money to stop spending, cost per click for Meta Ads; tested on made-up sample reports in `tests/fixtures/ads/`. The file never leaves the browser.
- **The library from Duels** (L6, §3.12) with the v2.3 rules (C1–C12).
- **Inside pages:** the ad-test planner (never fewer than 7 days), *Real lessons*, the Test store with links to executed notebooks, For nerds with the hybrid map and each box's free allowance or free plan, the full Methods page.
- **Data for the site:** `ebook_ab.site_export` (Statement Execution API) + `contracts/site/*.schema.json`; the Share for ad tests and its receipt code.

Acceptance: the game serves only approved Duels, and a vote there counts in that Duel and adds one credit; the Ad Doctor gives the expected verdict on every sample report and sends nothing; the library shows "0 of 5" until real results arrive; every number block carries a badge; the planner never plans fewer than 7 days; For nerds passes the demo's checks on the live page; a test share lands in DynamoDB and its receipt code withdraws it; the 6th share in a day is refused; Vitest green; 0 cross-origin requests; build < 5 MB.

Verification

```
npm --prefix web ci && npm --prefix web test && npm --prefix web run build
curl -s https://<distribution>.cloudfront.net/data/library.json | jq '[.cells[] | select(.enough)] | length'   # 0
until real results
```

Phase 8: The daily run, maintenance and alarms (16–22 h)

- `daily-e2e.yml`, `weekly-maintenance.yml`, `backfill.yml` and the finished `ci.yml` and `deploy.yml` (§3.14, §3.17).
- The `ebook_daily` Job in the bundle (pipeline task, notebook tasks, the gate, the paused fallback schedule, e-mail on failure, D15).
- The CloudWatch alarms for the edge, `ops.run_log`, `ops.cost_log`.
- The prod targets: catalog `ebook_ab`, its volume and pipeline; the prod prefixes in S3.
- **STOP S8.1** before enabling the daily schedule and the daily public refresh.

Acceptance: the daily workflow is green end to end; a forced failure in Silver fails the pipeline update and the Job, stops the workflow before `site_export`, and both GitHub and Databricks e-mail you; the daily Databricks work stays under the quota with room (O35); CI green on a PR; a push to `main` deploys dev; the site shows "data as of" today; a test share reaches `silver.community_results` in the next run and its withdrawal removes it the run after; a test Duel's votes reach `silver.duel_votes`; the weekly maintenance runs; the month's out-of-pocket cost is $0.00 and the gross cost under $1.

Verification

```
gh workflow run daily-e2e.yml && gh run watch
curl -s https://<distribution>.cloudfront.net/data/freshness.json | jq .data_as_of                          # today
aws ce get-cost-and-usage --time-period Start=<month-start>,End=<today> --metrics UnblendedCost \
  --granularity MONTHLY --profile ebook                                                                      # gross <
1.00
```

Phase 9: Lab on credits: Glue Spark UI lab, Delta demos (4–6 h)

Everything in this phase is Lab (D31, D55, D61): portfolio practice that the product doesn't need (§3.0). The Glue part is paid by the sign-up credits; the Delta demos run on Free Edition.

- Deploy the `lab` stack by hand: a Glue 6.0 job pointing at `jobs/perf_lab.py`.

- `perf/spark_ui_lab` on Flex workers: generate 20 M product views with a 30% bestseller, then before/after runs for a broadcast join, AQE skew join or salting, and file sizing; screenshots from Glue's serverless Spark UI.
- `notebooks/40_delta_ops.ipynb` on Free Edition: `DESCRIBE HISTORY`, time travel, the Change Data Feed, `OPTIMIZE`, `VACUUM`, `RESTORE` on `experiment_results`.
- `cdk destroy` the `lab` stack.

Acceptance: every optimization improves a metric (wall time, shuffle bytes, or max/median task ratio); each Delta feature has one demo with output; the Lab's total is under $5 of credits and $0 out of pocket; the `lab` stack no longer exists.

Phase 10: Docs, report and interview prep (6–11 h)

- The README: problem, impact, the hybrid architecture, the free-allowance table with the real Cost Explorer numbers and the Databricks run durations, how to run, results, screenshots (site, the daily workflow, the pipeline graph, Unity Catalog lineage, a Duel's lifecycle, Glue's Spark UI from the Lab).
- The runbook, the final ADR-001…011, the data dictionary, the memos, the headline-tests report.
- §3.0 updated with the measured numbers (`docs/interview.md`), and practised aloud.
- The README marks the Lab parts (D31) and links the For nerds page. An optional short write-up for LinkedIn or a blog.

Acceptance: a clean-clone dry run of the runbook, into a fresh `dev` stage and catalog, reaches a readout and a site build; every link resolves (checked in CI). # 6. Risk register

L = likelihood, I = impact (H/M/L). Retired in v4.0, because their cause left the design: R1 (Free Edition blocks Aiven), R3–R5 (Aiven power-off, throttling, topic quota), R23 (continuous demo burns quota), R24–R25 (open-source pipelines and the local cluster's RAM). Back from v3.0 with the hybrid: R2, R6–R9, R20.

ID	Risk	L	I	Mitigation	Phase
R2	Free Edition's daily quota stops compute mid-run	M	M	Triggered runs, small data; heavy proofs weekly; the backfill split across days if needed; Phase 3 measures (O35); a failed run leaves yesterday's site, and the next run catches up from S3	All
R6	One active pipeline per type blocks dev + prod	M	M	P5 in Phase 0 (O3); run dev and prod at different hours, or dev on a fixture	0, 8
R7	Can't create a catalog, groups or a secret scope on Free Edition	M	M	P2 in Phase 0 (O2); fallback: the workspace catalog with schema prefixes, user grants	0
R8	No service principal or federation for CI on Free Edition	M	L	D48's chain: federation (O34) → SP OAuth secret → a PAT of at most 90 days in GitHub secrets, with a rotation note	0, 8
R9	PyPI blocked on serverless	M	L	P6 in Phase 0 (O6); numpy + scipy stats; statsmodels in CI only; or a wheel in the volume	6
R20	Free Edition deletes an inactive account or the workspace is lost	L	M	The daily run keeps it active; S3 keeps every raw file 180 days (D62); bundles and CDK rebuild everything; the backfill workflow refills the tables	All
R64	The Files API or the sync fails, or Free Edition's outbound rules change	M	M	P4 in Phase 0; fallback `databricks fs cp`; the sync checks counts and fails loudly; S3 keeps the files, so the next run catches up	0, 3
R65	GitHub OIDC trust to AWS misconfigured (too wide or broken)	M	H	Trust limited to `repo:Pipe199x/ebook-ab-platform` and per-role branch conditions; P8 proves a refusal	0, 8
R10	Backfill ordering breaks dedup or SCD2	M	H	The 1 h watermark is wide for minute-level re-sends (D10); AUTO CDC sequences by `(observed_at, topic, offset)`; exact reconciliation	4

R11	A full refresh loses Bronze	M	H	Bronze is append-only; full refresh only in dev; S3 landing files kept 180 days are its backup	3
R12	Clear PII in Bronze	H (by design)	M	Unity Catalog column masks on Bronze; hash and mask at Silver; the `no_clear_pii` expectation fails the update	4
R13	Secrets leak into the public repo	L	H	No AWS keys (OIDC); the Databricks credential only in GitHub secrets (D48); `.env` ignored; detect-secrets; push protection	All
R14	Statistical malpractice	M	H	Prereg tag before data, fixed horizon, SRM gate, Holm/BH, truth loaded last; the Duel's rules fixed in code and the server opens the result only at the end (D34)	1, 2, 6
R15	Seed luck (truth outside one CI)	M	L	Pre-registered seed + coverage simulation	6
R16	Lagging chargebacks bias the CHK guardrail	M	H	Matured orders only; single final readout on 09-21	6
R17	Kafka timestamps in simulated time → instant deletion	M	H	Send time always; unit test	2
R18	AWS free allowances, Free plan rules, Free Edition limits or prices change	M	M	Re-verify at each phase; the BOM cites sources with dates; the budgets catch any new charge within a day	All
R19	WSL 9P slowness	M	L	Heavy data on ext4	2
R21	Synthetic IDs collide with real people	M	M	Reserved-looking ranges, example.com, hashing (O9)	2
R22	Chrome headless issues in WSL	L	L	`--no-sandbox`; Windows fallback	docs
R26	Headline-click effects ≠ ebook purchase effects	H	M	Prior scoped to title/cover/copy tests; neutral-prior switch; sensitivity table	6, 7
R27	Site mistaken for a real company or real sales	L	M	No brand name; banner + footer; synthetic flag	1, 7
R28	Writers over-trust the tool	M	M	Verdict-first wording, "not sure yet" with the days still needed, source badges, "worked" = clicks or readers, not sales	1, 7
R29	Scheduled workflows auto-disable after 60 days of inactivity	H (after the project)	L	Documented; one-click re-enable; the site keeps its last data	8
R30	Copyright on texts or covers	M	M	CC0 metadata; generated SVG covers in the store; writers confirm a Duel's covers are theirs; report and removal; deletion by the lifecycle	1, 2, 7
R31	GitHub Actions usage terms or limits get in the way of the daily run	L	M	A short daily run that publishes the site; nothing 24/7. Exit plan in ADR-002: Kafka on Fargate (costs money, so only if forced)	8
R32	Frontend tool churn	M	L	Pin versions	1, 7
R33	Ad tests mislead	M	M	Platform A/B features with equal budgets; "Not fair yet" on uneven views; at least 7 days (D29)	7
R34	Spam or poisoned shares skew the library	M	H	D25 rules; WAF rate rule; quarantine counts on the data-health badge; a Duel joins the library only after a fair end	6, 7
R35	Cold start: the library stays empty	H	M	Honest empty state with "x of 5"; real headline lessons meanwhile; Founding writers; Duels fill the library without past tests (L6)	1, 7
R37	A writer in a small group is recognized	L	M	No personal fields; summaries only for groups of 5+; rounded lifts; raw records never published (D27)	6, 7

R38	Writers share mostly wins	M	M	Share offered for every fair verdict; share of wins watched; every Duel ending can join the library	1, 7
R39	Abusive or illegal images uploaded as covers	M	H	Unlisted links; you approve the game; a report button; `moderate` removes a Duel and its covers at once; no public gallery	1, 7
R40	Votes are not sales	H	M	One honest question; “interest, not sales” everywhere; Real or luck? for sales	1, 7
R41	Vote stuffing	M	M	Random version per reader; one vote per visitor per Duel (conditional write); rate limits and WAF; the Silver bot filter; the split check	1, 4
R42	A Duel never reaches 330 readers in 14 days	H	M	The game; “about N more readers”; sharing tips	1, 7
R43	Ad report formats differ or change	M	M	Known column maps in both languages; says which columns it couldn’t find; sample reports	7
R45	The Free plan closes the account (credits gone or 6 months)	M	H	Upgrade at S1.1, before anything is public (D54); credit alerts	0, 1
R46	A paid service or a runaway loop creates a bill after the upgrade	M	H	The CI allow-list blocks paid resource types in production; budgets at $0.01 out of pocket and $1 gross; the $5 action denies the pipeline and edge roles; Lambda concurrency caps; DynamoDB provisioned (throttles, never bills more); CloudFront flat-rate has no overage (D53)	0, 8
R47	The Free plan blocks a service we need	L	M	P1 checks the console in Phase 0; fallback: upgrade early	0
R50	Small files pile up in the volume and the landing prefixes	H	L	Few, large files (`events[]` batching); the weekly job deletes volume files older than 30 days; S3 lifecycle at 180 days	8
R51	A bucket or table becomes public by mistake	L	H	Account-wide Block Public Access; origin access control; cdk-nag; IAM Access Analyzer	1, 8
R53	Expired vote markers linger days in DynamoDB	H	L	The salt that made them is destroyed daily, so they can’t be linked to an IP (D56); filters on read	1
R55	International transfer of Duel data to the US under Ley 1581	M	M	The notice names AWS, Databricks and the US; O26 checks what more the law needs before S1.1	1
R66	Free Edition’s terms allow Databricks to use workspace data	H (by terms)	L	Only synthetic store data and anonymous aggregate Duel and share records go there; no IPs, names or covers (§3.18)	1, 3
R56	Lock-in to Databricks or AWS	L	L	Pure PySpark transforms and portable Gold SQL; raw files in S3; contracts in Git; ADR-003 lists what would move	10
R57	A viral Duel exceeds DynamoDB’s free 25 writes a second	L	M	Burst capacity; the site retries with “Lots of readers right now”; votes are never lost silently (the browser keeps and resends); switch to on-demand only if it happens often, accepting a small bill	1, 7
R59	S3 requests creep up (small files, chatty listings) and the “cents” grow	M	L	Few, large landing files (`events[]` batching); the sync lists only today’s and yesterday’s prefixes; `ops.cost_log` watched; the $1 gross budget	3, 8
R60	More than 650 new Duels a month pass Step Functions’ 4,000 free transitions	L	L	A happy problem; each extra 1,000 transitions costs $0.025; switch to EventBridge Scheduler one-time schedules (14 M free) if it becomes regular	7

R61	CloudFront's flat-rate Free plan refuses a Lambda function URL origin	M	M	P9 at S1.1; fallback API Gateway HTTP API ($1 per million requests after its first year: cents at our traffic)	1
R62	The S3 storage credit stops covering the landing files and covers (over 5 GB)	L	L	CI alert at 4 GB; covers deleted 30 days after each Duel; old landing files expire at 180 days	7, 8

7. Gold catalog: draft of the 20 views

All views live in `<catalog>.gold` (`ebook_ab` in prod, `ebook_ab_dev` in dev), are materialized views in the pipeline, are readable by the `bi_readers` group through Unity Catalog grants (D45), carry no PII, and use Bogotá dates. Every view names its consumer. They are refreshed each run by the Gold factory (D57); "Cluster by" is the `cluster_by` column list in the spec.

#	View	Grain	Sources	Purpose	Consumer	Cluster by
1	exp_exposures	experiment × user	silver.experiment_events	First assignment and exposure, variant, bucket, timestamps	Analysis	experiment_id, first_exposure_date
2	exp_srm_daily	experiment × as_of_date	exp_exposures, registry	Cumulative exposed users per arm vs design; χ^2; flag at $p < 0.001$	Site: experiment cards; analysis gate	experiment_id, as_of_date
3	exp_invariants	experiment × check × stratum	exp_exposures, Silver order and storefront events	CHK: risk-band decline parity. PDP/BUN: prices shown identical. Pre-period balance.	Analysis	experiment_id
4	exp_user_metrics_chk	user (CHK exposed)	exp_exposures, silver.order_events, aftersales	First checkout paid within 30 min, method, latency, fraud flag, matured-chargeback flag, revenue	Analysis	variant, first_exposure_date
5	exp_user_metrics_pdp	user (PDP exposed)	exp_exposures, storefront, order, aftersales	Purchase within 24 h, add-to-cart, refund within 7 days, net purchase	Analysis	variant, first_exposure_date
6	exp_user_metrics_bun	user (BUN exposed)	exp_exposures, order, aftersales, pre-period revenue	7-day net revenue, any purchase, refunds, books per order, CUPED covariate	Analysis	variant, first_exposure_date
7	exp_guardrails_daily	experiment × variant × date × metric	exp_user_metrics_*, silver.ops_alerts	Guardrail values plus monitor alert counts	Site: guardrail charts	experiment_id, metric_date
8	exp_portfolio	experiment	exp_srm_daily, exp_guardrails_daily, registry, experiment_results	Status, days running, n vs required, SRM, latest decision	Site: portfolio page	experiment_id
9	mkt_daily_kpis	date	storefront, order, aftersales events	Visitors, buyers, orders, GMV, net revenue, refund rate	Site: business page	date_bogota
10	mkt_funnel_daily	date × step × device	storefront, order events	Product view → cart → checkout → paid	Site: funnel chart	date_bogota, step
11	mkt_refunds_daily	date × reason	aftersales events	Refunds, rate, top reasons	Site; PDP memo	date_bogota
12	pay_method_daily	date × method	order events	Share, success rate, p95 latency per payment method	Site: payments panel	date_bogota, method
13	pay_risk_daily	date × method	order, aftersales events	Fraud flags, matured	Site: payments panel; risk	date_bogota, method

				chargebacks, amounts		
14	cat_top_books_weekly	week × book	silver.books, storefront, order events	Views, buyers, conversion, refunds per real title	Site: bestsellers; perf-lab skew check	week_start, book_id
15	real_headline_tests	test	silver.real_headline_packages	Packages, impressions, CTR range, best-vs-reference log-OR and SE	Site: real lessons	test_week
16	real_headline_effects	test × package	silver.real_headline_packages	Per-package log-OR vs reference, SE, z	Analysis (prior fit); site histogram	test_week
17	ops_dq_summary	run × check	ops.dq_reconciliation, ops.validation_results	Pass/fail per check	Site: data-health badge	run_date
18	ops_data_freshness	dataset	ops.run_log, Silver/Gold max timestamps	Last update and lag per dataset	Site footer: “data as of”	dataset
19	duel_results	Duel	silver.duels, silver.duel_votes	Readers and yes per version, rates, SRM p, the gap and its 95% range, the ending once over (D34); votes the bot filter dropped	`13_community_` ; the `summary` check against the API’s reveal	ended_date
20	duel_library	genre × book language × what changed	duel_results (ended, fair, added to the library)	Duels per group, wins by version type, the typical gap, idea tags	`13_community_` → `summary` `library.json`	genre, book_language

8. Pre-registration drafts

These are frozen as `docs/prereg/*.md` with tag `prereg-v1` before the full backfill. The simulator's truth block is deliberately not referenced here. Nothing in this section depends on the cloud; it is unchanged from v3.0.

Common rules: randomization unit `user_id`, by deterministic salted hash, 50/50; exposure-based analysis (metric windows start at first exposure); SRM gate χ^2 $p < 0.001$ invalidates the readout; an A/A sanity check via EXP-AA-000; fixed horizon, one analysis after the cutoff; seed 20260706; no price differences between arms.

Field	EXP-CHK-001 Wallet-first checkout	EXP-PDP-002 New cover + subtitle	EXP-BUN-003 3-book bundle banner
Hypothesis	Showing local wallets and PSE first, with one tap for saved wallets, raises checkout completion without more fraud, more chargebacks or slower checkout. The risk engine is identical across arms.	Redesigned cover thumbnails and benefit-led subtitles for the 200 most-viewed titles raise product-page conversion without raising refunds.	A "pick any 3 books, save 25%" banner raises net revenue per visitor without raising refunds or lowering purchase incidence.
Trigger / exposure	Checkout opened	First view of a test-set product page	Bundle-slot evaluation on a product or cart page (control logs the counterfactual)
Primary metric	Share of exposed users whose first post-exposure checkout ends paid within 30 min	Share of exposed users who buy the viewed title within 24 h	Net revenue (COP, net of refunds) per exposed user within 7 days. Welch + bootstrap CI; CUPED on pre-period revenue is the pre-registered estimator.
Secondary (Holm)	Payment-method mix; order-level success ratio (delta method); net revenue per exposed user	Add-to-cart within 24 h; net purchase rate	Orders per user; books per order; bundle take-up (descriptive)
Guardrails	Fraud-flag rate on paid orders, NI margin +0.50 pp (power 0.88). Chargeback rate on orders ≥ 45 days old, NI +0.50 pp (power 0.98). p95 checkout latency, NI +100 ms (cluster bootstrap).	Refund within 7 days per exposed user, NI margin +0.25 pp (one-sided $\alpha = 0.05$; power ≈ 1.00)	Refund within 7 days per exposed user, NI +0.25 pp. Any-purchase rate, NI −1.0 pp (power 0.98).
Invariants	Decline rate by risk band equal across arms (χ^2 per band, Holm); pre-period checkout starts balanced	Same price shown in both arms; pre-period product views balanced	Same book prices in both arms; pre-period revenue balanced
Baseline / MDE	70% / +3.0 pp	4.0% / +0.6 pp (+15% rel.)	COP 2,400 / +15% (COP 360)
α / power / n per arm	0.05 two-sided / 0.80 / 3,554	0.05 / 0.80 / 17,943	0.05 / 0.80 / 8,373 ($\rho = 0.5$; 11,163 without CUPED)
Duration	Enrollment 2026-07-06 → 08-02. Single final readout after 09-21, when chargebacks mature.	Same enrollment; cutoff 08-10	Same enrollment; cutoff 08-17
CUPED covariate	Pre-period revenue (secondary)	Pre-period purchases	Pre-period revenue
Decision rule	Ship if SRM is OK, invariants hold, primary $p < 0.05$ with lift > 0, and all guardrails are non-inferior.	Ship if SRM is OK, primary $p < 0.05$ with lift > 0, and the refund guardrail is non-inferior. Else don't ship / iterate.	Ship if SRM is OK, primary $p < 0.05$ with lift > 0, and both guardrails are non-inferior.
Threats	Novelty; payment-provider outages; fraud is rare	"Clickbait" covers raise refunds; covers also appear in search lists (randomized by user)	Cannibalization of single-book sales (captured by net revenue); discount anchoring

Duel pre-registration (D34). Every Duel follows the same rules, fixed in code before it starts and shown on the Methods page.

Field	Rule
Question	"Would you read the first page?" One tap: yes or no.

Unit and split	One reader: one vote per visitor per Duel. The browser picks version A or B, 50/50, from a hash of the Duel and a random number that never leaves the device.
Primary metric	The share of readers who answer yes, per version.
Target	330 readers in total, or 14 days, whichever comes first. 330 readers detect 30% → 45% at 95% confidence and 80% power.
No peeking	Progress shows readers so far and the split check, never the answers. The server opens the result only at the end.
Split check (SRM)	χ^2 $p < 0.001$ gives “Not fair yet”.
Decision	A gap under 5 points gives “Too close to call: both work”. Otherwise a two-proportion z-test at 95%: $p < 0.05$ gives “Readers chose version X”; else “Not sure yet: about N more readers”.
Not counted	The writer’s own device, and votes the Silver bot filter drops. Gold’s `duel_results` recomputes the ending after the filter, and the library uses Gold’s.
What it doesn’t measure	Sales. A Duel measures interest in the first page, in a fair test (R40).
Threats	The writer’s friends vote (R41); a small following never reaches 330 readers (R42).

9. Assumptions and open questions

Assumptions

- A1 Everything runs in `us-east-1` (D38).
- A3 You can run sudo in WSL for Docker (local Kafka and DynamoDB Local).
- A4 Your laptop is not needed for the daily run or the backfill; both run in GitHub Actions, AWS and Databricks.
- A5 The out-of-pocket bill stays at $0.00 in year 1 and at a few cents a month after. Measured from Cost Explorer at each phase (O31).
- A6 Free Edition's daily quota, which Databricks doesn't publish as a number, fits one daily run of our size (measured in Phase 3, O35).
- A7 `dev` and `prod` share one bucket with separate prefixes, and use separate catalogs in Databricks.
- A8 A 4-week fixed horizon is acceptable.
- A9 WSL can spare about 2 GB for local Kafka, and 4 GB for local PySpark unit tests.
- A10 Payment-method shares, success rates, refund and chargeback rates are invented but plausible, documented in `simulation.yml`.
- A11 Writers will share results if sharing is anonymous, takes one click, and shows exactly what is sent. Unproven until real writers use the site (O18).
- A12 A writer can bring a Duel to 330 readers within 14 days with one shared link plus the game's credits. Unproven until real Duels run (O21).
- A13 You upgrade to the paid plan at S1.1 and add a card; the design keeps charges at $0 out of pocket while the credits last (D53, D54).

Retired: A2 (Aiven throughput).

Open questions (each resolved by a named probe). Back from v3.0 with the hybrid: O2, O3, O6, O7, O8. Retired in v4.0: O1, O4, O5, O10, O11, O13, O14 (Aiven and Path A), O16–O17 (Cloudflare and D1), O24 and O29 (Spark on GitHub runners).

ID	Question	Resolved by
O2	Can we create catalogs, groups for grants, secret scopes, and service principals with OAuth on Free Edition?	P2, P3, Phase 0
O3	Does "one active pipeline per type" block creating a second pipeline, or only running it at the same time?	P5, Phase 0
O6	Is PyPI reachable from serverless notebooks (scipy, statsmodels pinned)?	P6, Phase 0
O7	Does `dbutils.secrets.get` work inside the pipeline without leaking the pepper?	Phase 4
O8	Do column masks work on AUTO CDC SCD2 targets and on streaming tables?	P7, Phase 4
O9	Are the synthetic 99… document and +57 399… phone ranges unassigned?	Phase 2
O12	What do Ley 1480 de 2011 (retracto, truthful information) and its exceptions say for digital content?	Phase 2
O15	How large are headline-click effects compared with purchase effects?	Phase 6 sensitivity analysis; stated as a limitation
O18	How many writers will share results, and how fast?	After launch; the public counter
O19	Does Meta's A/B test feature recommend a minimum test length?	Phase 7 start (D29 stays at 7 days either way)
O20	What are the metric names in the Spanish versions of Meta Ads Manager, Amazon Ads and TikTok Ads Manager?	Phase 7
O21	How many readers does a typical Duel reach in 14 days, and how many votes does the game add?	Phase 1 launch logs (counts only)
O22	Which columns do Amazon Ads search-term reports use in English and Spanish?	Phase 7 start
O23	Does the daily-salted visitor hash need a notice under Ley 1581?	Phase 1: the official text; the Methods page explains the hash either way
O25	Can the S3 credit of the CloudFront Free plan be seen on the bill, and does it cover covers and lake together?	S1.1 and the first paid-plan bill

O26	For Ley 1581's international-transfer rules, is the United States treated as adequate, or does the notice need consent wording?	Phase 1, before S1.1
O27	Which sign-up experience does the new account get, does it ask for a card, and does the console allow every service in P1?	S0.2 and P1
O28	Does the CloudFront flat-rate Free plan accept a Lambda function URL origin with origin access control, and a WAF rate rule on `/v1/*`?	P9, at S1.1
O31	How many S3 requests does a daily run make, and what does a month really cost?	P12 in Phase 0; `ops.cost_log` from Phase 8
O33	How many state transitions does one Duel's lifecycle really use?	P11, Phase 0; the console count in Phase 1
O34	Does Free Edition allow service principals with workload identity federation for GitHub Actions?	P3, Phase 0
O35	How much of the daily quota does the daily run use, and does the 92-day backfill fit in one day?	P5 in Phase 0; Phase 3

Appendix A. Repo tree (target)

```
D:\abtesting (github.com/Pipe199x/ebook-ab-platform, public, MIT)
├── README.md LICENSE pyproject.toml uv.lock .python-version .env.example .gitignore .pre-commit-config.yaml
├── databricks.yml    the bundle: targets dev (ebook_ab_dev) and prod (ebook_ab)
├── resources/{ebook_pipeline.pipeline.yml, ebook_daily.job.yml, schemas.yml, volumes.yml, grants.yml}
├── pipelines/ebook/{bronze.py, silver.py, gold_factory.py, quarantine.py}
├── .github/workflows/{ci.yml, deploy.yml, site.yml, probes.yml, backfill.yml, daily-e2e.yml, weekly-maintenance.yml}
├── config/{topics.yml, catalog.yml, simulation.yml, experiments.yml, gold_views.yml, quality/*.yml}
├── contracts/{envelope.v1|v2, storefront/*, orders/*, aftersales/*, experiments/*, ops/*, site/*,
│             community/result, duels/*}.schema.json
├── src/ebook_ab/
│   ├── common/       config, assignment hash, kafka config, aws, databricks client, logging
│   ├── catalog/      Gutenberg RDF parser, title selection, price model
│   ├── simulator/    population, sessions, payments, risk, refunds, effects, faults, cli (backfill|live)
│   ├── bridge/       consumer loop, reconcile-on-assign, writers (S3 | local dir)
│   ├── monitor/      guardrail windows, alerts, DLQ, lag
│   ├── transforms/   pure PySpark: parse, route, explode, hash, mask, dedup keys
│   ├── quality/      YAML rules → expectations: fail / drop + quarantine / warn
│   ├── gold/         YAML spec model, SQL builder, data dictionary
│   ├── stats/        ztest, welch, bootstrap, cuped, delta, multiplicity, srm, power, bayes, meta
│   ├── realdata/     Upworthy + Gutenberg loaders and checks
│   ├── site_export/  Gold (Statement Execution API) → JSON (schema-validated), including library.json
│   └── tools/        topics, groups, probes, sync_volume, export_runs, moderate, cost
├── jobs/perf_lab.py  the Glue Lab job
├── lambdas/{export_edge/, salt_rotate/, close_duel/, delete_covers/}
├── notebooks/{10_ab_analysis, 11_validation, 12_real_headlines, 13_community_summary, 20_dq_report,
│              40_delta_ops}.ipynb
├── edge/             the Duel service: src/{handler.ts, duels.ts, votes.ts, game.ts, results.ts, admin.ts, sha.ts},
│                     package.json, test/
├── infra/            AWS CDK app (Python): app.py, stacks/{foundation, edge, site, ci, lab}.py,
│                     statemachines/duel_lifecycle.asl.json
├── sql/verify/phase0..10.sql
├── web/              Vite + React + TypeScript: src/{pages, components, stats, i18n}, public/{data/, fonts/}, tests/
├── tests/{unit, contract, integration, stats, transforms, quality, edge, vectors/, fixtures/ads/}
├── docker/compose.yml (Kafka; DynamoDB Local)
├── perf/spark_ui_lab/{generate_data.py, README.md}
├── tools/{gen_data_dictionary.py, check_free_allowlist.py, make_plan_md.py, render_docs.sh, build_companion.py,
│          check_demo.cjs}
└── docs/{plan/(README.md, PLAN.md, PLAN.pdf, DEMO.html, render/, diagrams/, charts/, shots/, answers/, old/),
          adr/ADR-001..011, prereg/, memos/, reports/, examples/, launch/founding-writers.md,
          data_dictionary.md, runbook.md, interview.md, evidence/phase0..10/, perf/}
```

ADRs:

- 001 incremental ingestion: S3 → Files API sync → volume → Auto Loader, vs reading S3 directly (not possible on Free Edition) vs Kafka read by Spark (Path A);
- 002 Kafka off AWS: GitHub Actions and Docker vs Fargate per run vs MSK vs Kinesis (with Phase 0 evidence);
- 003 where Spark runs: Databricks Free Edition ($0) vs Spark on a GitHub runner vs Glue vs paid Databricks; what would move and when;
- 004 JSON contracts vs Avro;
- 005 the $0 architecture on two platforms: free allowances only, the allow-list, the budgets, Free plan → paid at S1.1; Unity Catalog for data, IAM for AWS;
- 006 Delta Lake as Unity Catalog managed tables vs Iceberg; S3 as the replay log (D62);
- 007 static site on S3 + CloudFront + a daily JSON export vs a live API;
- 008 real data: sources, licenses, and why the simulation stays;
- 009 anonymous Duels: Lambda function URLs + DynamoDB + Step Functions vs API Gateway; privacy, moderation and anti-abuse; why votes reach the lakehouse as files (D30);
- 010 infrastructure as code: CDK + Declarative Automation Bundles vs Terraform for both;
- 011 data quality: pipeline expectations vs Great Expectations vs Glue Data Quality.

Appendix B. Deliverables → phase

Deliverable	Where	Phase
0 PLAN.pdf + DEMO.html + README.md	docs/plan	post-approval (now v4.0)
1 Diagrams + ADR-001…011	docs/plan/diagrams, docs/adr	0 (002, 005, 006, 010), 1 (007, 009), 3 (001, 003), 4 (004, 011), 6 (008); finalized in 10
2 Repo tree	Appendix A, materialized from Phase 0	0–2
3 Code, file by file	src/, pipelines/, resources/, lambdas/, notebooks/, edge/, infra/, web/	1–9
4 Gold catalog	§7, then generated into docs/data_dictionary.md	5
5 Runbook	docs/runbook.md, grown each phase	0–10
6 The live site and the Duel service	CloudFront; Lambda + DynamoDB + Step Functions	1 (the Duel, Real or luck?, home), 7 (the rest)
7 README and interview answers	README.md, docs/interview.md (§3.0 with measured numbers)	10

Appendix C. Changelog

v3.0 → v4.0 (2026-10-19)

- Round 7 (§2.10): your decisions (AWS; serverless only; $0) and answers Q17–Q23: Delta tables (your decision 4); Kafka stays with the simulated store and Duel votes stay files (Q22); Kafka in GitHub Actions; the Duel service and site move to AWS; a new AWS account; the AWS always-free design; and the hybrid (Q23): Databricks Free Edition keeps the lakehouse.
- The platform stays on Databricks Free Edition: one triggered Lakeflow pipeline (Auto Loader, expectations, AUTO CDC, 20 materialized views), Unity Catalog (grants, masks, tags, lineage), the `ebook_daily` Job with notebook tasks. New: Auto Loader reads a Unity Catalog volume that a sync step fills from S3, because Free Edition can't be assumed to reach your bucket (D41, D62); GitHub Actions triggers the Job by API (A14); `site_export` reads Gold through the Statement Execution API; executed notebooks are exported to S3 (D51).
- S3 becomes the durable landing zone and replay log outside Databricks (D62).
- Kafka: Aiven and Path A removed; Kafka runs in Docker and in GitHub Actions only; the backfill becomes a manual workflow.
- Edge and site: Lambda function URLs behind CloudFront + DynamoDB (provisioned, free) + S3 covers + a Step Functions lifecycle per Duel replace the Cloudflare Worker + D1 and its cleanup cron; S3 + CloudFront's flat-rate Free plan replace GitHub Pages; covers up to 300 KB instead of 60 KB; admin signed with IAM; a daily-destroyed salt for the visitor hash.
- Infrastructure as code twice: AWS CDK for AWS, bundles for Databricks; GitHub OIDC for AWS (no keys).
- Serverless only: the local Docker Spark cluster, the job-cluster target and the plain Structured Streaming track removed; the Spark UI lab moves to AWS Glue (credits); Delta maintenance stays Lab.
- §3.0 rewritten as interview answers: the formula, five common questions, a card per technology.
- Section G: "$0 by design, with one honest footnote": S3 requests (cents), paid by credits in year 1; Free Edition can't bill; the upgrade at S1.1; the allow-list and budgets.
- New decisions D38–D62 (D42 retired); superseded D5, D23; amended D2, D3, D9, D10, D11, D14, D15, D19, D25, D31, D35, D36.
- Risks: new R45–R47, R50, R51, R53, R55–R57, R59–R62, R64–R66; back from v3.0: R2, R6–R9, R20; retired R1, R3–R5, R23–R25. Open questions: new O25–O28, O31, O33–O35; back from v3.0: O2, O3, O6–O8; retired O1, O4, O5, O10, O11, O13, O14, O16, O17.
- Phases: same numbering; 230 h (192–268), 12–14 weeks; Phase 0 covers both platforms; Phase 1 includes the paid-plan upgrade; Phase 9 is a Lab; Phase 10 adds interview prep.
- Pictures: new diagrams for the big picture, the hybrid map, both journeys, ingestion, the commit protocol, the lakehouse, the daily run and the edge; the phone screens and the traffic chart are unchanged.

Earlier versions (details in `docs/plan/old/`): v2.3 → v3.0 (2026-10-17), the lifesaver approach L1–L9 and L11, For nerds, the real-use table, D30–D37; v2.2 → v2.3 (2026-10-15), C1–C12; v2.1 → v2.2, the community library and opt-in sharing; v2.0 → v2.1, the writer-first ad-test tool; v1 → v2.0, the ebook marketplace, real data, the React site, the laptop-free daily run.

Verification (end-to-end)

At the end of Phase 8, one `daily-e2e.yml` run must chain these steps, and `ops.validation_results` must be all PASS:

1. simulate a day (CI Kafka);
2. Kafka → bridge → `landing/` in S3 (OIDC role `gha-pipeline`);
3. `export_edge`: the day's news in DynamoDB as files (D30): Duels and votes → `landing_real/duels/`, shared results and withdrawals → `landing_real/community/`;
4. the sync: only the new files → the volume, counts checked (D62);
5. the `ebook_daily` Job: the pipeline update, Bronze → Silver (expectations and reconciliation passing) → 20 Gold views; then the notebooks: the DQ gate, analysis, the Duel endings, the library summary;
6. the executed notebooks exported to S3;
7. the site export through the Statement Execution API, including `library.json`;
8. CloudFront serves "data as of" today.

The backfill readouts must also give the 3 pre-registered decisions and pass proofs (a)–(c). CI must be green: Python unit, contract, stats, quality and Kafka-integration tests including the bridge kill test; local PySpark transform tests; `databricks bundle validate`; web tests including vector parity and the no-third-party-request check; edge tests; `cdk synth` with cdk-nag and the free-allowance allow-list. A test Duel's votes and a test share made on the live site must reach Silver in the next daily run, and the share must leave it after its withdrawal; a test Duel's lifecycle must delete its covers. The month's out-of-pocket cost must be $0.00, and the daily Databricks work must stay inside the quota. Phase 10's clean-clone runbook dry run is the final gate.